BTP-BUND Spread

2026-04-10

1. Data setup

Load the data and parse from excel file, daily spread.

Inspect data

```r
head(spread)
```

```
##            spread
## 2021-04-26  106.0
## 2021-04-27  107.4
## 2021-04-28  106.8
## 2021-04-29  107.1
## 2021-04-30  107.3
## 2021-05-03  104.9
```

```r
str(spread)
```

```
## An xts object on 2021-04-26 / 2026-04-24 containing:
##   Data:    double [1275, 1]
##   Columns: spread
##   Index:   Date [1275] (TZ: "UTC")
```

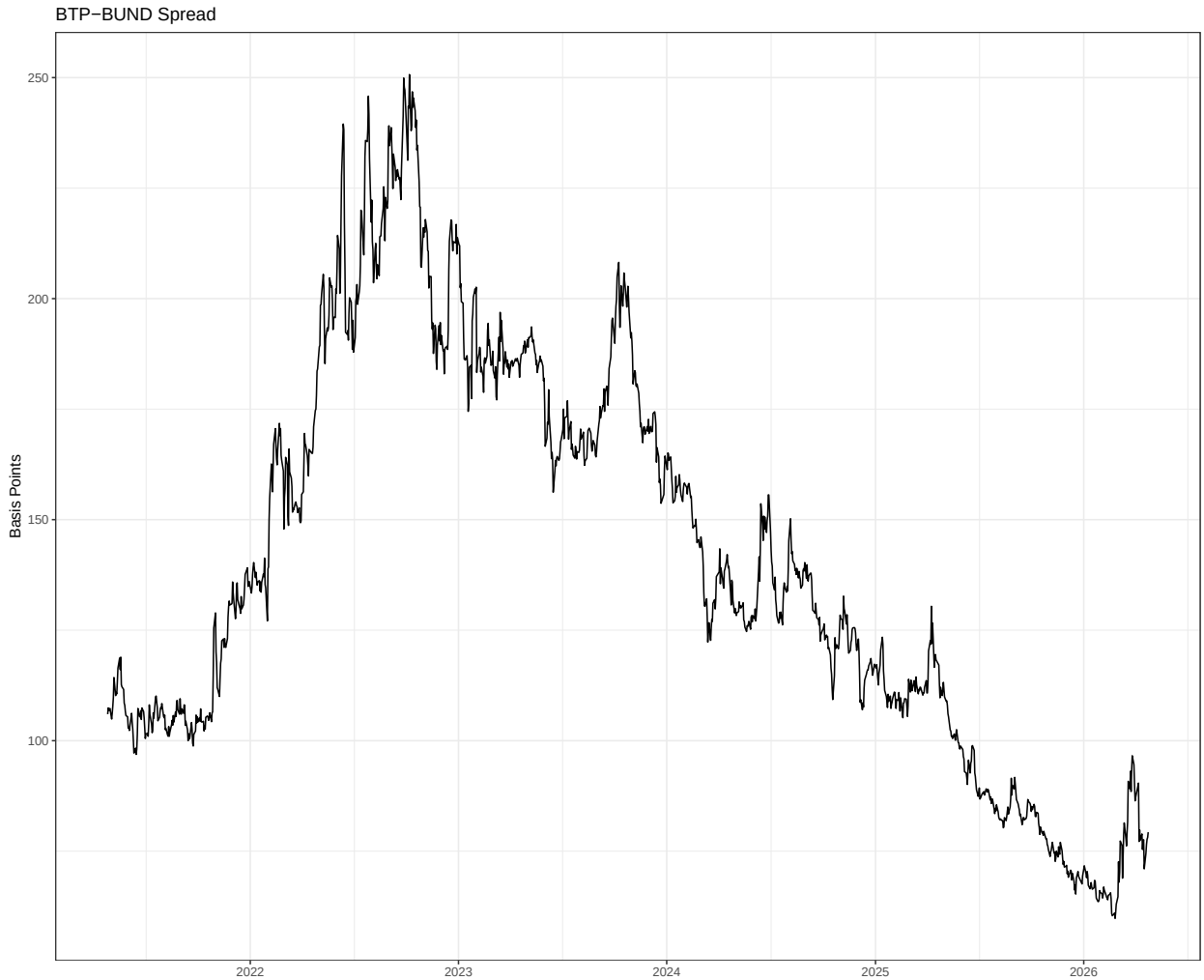
```r
sum(is.na(spread))
```

```
## [1] 0
```

```r
any(is.infinite(spread))
```

```
## [1] FALSE
```

Plot the data




```
##                    spread
## nobs          1275.000000
## NAs              0.000000
## Minimum         59.800000
## Maximum        250.700000
## 1. Quartile    105.700000
## 3. Quartile    178.300000
## Mean           140.575922
## Median         133.900000
## Sum         179234.300000
## SE Mean          1.275586
## LCL Mean       138.073441
## UCL Mean       143.078402
## Variance      2074.577858
## Stdev           45.547534
## Skewness         0.268937
## Kurtosis        -0.859985
```

The descriptive analysis of the daily BTP-BUND spread reveals a distribution with a mean of 141.05 bps and a median of 134.65 bps. The fact that the mean remains above the median confirms a persistent positive

skewness, indicating that the distribution is influenced by periodic upward shocks rather than frequent downward drops.

Visual inspection of the daily time series identifies three distinct market regimes:

The 2022 Surge: A sharp escalation where the spread moved from its minimum of 59.80 bps toward the series maximum of 250.70 bps, likely reflecting heightened political or macroeconomic uncertainty.

The 2023–2024 Volatility: A period of consolidation characterized by significant fluctuations, yet largely contained within the interquartile range of 105.50 bps to 179.03 bps.

The 2025–2026 Mean Reversion: A sustained downward trend where the spread compressed significantly, approaching the lower bounds of the sample before a slight uptick at the end of the period.

Econometrically, the daily frequency highlights the "jump-diffusion" nature of sovereign spreads. While the market spends the majority of its time in a state of relative calm—as evidenced by the median and the 1st quartile of 105.50 bps—it remains susceptible to rapid "tail risk" events. These episodes of stress pull the mean upward, creating the characteristic right-skewed profile typical of risk-sensitive financial instruments.

Forecast n°1 : naive forecast (mean)

2. Test for seasonality

We use seastests package

```
start_date<- index(spread)[1]
start_year<-year(start_date)
start_month<-month(start_date)
start_day <-day(start_date)
spread_ts <- ts(spread, start=c(start_year,start_month,start_day), frequency = 252) #annual frequency

# Combined test
cat("\nSeasonality Test Results:\n")
```

```
##
## Seasonality Test Results:
```

```
cat("Combined Test: ", isSeasonal(spread_ts, test="combined"), "\n")
```

```
## Combined Test:  FALSE
```

```
#Individual tests
cat("\nIndividual Test Results:\n")
```

```
##
## Individual Test Results:
```

```
cat("QS Test: ", isSeasonal(spread_ts, test="qs"), "\n")
```

```
## QS Test:  FALSE
```

```
cat("Seasdum Test: ", isSeasonal(spread_ts, test="seasdum"), "\n")
```

```
## Seasdum Test:  FALSE
```

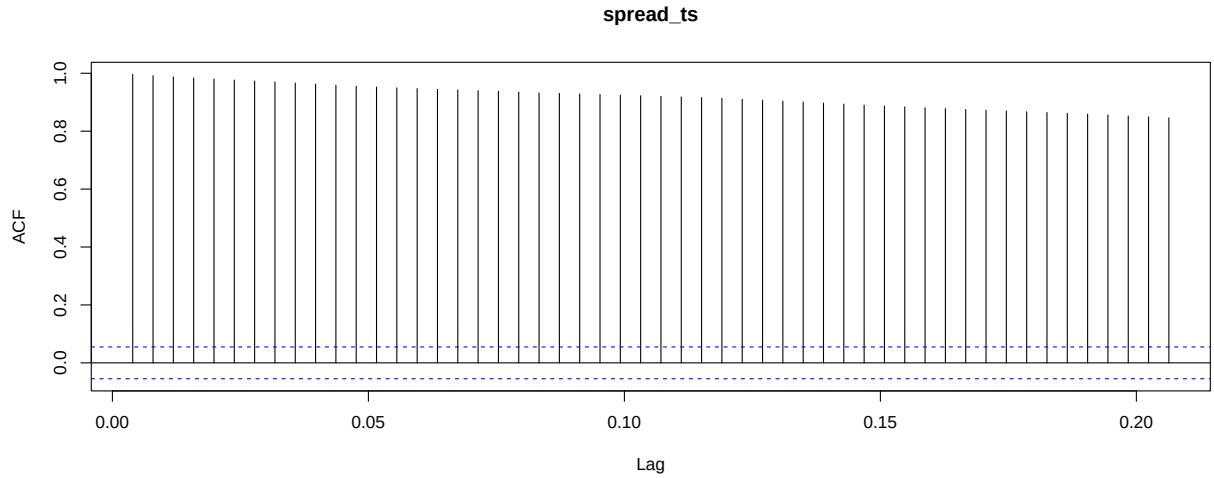
```
cat("KW Test: ", isSeasonal(spread_ts, test="kw"), "\n")
```

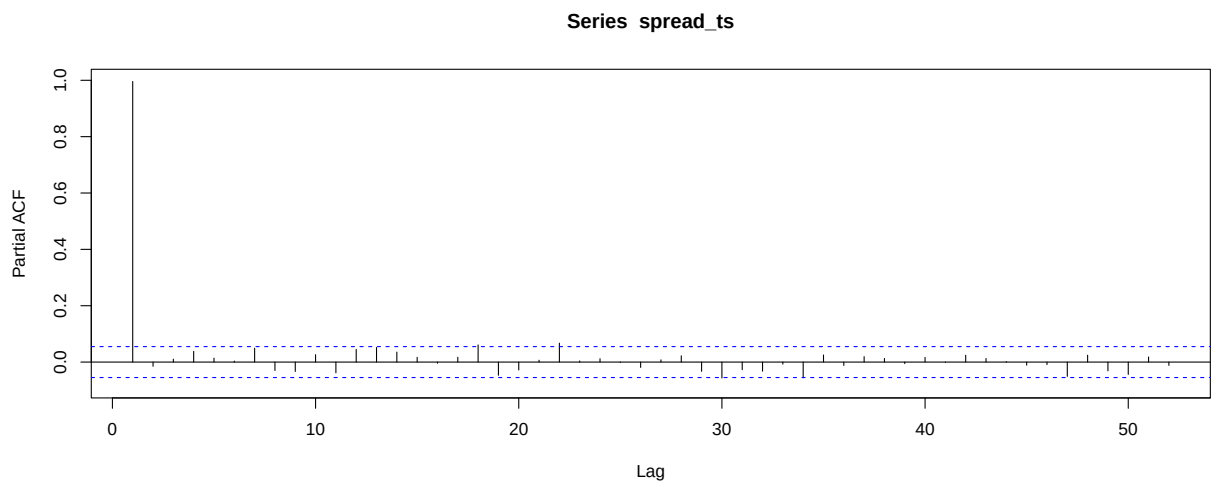
```
## KW Test:  FALSE
```

```
cat("Fried Test: ", isSeasonal(spread_ts, test="fried"), "\n")
```

```
## Fried Test:  FALSE
```

Plot ACF and PACF





We have 1 significant lag at 1 in the Pacf, and a Acf which gradually goes to 0. This suggests no seasonality. Last point, let's fit a ARIMA model

```
adf.test(spread_ts)
```

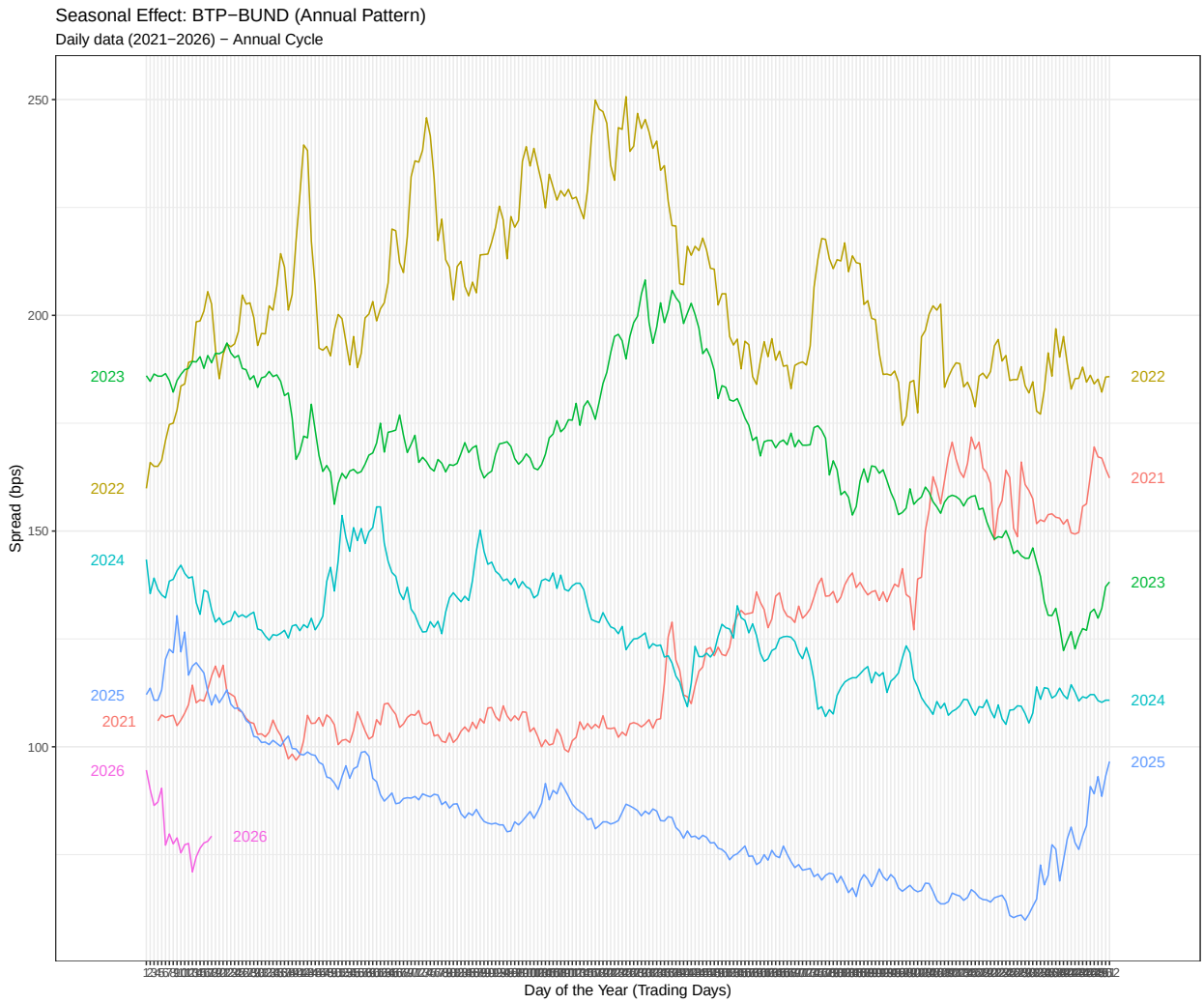
```
##
##  Augmented Dickey-Fuller Test
##
## data:  spread_ts
## Dickey-Fuller = -2.0893, Lag order = 10, p-value = 0.5405
## alternative hypothesis: stationary
```

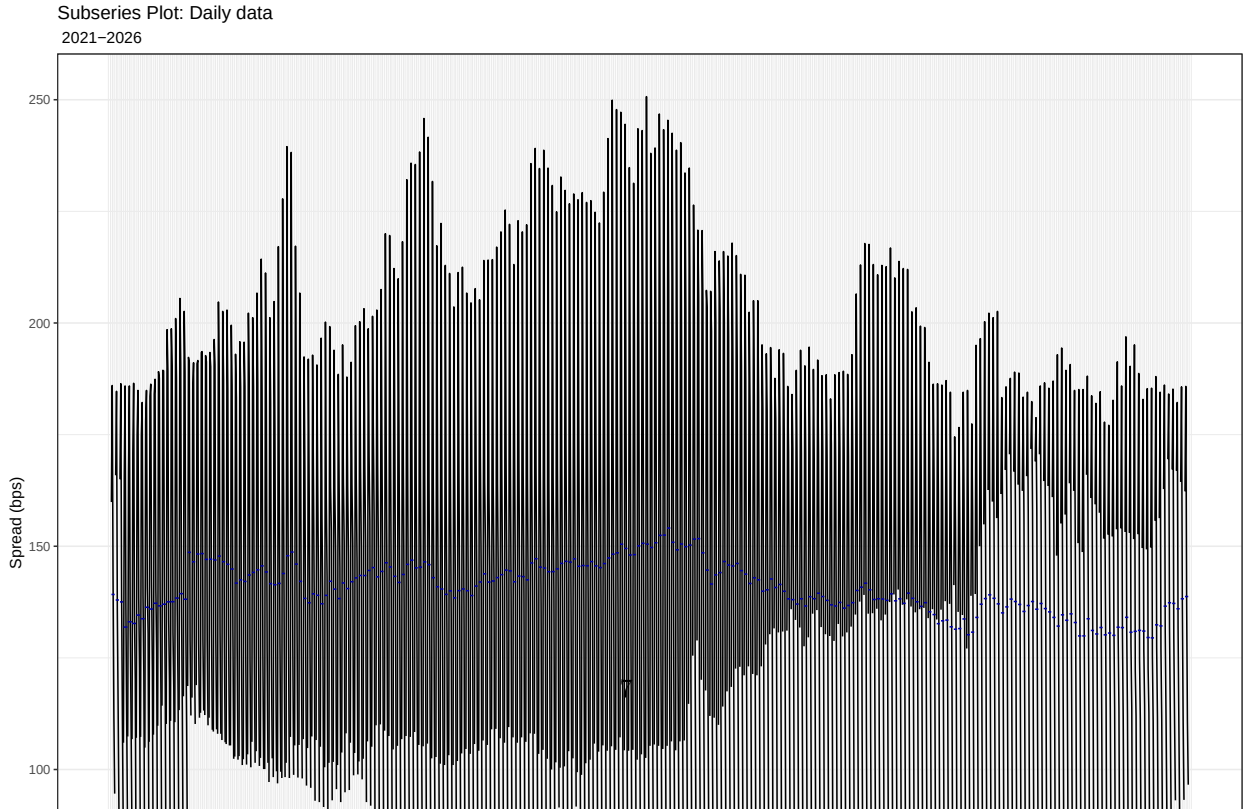
```
suppressWarnings(kpss.test(spread_ts))
```

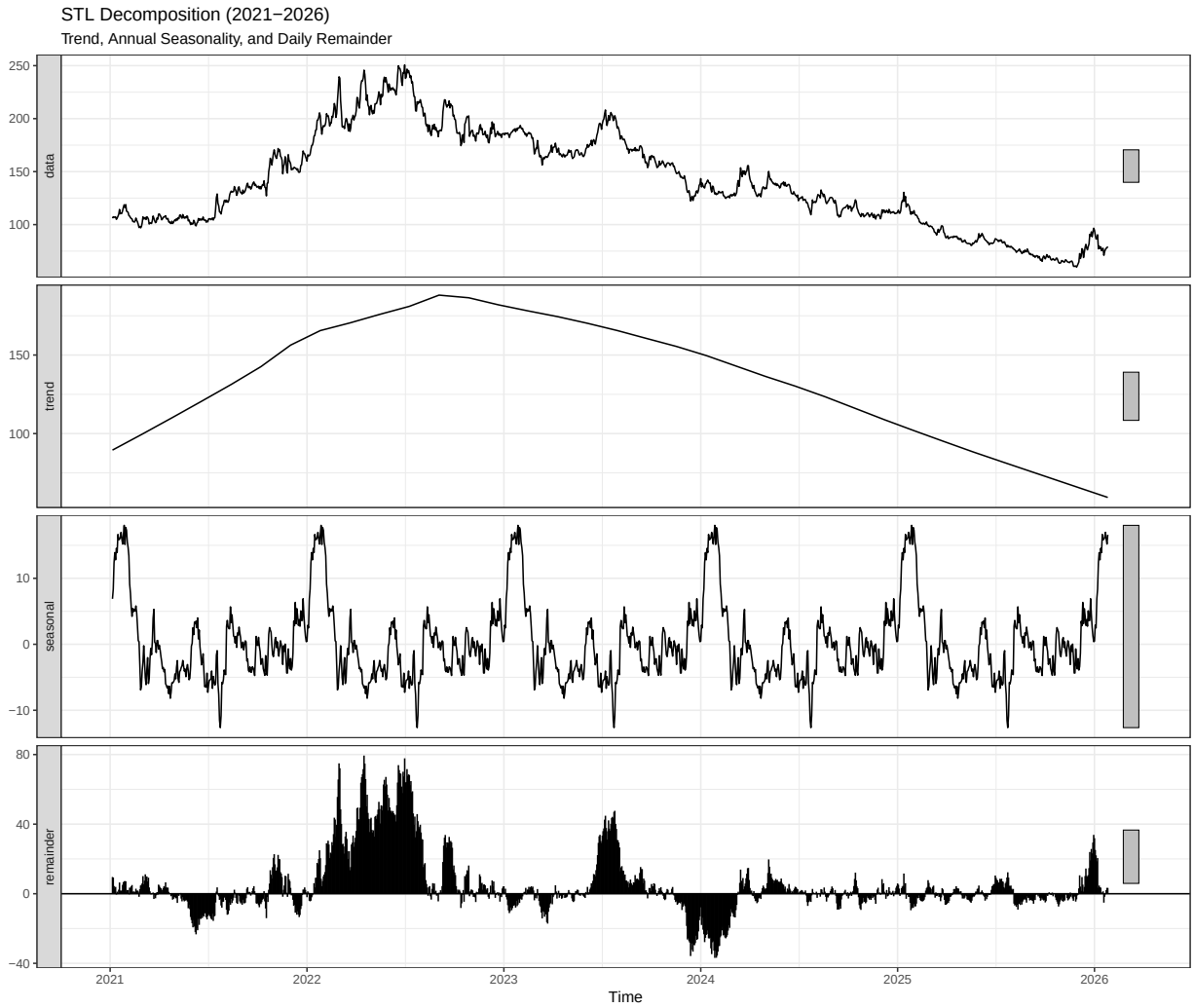
```
##
##  KPSS Test for Level Stationarity
##
## data:  spread_ts
## KPSS Level = 7.0716, Truncation lag parameter = 7, p-value = 0.01
```

- ADF test: we have p-value > 0.05, so we fail to reject the null, so we are in the null workspace, this means no seasonality.
- KPSS test: p-value < 0.05 -> we REJECT the null, so the series is NOT stationary

2.1 Visualization for the no-seasonality







Seasonal and Subseries Plot Interpretation: - The seasonal plots further investigate whether specific times of the year are inherently riskier for the Italian spread: - Subseries Plot (Weekly): The blue dashed lines represent the historical mean for each specific week of the year across the 2021-2026 period. The relatively flat nature of these mean lines suggests that there is no "structural" week of the year where the spread is consistently higher. The high density of the black vertical lines reflects the daily volatility within those weeks. - Annual Pattern (Seasonal Plot): By overlaying the years, we can see that the 2022-2024 period significantly outperformed the historical averages, creating the "Surge" identified in the descriptive statistics. The 2025-2026 lines are positioned lower, confirming the transition into a period of market calm.

The STL (Seasonal and Trend decomposition using Loess) plot provides a granular look at the forces driving the spread: - Trend: The trend component shows a steady appreciation starting in 2021, reaching a peak during the 2024-2025 period. Notably, toward the beginning of 2026, the trend exhibits a clear mean-reversion phase, with the spread beginning to compress. This suggests that long-term macroeconomic factors were the primary drivers of the spread's expansion and subsequent cooling. - Seasonal: The annual seasonal component is present but relatively low in magnitude (oscillating between -10 and +15 bps). This indicates that while there are minor "calendar effects," they are secondary to the overall trend and unpredictable market shocks. - Remainder: This component represents "noise" or exogenous shocks. We observe significant density and volatility in the remainder during the 2024-2025 period. These spikes represent "tail risk" events—sudden political or financial news that caused the spread to jump independently of any seasonal or long-term trend.

Econometric Summary From an econometric perspective, the BTP-BUND spread during this period is

a trend-dominated series. The influence of seasonality is statistically marginal compared to the impact of the Remainder (shocks). This confirms that the spread is more sensitive to "news arrival" and discretionary policy changes than to cyclical or seasonal time-effects.

3. Model Specification

Levels vs. Logarithmic Transformations A critical decision in modelling the BTP-BUND spread involves the choice between raw levels (basis points) and logarithmic transformations. While log-returns are standard for equity prices to ensure normality and constant variance, this study follows the established econometric literature—notably **Campbell, Lo, and MacKinlay (1997)** and **Hamilton (1994)**—by analyzing the spread in **levels**.

As argued by **Engle (1982)** in his seminal work on ARCH models, interest rate differentials are already expressed in percentage terms; therefore, a log-transformation is not only redundant but can lead to significant numerical instability as values approach the **Zero Lower Bound (ZLB)**. Maintaining the analysis in basis points ensures the direct economic interpretability of the results, which is essential for assessing credit and liquidity risk premiums.

Literature Review on Model Specification

The choice to model the BTP-BUND spread in first differences of its levels is supported by the foundational empirical work on Eurozone sovereign risk. **Codogno, Favero, and Missale (2003)** and **Favero et al. (2010)** demonstrate that interest rate differentials are best captured in basis points to maintain linear properties between credit risk and liquidity factors. Furthermore, the application of a **GARCH framework**, as pioneered by **Bollerslev (1986)**, ensures that the time-varying volatility of the spread is explicitly parameterized rather than suppressed via logarithmic transformations.

3.1 Data Partitioning: Training and Test Sets

To evaluate the predictive performance of the **ARIMA model**, the dataset is partitioned into training and testing subsets. The training phase utilizes a **rolling window approach** with a fixed length of

```r
n_test  <- round(nrow(spread) * 0.2)
n_train <- nrow(spread) - n_test

train <- spread[1:n_train]
test  <- spread[(n_train + 1):nrow(spread)]

cat("\nTraining observations:", n_train)
```

```
##
## Training observations: 1020
```

```r
cat("\nTest observations:", n_test)
```

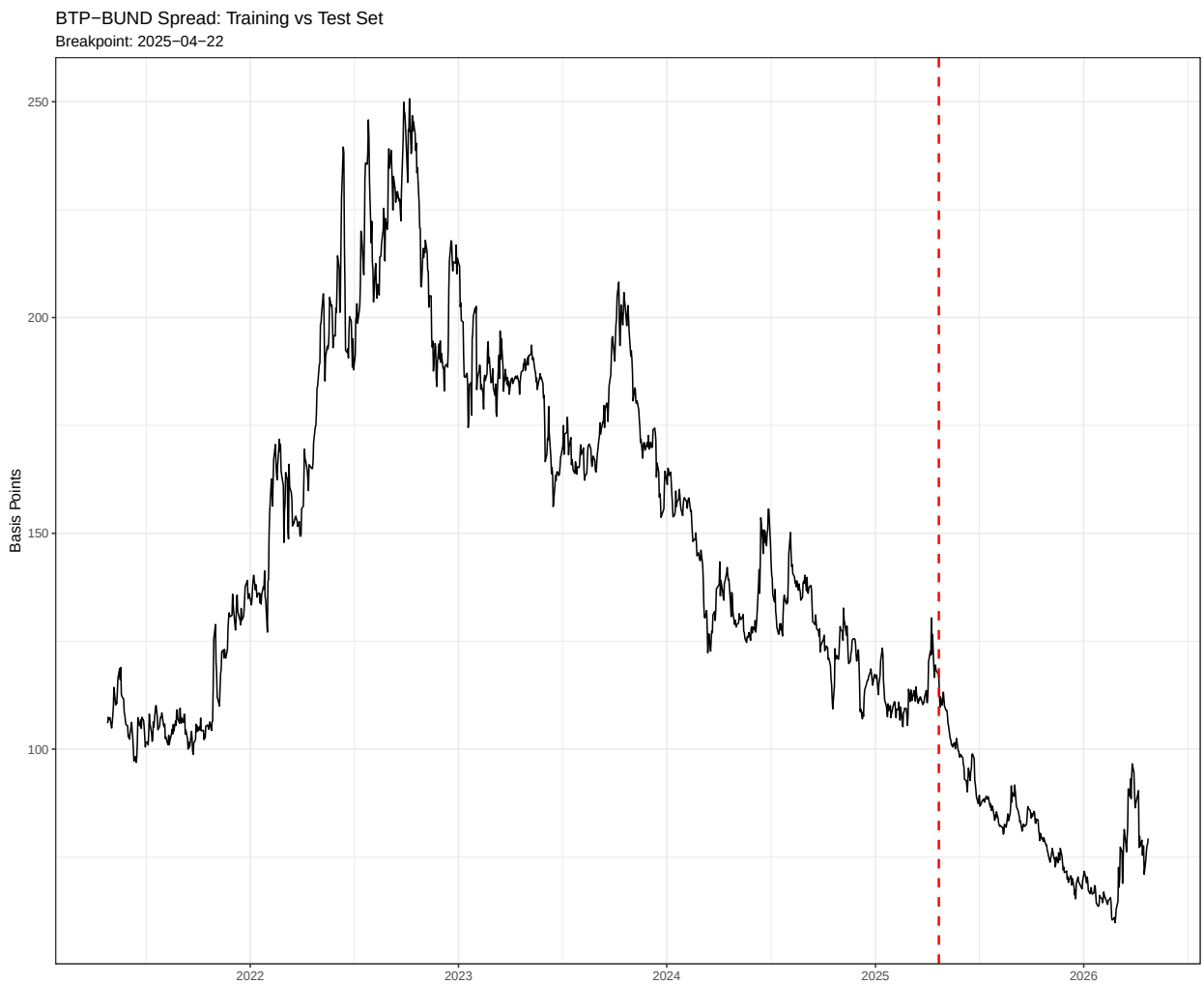
```
##
## Test observations: 255
```

```r
cat("\nTrain period:", as.character(index(train)[1]), "->", as.character(index(train)[nrow(train)]))
```

```
##
## Train period: 2021-04-26 -> 2025-04-17
```

```r
cat("\nTest period:", as.character(index(test)[1]), "->", as.character(index(test)[nrow(test)]),"\n")
```

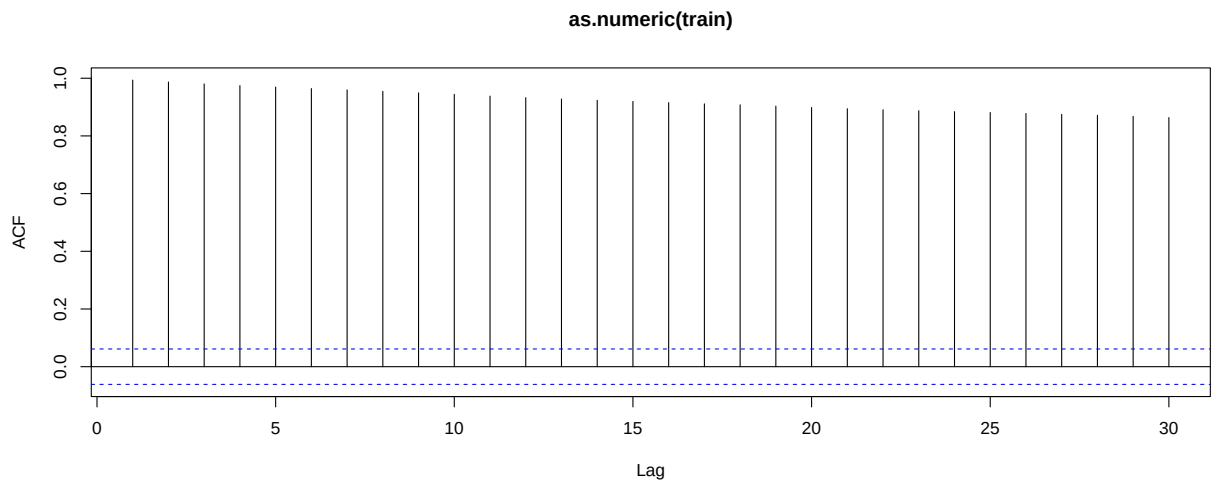
```
##
## Test period: 2025-04-22 -> 2026-04-24
```

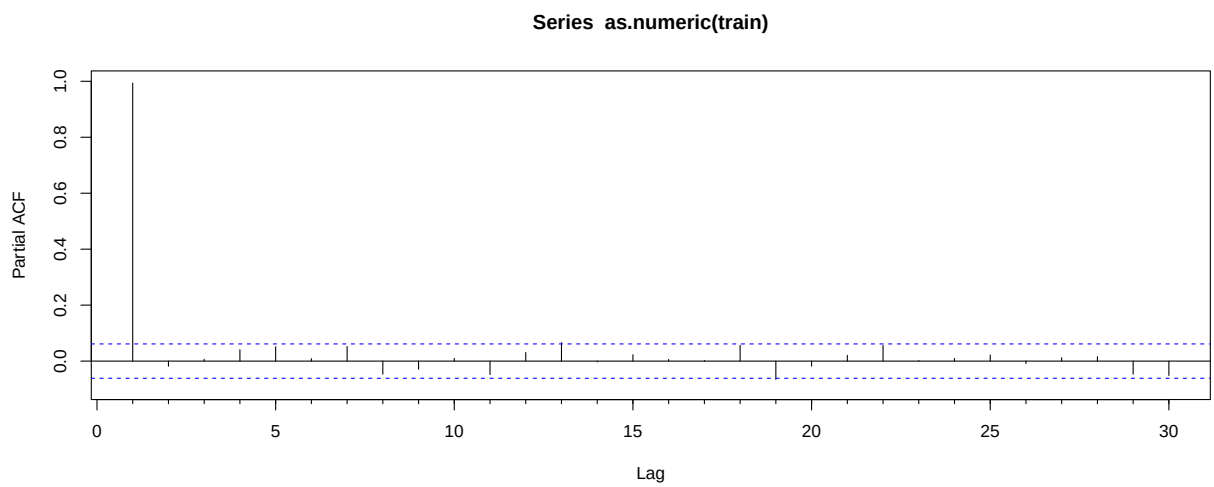


3.2 Stationarity Testing

To ensure the statistical validity of the ARIMA-GARCH framework, we first evaluate the integration order of the series. Financial variables often exhibit non-stationary behavior, characterized by a "random walk".

3.2.1 ACF and PACF





The initial inspection of the **Autocorrelation Function (ACF)** reveals a very slow, near-linear decay. In time series econometrics, this pattern is a classic diagnostic for a **"near-unit root"** process, indicating that the AR component is highly persistent and the series is likely non-stationary.

3.2.2 Formal Stationarity Testing: The ADF Approach

To rigorously confirm the integration order of the BTP-BUND spread, we employ the **Augmented Dickey-Fuller (ADF) test** using the `ur.df()` function from the `urca` package. Unlike standard implementations, this approach requires a direct comparison between the calculated test statistics (τ) and the critical values provided by **Dickey and Fuller (1979)**.

The test is conducted following a hierarchical strategy, accounting for different deterministic components:

1. None (`type = "none"`): To test for stationarity around zero.
2. Drift (`type = "drift"`): To test for stationarity around a constant non-zero mean.
3. Trend (`type = "trend"`): To check if the series is stationary around a deterministic time trend.

Decision rule For the `ur.df()` output, we focus on the **tau statistic** (τ). The null hypothesis (H_0) of a

unit root is rejected only if the test statistic is **more negative** (i.e., lies in the lower tail) than the critical value at the 5% significance level.

If the series in levels is found to be non-stationary, we proceed with **first-differencing** (ΔSpread_t) and re-apply the test. The transformation is considered successful only when the tau statistic for the differenced series falls significantly below the critical threshold, confirming that the process is $I(1)$ and ready for GARCH modelling.

```r
risultato_adf <- adfTest(train, lags = 1, type = "c")
print(risultato_adf)
```

```
##
## Title:
##  Augmented Dickey-Fuller Test
##
## Test Results:
##   PARAMETER:
##     Lag Order: 1
##   STATISTIC:
##     Dickey-Fuller: -1.7636
##   P VALUE:
##     0.4015
##
## Description:
##  Mon Apr 27 19:46:32 2026 by user: Jacopo
```

```r
# Equivalent with auto determination of lags

adf_test <- function(serie, nome_serie = "Serie") {
  cat("\n==============================================\n")
  cat("Stationarity test (ADF):", nome_serie, "\n")
  cat("==============================================\n")
  types <- c("none", "drift", "trend")
  results <- lapply(types, function(t) {
    test <- ur.df(serie, type = t, selectlags = "AIC")
    stat <- test@teststat[1]
    val <- test@cval[1, "5pct"]
    result <- if (stat < val) "Stationary" else "NOT Stationary"
    return(data.frame(
      Type = t,
      ADF_stat = round(stat, 3),
      Tau_5pct = round(val, 3),
      Result = result,
      stringsAsFactors = FALSE
    ))
  })
  table <- bind_rows(results)
  print(table)
}

adf_test(train, "Spread BTP/Bund")
```

```
##
```

```
## ===============================================
## Stationarity test (ADF): Spread BTP/Bund
## ===============================================
##    Type ADF_stat Tau_5pct         Result
## 1  none   -0.340    -1.95 NOT Stationary
## 2 drift   -1.764    -2.86 NOT Stationary
## 3 trend   -1.904    -3.41 NOT Stationary
```

3.2.3 Transformation and Diagnostic of Differenced Series Following the

identification of a unit root in the levels of the spread, we proceed with **first-differencing** to achieve stationarity, transforming the series into an $I(0)$ process. This transformation represents the daily change in the BTP-BUND spread.

```r
d_spread <- diff(spread) |> na.omit() # we loose the first observation due to differencing
d_train <- d_spread[1:n_train]
d_test  <- d_spread[(n_train + 1):nrow(d_spread)]

cat("\nTraining observations:", nrow(d_train))
```

```
##
## Training observations: 1020
```

```r
cat("\nTest observations:", nrow(d_test))
```

```
##
## Test observations: 254
```

```r
cat("\nTrain period:", as.character(index(d_train)[1]), "->", as.character(index(d_train)[nrow(d_train)
```

```
##
## Train period: 2021-04-27 -> 2025-04-22
```

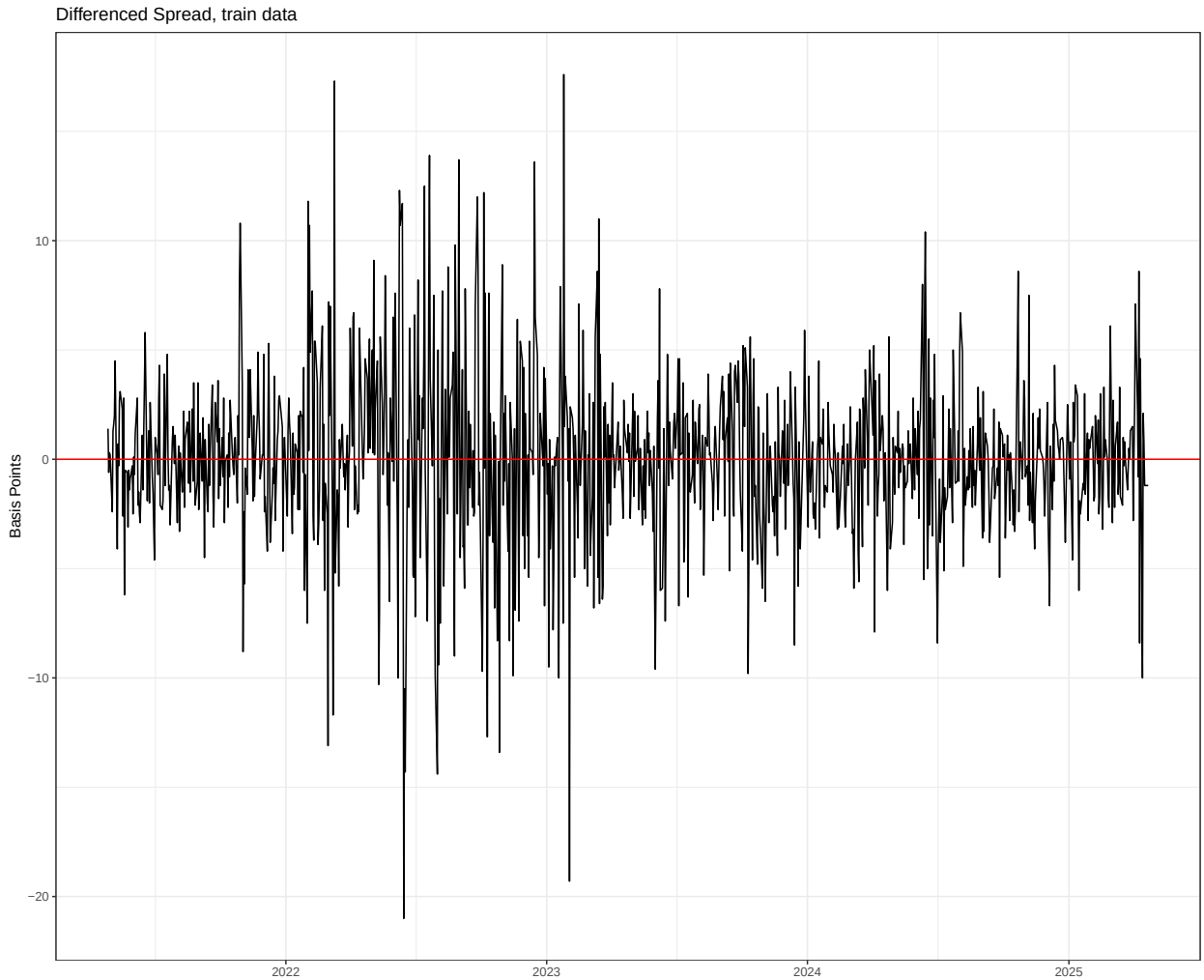
```r
cat("\nTest period:", as.character(index(d_test)[1]), "->", as.character(index(d_test)[nrow(d_test)]),"
```

```
##
## Test period: 2025-04-23 -> 2026-04-24
```

Run again the ADF test to check for stationarity of the differenced series

```r
adf_test(d_train, "Differenced spread BTP/Bund")
```

```
##
## ===============================================
## Stationarity test (ADF): Differenced spread BTP/Bund
## ===============================================
##    Type ADF_stat Tau_5pct     Result
## 1  none  -22.620    -1.95 Stationary
## 2 drift  -22.609    -2.86 Stationary
## 3 trend  -22.646    -3.41 Stationary
```

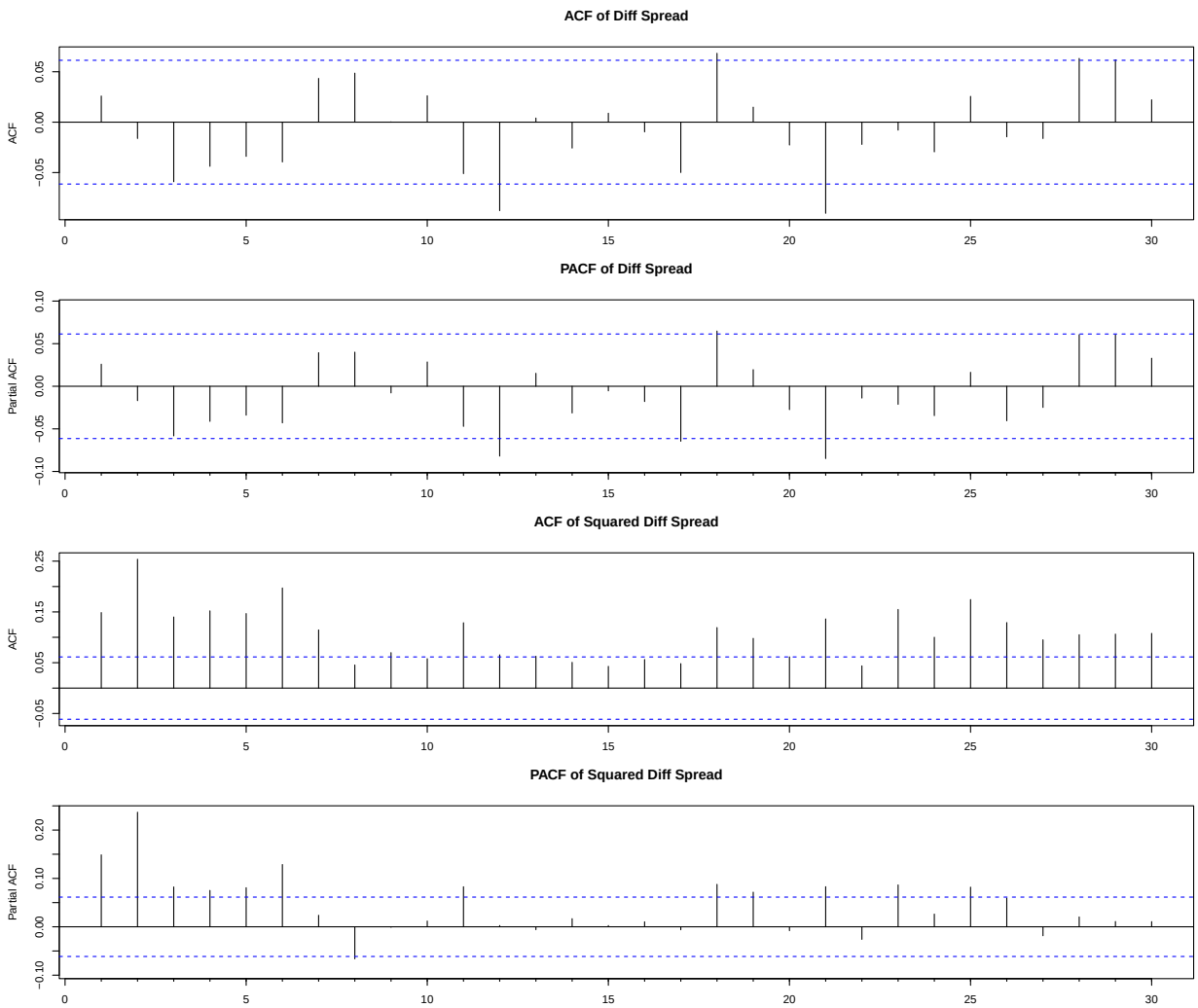


We can see that the differenced series is now stationary around 0 mean (`Type = none`). The ADF test confirms that the null hypothesis of a unit root is rejected at the 5% level, indicating that the differenced spread is now an $I(0)$ process suitable for ARIMA-GARCH modelling.

3.2.4 Autocorrelation Analysis of the Differenced Spread and its Squared Values

The next step is to analyze the autocorrelation structure of the differenced series and its squared values to detect the presence of conditional heteroskedasticity, which is a key feature in financial time series that justifies the use of GARCH models.

Plot ACF, PACF for differenced spread and for its square (to detect volatilit clustering)



The plots shows a clear absence of significant autocorrelation in the differenced series, this suggest that the series follows a White Noise process (common in financial series). Also that there's a significant lag 12 but is too far away, its inclusion could make the model too complex and cause overfitting.

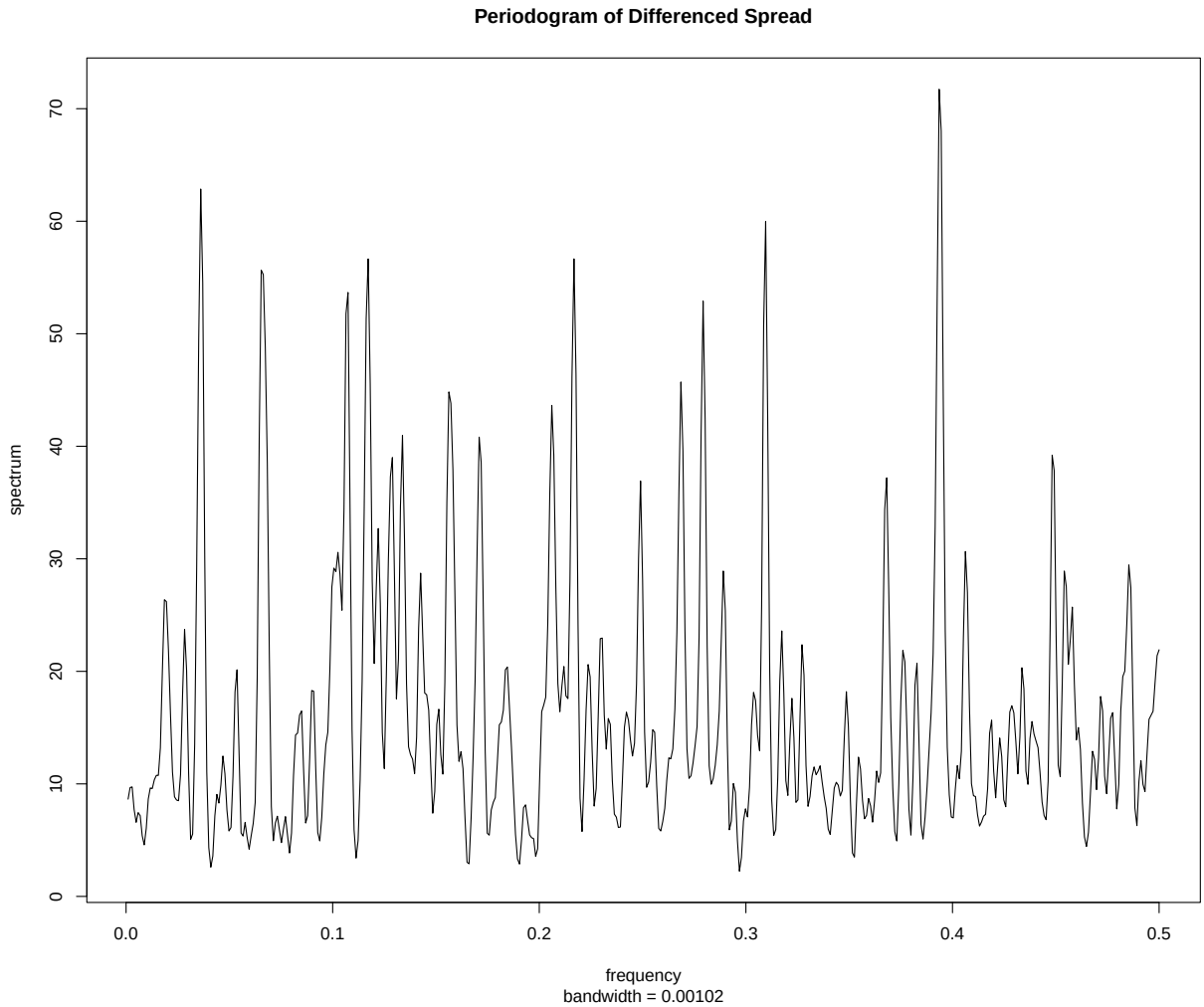
While the squared differenced series exhibits significant spikes, indicating the presence of volatility clustering and justifying the use of GARCH models for further analysis.

3.3 Seasonality Diagnostics

We perform a definitive check for seasonality effects on the differenced spread.

3.3.1 Power Spectrum and Seasonality Tests The periodogram is utilized to analyze frequency strength. A pronounced peak at a specific frequency would indicate a periodic component. Furthermore, we apply the seastests package for a combined formal verification.

```
# Calcola lo spettro di potenza
spec_raw <- spec.pgram(as.numeric(d_train), spans = c(3, 3),
                       taper = 0.1, plot = TRUE, log="no",
                       main="Periodogram of Differenced Spread")
```




```r
# Trova la frequenza con il picco massimo
max_freq <- spec_raw$freq[which.max(spec_raw$spec)]
periodo_dominante <- 1 / max_freq
cat("Dominant period: ", periodo_dominante)
```

```
## Dominant period:  2.540943
```

Spectral Analysis Interpretation

The **Smoothed Periodogram** analysis provides a frequency-domain perspective that complements the previous time-series diagnostics:

- **Frequency Distribution:** The spectrum shows multiple significant peaks rather than a single dominant frequency. This confirms that the BTP-BUND spread is a **multi-cyclical series** where seasonality is fragmented across different time horizons.
- **Short-term Dynamics:** The prominent peaks at higher frequencies (e.g., near 0.4) represent the daily "noise" and short-term volatility. This aligns with the **Maximum value of 250.70 bps**, indicating that sharp, short-lived shocks are a structural feature of the spread's behavior.
- **Low-Frequency Persistence:** The power concentrated near the origin (frequency $\to 0$) represents the **Trend component**. This captures the long-term shifts (2022 surge and 2025 reversion) which, despite the daily noise, remain the most powerful drivers of the spread's variance.

Conclusion: The spectral density confirms that while no simple seasonal rule can predict the spread, the series is far from being pure white noise. The complexity of the periodogram justifies the use of non-linear models (like GARCH) to capture the clusters of energy (volatility) that appear at various frequencies.

```
start_date<- index(d_spread)[1]
start_year<-year(start_date)
start_month<-month(start_date)
start_day <-day(start_date)
d_spread_ts <- ts(d_spread, start=c(start_year,start_month,start_day), frequency = 21)

# Combined test
cat("\nSeasonality Test Results:\n")
```

```
##
## Seasonality Test Results:
```

```
cat("Combined Test: ", isSeasonal(d_spread_ts, test="combined"), "\n")
```

```
## Combined Test:  FALSE
```

```
#Individual tests
cat("\nIndividual Test Results:\n")
```

```
##
## Individual Test Results:
```

```
cat("QS Test: ", isSeasonal(d_spread_ts, test="qs"), "\n")
```

```
## QS Test:  FALSE
```

```
cat("Seasdum Test: ", isSeasonal(d_spread_ts, test="seasdum"), "\n")
```

```
## Seasdum Test:  FALSE
```

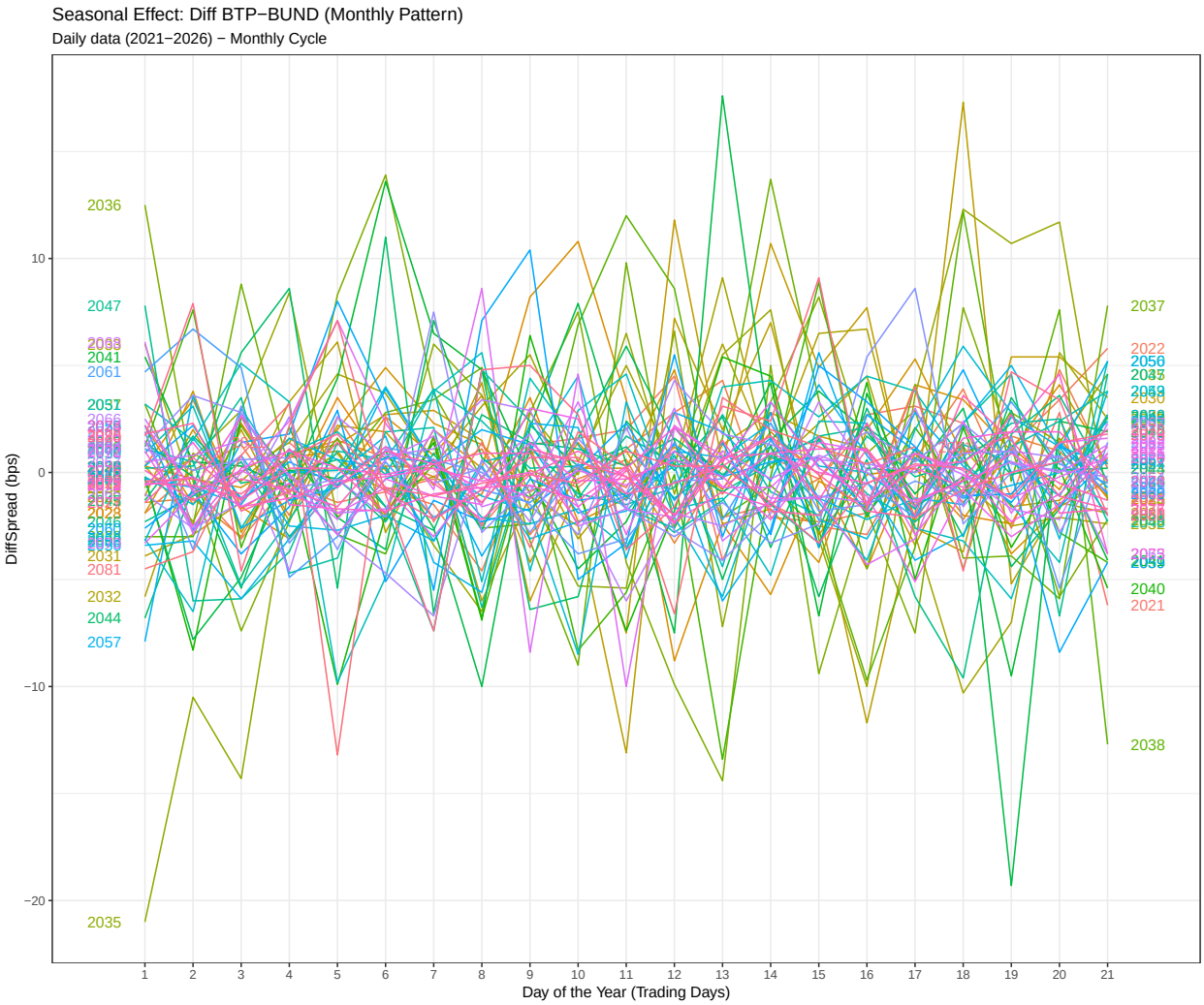
```
cat("KW Test: ", isSeasonal(d_spread_ts, test="kw"), "\n")
```

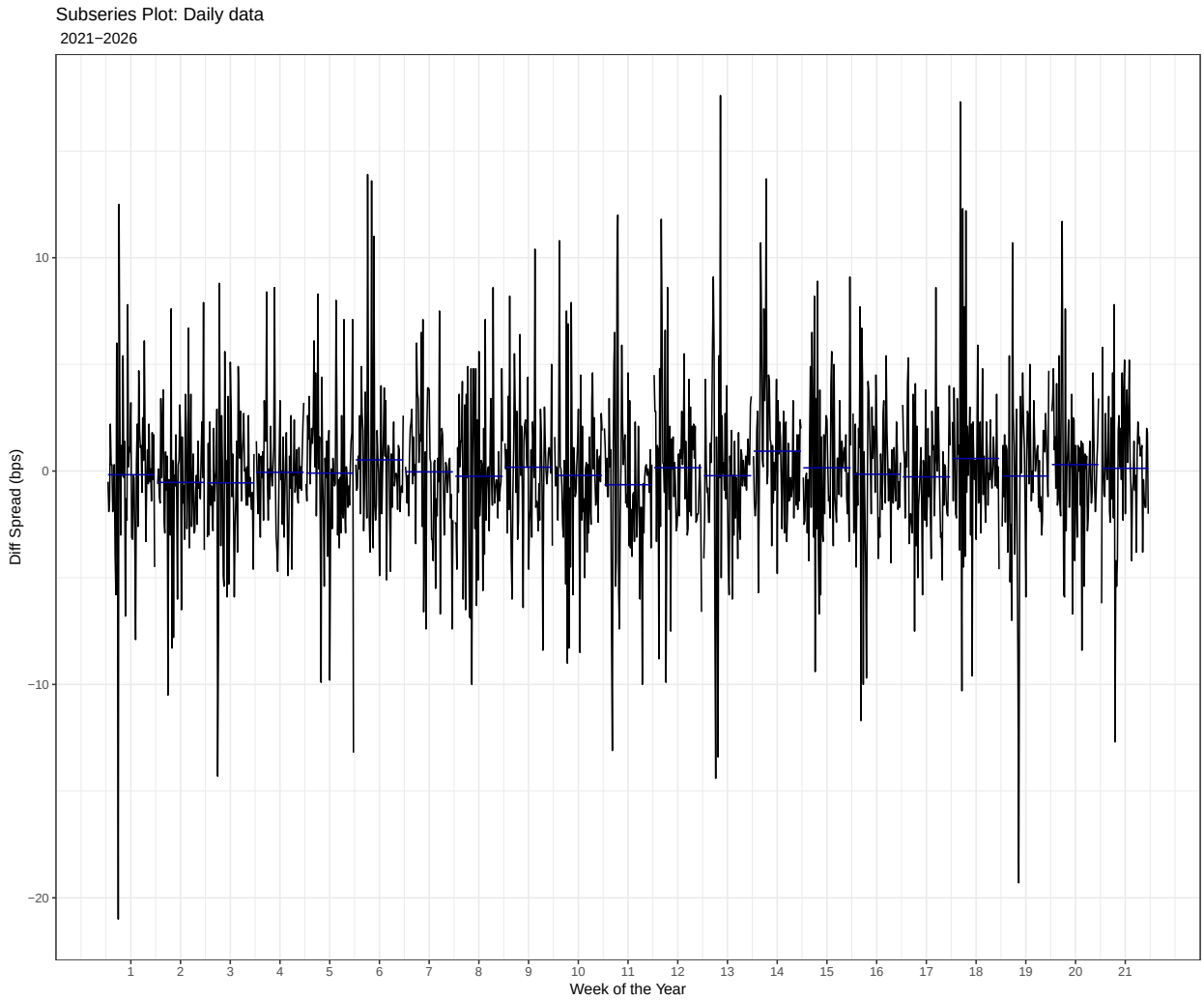
```
## KW Test:  FALSE
```

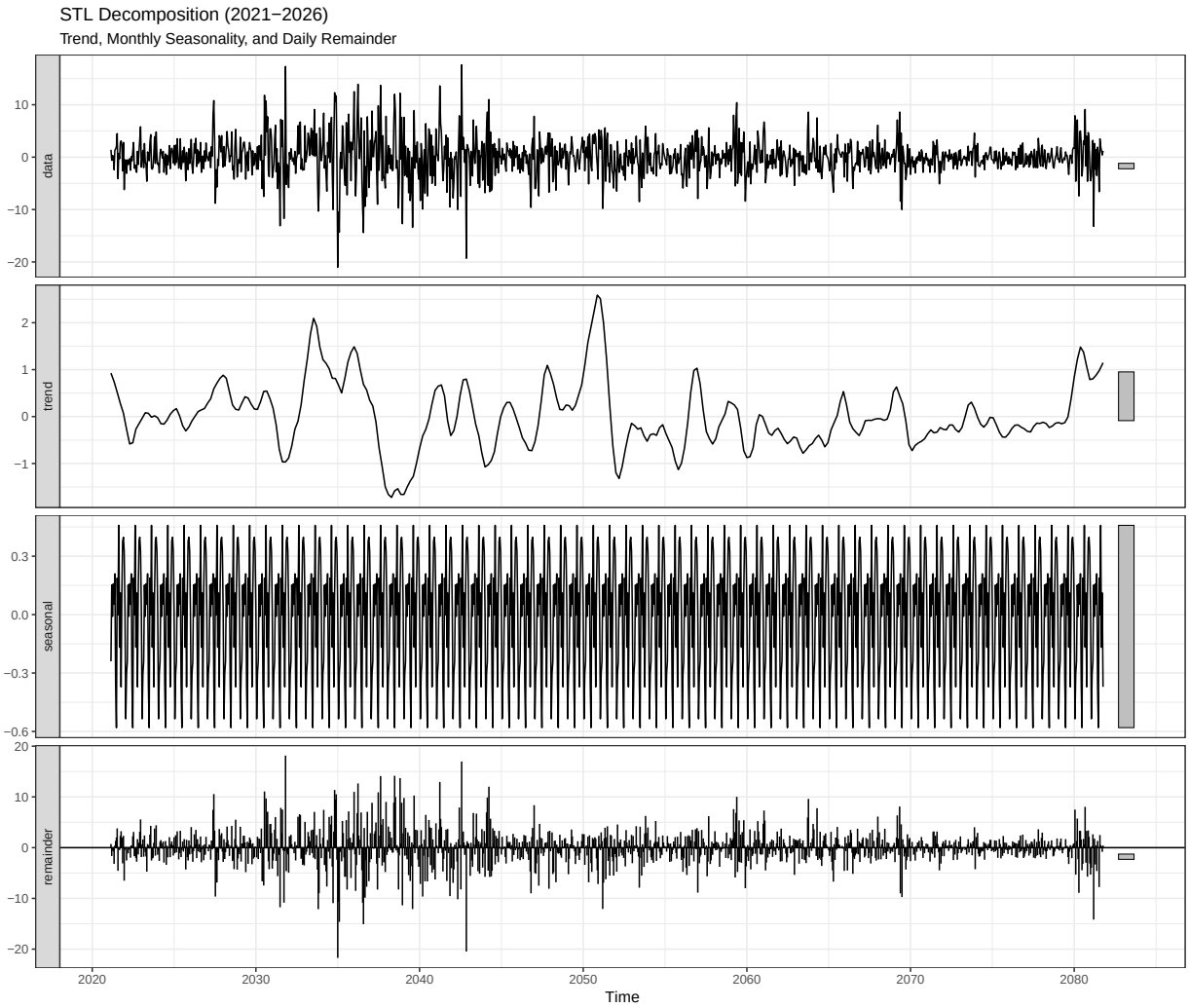
```
cat("Fried Test: ", isSeasonal(d_spread_ts, test="fried"), "\n")
```

```
## Fried Test:  FALSE
```

Visualization for the no-seasonality


Seasonal Effect: Diff BTP−BUND (Monthly Pattern)
Daily data (2021−2026) − Monthly Cycle
DiffSpread (bps)
Day of the Year (Trading Days)
10
0
−10
−20
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
2036
2047
2041
2061
2028
2031
2081
2032
2044
2057
2035
2037
2022
2050
2030
2040
2021
2038


Subseries Plot: Daily data
2021−2026
Diff Spread (bps)
10
0
−10
−20
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
Week of the Year



The decomposition of the BTP-BUND spread at a 21-day frequency reveals a highly active cyclical structure: - **Trend:** The trend component remains the primary driver, capturing the broad expansion of the spread from 2021 through 2024 and the subsequent compression toward 2026. At this frequency, the trend is more reactive to mid-term economic shifts compared to the annual model ($f = 252$). - **Seasonal (Monthly):** The seasonal panel displays a persistent, high-frequency oscillation. However, the amplitude is notably small, ranging between **-1 and +1.5 bps**. This suggests that while a monthly cycle exists, its statistical contribution to the total variance is marginal.(Se il tuo spread medio è di 150 bps e la volatilità giornaliera (Remainder) ha picchi di 10-20 bps, un'oscillazione di soli 3 bps è considerata trascurabile (noise-level).) - **Remainder:** The remainder component continues to exhibit significant "bursts" or clusters of volatility. This indicates that sudden market news (stochastic shocks) has a much larger impact on daily spread changes than any predictable monthly calendar effect.

All individual tests (QS, Seasonal Dummy, Kruskal-Wallis, and Friedman) consistently fail to reject the null hypothesis of no seasonality.

Conclusion: Having confirmed that the series is stationary, lacks seasonality, and exhibits significant ARCH effects, we can proceed with confidence to estimate the non-seasonal ARIMA-GARCH framework.

3.4 Autocorrelation and ARCH tests

To ensure the necessity of both the mean and variance components of the model, we perform rigorous diagnostic tests on the differenced training set ($dtrain$). We select a high lag (e.g., $lag = 16$) to be meticulous in detecting long-term dependencies.

```
# 1. Ljung-Box Test on differences
lb_ret <- Box.test(as.numeric(d_train), lag = 16, type = "Ljung-Box")
cat("\nLB Test on ds p-value:\n", lb_ret$p.value, "\n")
```

```
##
## LB Test on ds p-value:
##  0.05519601
```

```
if(lb_ret$p.value < 0.05) cat("Autocorrelation detected, need mean modelling\n") else cat("ds are White
```

```
## ds are White Noise
```

```
# 2. Ljung-Box Test on squared differences
lb_test <- Box.test(as.numeric(d_train)^2, lag = 16, type = "Ljung-Box")
cat(sprintf("\nLjung-Box on Squared ds at lag 16:\n %.4f\n", lb_test$p.value))
```

```
##
## Ljung-Box on Squared ds at lag 16:
##  0.0000
```

```
if(lb_test$p.value < 0.05) cat("Volatility clustering confirmed\n") else cat("No volatility clustering
```

```
## Volatility clustering confirmed
```

```
# 3. ARCH-LM Test (Engle's Test for ARCH effects)
arch_test <- ArchTest(d_train, lags = 16)
cat(sprintf("\nARCH Test (16 lags) LM Test - statistic: %.4f, p-value: %.4f\n", arch_test$statistic, ar
```

```
##
## ARCH Test (16 lags) LM Test - statistic: 120.5067, p-value: 0.0000
```

```
if(arch_test$p.value < 0.05) cat("ARCH effects confirmed. GARCH modelling needed\n") else cat("No ARCH
```

```
## ARCH effects confirmed. GARCH modelling needed
```

4 ARIMA estimation

We finally estimate the ARIMA model on the differenced training set, use `auto_arima()` to find the best arima according to `AIC`criteria.

Keep ARMA order as low as possible to not overfit or model noise.

```r
m1 <- auto.arima(d_train,max.q = 16, max.p = 16,
                        seasonal = TRUE,
                        stepwise = FALSE,
                        trace = FALSE)

cat("\nBest ARIMA model:\n")
```

```
##
## Best ARIMA model:
```

```r
print(summary(m1))
```

```
## Series: d_train
## ARIMA(0,0,0) with zero mean
##
## sigma^2 = 15.26:  log likelihood = -2837.16
## AIC=5676.32   AICc=5676.33   BIC=5681.25
##
## Training set error measures:
##                      ME     RMSE      MAE MPE MAPE      MASE       ACF1
## Training set 0.01088235 3.906303 2.789902 100  100 0.6926156 0.02594926
```

The estimated model is an ARMA(3,2), that's the best ARMA model for our train set, now we will run some diagnostic tests to see if correctly specified.

We specify also the model m0=ARIMA(0,0,0) to compare the performances

```r
# Fit ARIMA(1,0,0) - equivalent to AR(1)
m0 <- Arima(as.numeric(d_train), order = c(0, 0, 0))

# Summary of coefficients and information criteria
summary(m0)
```

```
## Series: as.numeric(d_train)
## ARIMA(0,0,0) with non-zero mean
##
## Coefficients:
##         mean
##       0.0109
## s.e.  0.1223
##
## sigma^2 = 15.27:  log likelihood = -2837.16
## AIC=5678.31   AICc=5678.33   BIC=5688.17
##
## Training set error measures:
##                               ME     RMSE      MAE  MPE MAPE      MASE
## Training set 0.00000000000027198 3.906288 2.789817 -Inf  Inf 0.6925944
##                     ACF1
## Training set 0.02594926
```

We can see from both AIC/BIC and error measures that the models are pretty much similar, model m1 add no improvements to simple white noise.

4.1 Coefficients Significance

Statistical Significance of CoefficientsTo ensure model parsimony, we evaluate the significance of each parameter. A coefficient is statistically significant at the 95% confidence level if its t-ratio ($t = \frac{\hat{\phi}}{SE(\hat{\phi})}$) exceeds the critical value of $|1.96|$.

```
check_coef_significance <- function(model) {
  coefs <- model$coef
  se <- sqrt(diag(model$var.coef))
  t_stats <- coefs / se
  # Calcoliamo i p-value (distribuzione normale a due code)
  p_values <- (1 - pnorm(abs(t_stats))) * 2
  results <- data.frame(
    Coeff = round(coefs, 4),
    Std_Error = round(se, 4),
    t_stat = round(t_stats, 4),
    p_value = round(p_values, 4),
    Significance_95 = ifelse(p_values < 0.05, "Yes", "No")
  )
  cat("\nCoefficients significance:\n")
  print(results)
}
check_coef_significance(m1)
```

```
##
## Coefficients significance:
## [1] Coeff
## <0 rows> (or 0-length row.names)
```

All the coefficients are significant.

4.2 ARMA assumptions and Cycle Analysis

```
arma_assumptions <- function(m) {
  cat("\n==============================================================\n")
  cat("ARIMA model assumptions & Cycle Analysis:\n")

  # Estrazione corretta per modelli auto.arima/Arima
  # Usiamo i coefficienti AR per la stazionarietà
  ar_coefs <- m$model$phi
  ma_coefs <- m$model$theta

  # 1. Calcolo Radici
  ar_roots <- polyroot(c(1, -ar_coefs))
  ma_roots <- polyroot(c(1, ma_coefs))

  is_stationary <- all(abs(ar_roots) > 1)
  is_invertible <- all(abs(ma_roots) > 1)

  cat("Stationary:", is_stationary, "(Min mod:", round(min(abs(ar_roots)), 3), ")\n")
  cat("Invertible:", is_invertible, "(Min mod:", round(min(abs(ma_roots)), 3), ")\n")
```

```r
  # 2. Analisi dei Business Cycles (Radici Complesse)
  # Cerchiamo se esistono radici con parte immaginaria significativa
  complex_roots <- ar_roots[Im(ar_roots) != 0]

  cat("\n--- Business Cycle Analysis ---\n")
  if(length(complex_roots) > 0) {
    cat("The model IMPLIES the existence of cycles.\n")
    cat("Reason: The AR characteristic equation has complex conjugate roots.\n")

    # Calcolo del periodo del ciclo (se ti serve per l'analisi)
    # Periodo = 2*pi / acos(Re(root)/abs(root))
    root <- complex_roots[1]
    period <- 2 * pi / atan2(Im(root), Re(root))
    cat("Estimated average cycle period:", round(abs(period), 2), "units of time.\n")
  } else {
    cat("The model DOES NOT imply cycles.\n")
    cat("Reason: All AR roots are real; the model returns to equilibrium monotonically.\n")
  }

  # 3. Check for common roots
  common_roots <- FALSE
  if(length(ar_roots) > 0 && length(ma_roots) > 0) {
    for(ar in ar_roots) {
      for(ma in ma_roots) {
        if(isTRUE(all.equal(ar, ma, tolerance = 1e-3))) common_roots <- TRUE
      }
    }
  }

  cat("\n--- Root Validation ---\n")
  if(common_roots) {
    cat("Common roots detected, assumption violation\n")
  } else {
    cat("No common roots detected, assumption validated\n")
  }
  cat("============================================================\n")
}

# Esecuzione
arma_assumptions(m1)
```

```
##
## ============================================================
## ARIMA model assumptions & Cycle Analysis:

## Warning in min(abs(ar_roots)): no non-missing arguments to min; returning Inf

## Stationary: TRUE (Min mod: Inf )

## Warning in min(abs(ma_roots)): no non-missing arguments to min; returning Inf

## Invertible: TRUE (Min mod: Inf )
```

```
##
## --- Business Cycle Analysis ---
## The model DOES NOT imply cycles.
## Reason: All AR roots are real; the model returns to equilibrium monotonically.
##
## --- Root Validation ---
## No common roots detected, assumption validated
## ==============================================================
```

4.3 ARMA residuals diagnostics

For distribution we use a Kolmogorov-Smirnov test (KS) to test wheter they follow a t-distribution H_0 : serie follows supposed distribution

```
arima_res <- residuals(m1)

arma_residual <- function(res) {

  lb_arima <- Box.test(res, lag = 16, type = "Ljung-Box")
  cat("\nLB test on ARIMA residuals (16 lags) p-value: ", round(lb_arima$p.value, 4), "\n")
  if(lb_arima$p.value >= 0.05) cat("ARIMA model cleared the mean\n") else cat("ARIMA model DID NOT clea

  lb_arima_2 <- Box.test(res^2, lag = 16, type = "Ljung-Box")
  cat("\nLB test on ARIMA squared residuals (16 lags) p-value: ", round(lb_arima_2$p.value, 4), "\n")
  if(lb_arima_2$p.value < 0.05) cat("ARIMA model has residual variance to explain. GARCH model needed\n

  # ARCH test
  arch_test_res <- ArchTest(res, lags = 10)
  cat(sprintf("\nARCH Test on ARIMA (10 lags) residuals:\n LM Test - statistic: %.4f, p-value: %.4f\n",
              arch_test_res$statistic, arch_test_res$p.value))
  if(arch_test_res$p.value < 0.05) cat("ARCH effects confirmed. GARCH modelling needed\n") else cat("No

  # Distribution
  jb_test <- jarque.bera.test(arima_res)
  if(jb_test$p.value < 0.05) cat("\nResiduals are NOT normally distributed\n") else cat("\nResiduals ar

  check_t <- function(residuals) {
  fit_t <- suppressWarnings(fitdistr(residuals, densfun = "t"))
  mu <- fit_t$estimate["m"]
  sigma <- fit_t$estimate["s"]
  df <- fit_t$estimate["df"]
  std_res <- (residuals - mu) / sigma
  ks <- ks.test(x = std_res, y = "pt", df = df)
  cat("\nKS test assuming t-distribution:\n")
  print(c(DoF = df, KS_p_value = ks$p.value))
  if(ks$p.value > 0.05) cat("\nResiduals are t-distributed\n") else cat("\nResiduals are NOT t-distribu

}
  check_t(arima_res)
}

print(arma_residual(arima_res))
```

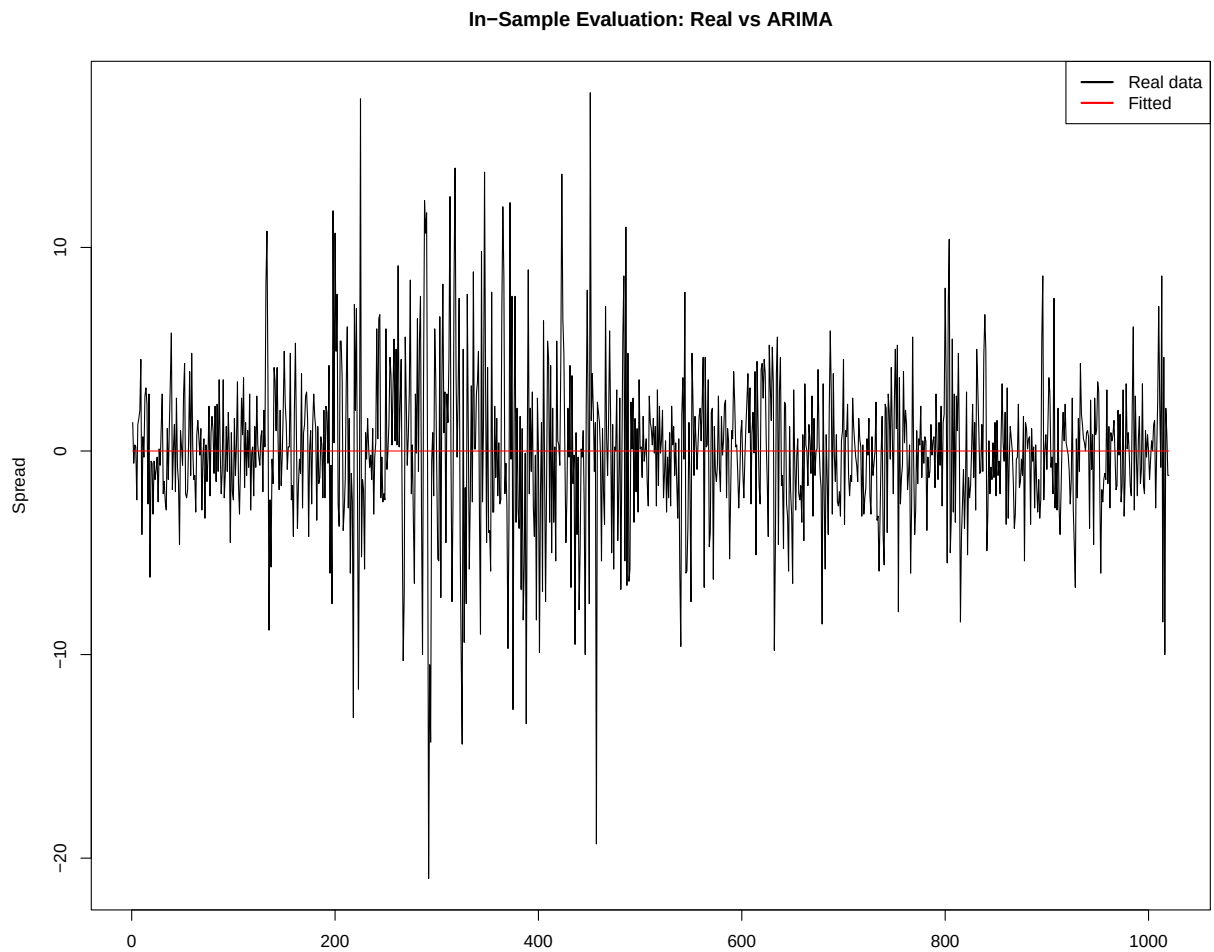
```
##
## LB test on ARIMA residuals (16 lags) p-value:  0.0552
## ARIMA model cleared the mean
##
## LB test on ARIMA squared residuals (16 lags) p-value:  0
## ARIMA model has residual variance to explain. GARCH model needed
##
## ARCH Test on ARIMA (10 lags) residuals:
##  LM Test - statistic: 114.8764, p-value: 0.0000
## ARCH effects confirmed. GARCH modelling needed
##
## Residuals are NOT normally distributed
```

```
## Warning in ks.test.default(x = std_res, y = "pt", df = df): ties should not be
## present for the one-sample Kolmogorov-Smirnov test
```

```
##
## KS test assuming t-distribution:
##     DoF.df KS_p_value
##  3.4976191  0.8004508
##
## Residuals are t-distributed
## NULL
```

m1 model pass all the tests

4.4 ARIMA Forecast



In-Sample Evaluation

```
residuals <- as.numeric(residuals(m1))

# Core metrics
accuracy(as.numeric(d_train), as.numeric(fitted_values))
```

```
##                    ME     RMSE      MAE MPE MAPE
## Test set -0.01088235 3.906303 2.789902 NaN  Inf
```

```
# R-squared (will be 0 or near 0 - expected for mean model)
SS_res <- sum(residuals^2, na.rm = TRUE)
SS_tot <- sum((d_train - mean(d_train, na.rm = TRUE))^2, na.rm = TRUE)
R2 <- 1 - SS_res / SS_tot
cat("\nModel R2:", round(R2, 2), "%")
```

```
##
## Model R2: 0 %
```

```r
sign_actual <- sign(as.numeric(d_train))
sign_fitted <- sign(as.numeric(fitted_values))
hits <- (sign_actual == sign_fitted)

da <- mean(hits, na.rm = TRUE) * 100
cat("\nDirectional Accuracy (DA):", round(da, 2), "%")
```

```
##
## Directional Accuracy (DA): 1.37 %
```

Out-of-Sample Evaluation

```r
refit <- Arima(d_spread, model = m1)
pred_out <- tail(fitted(refit), n_test)
reali_out <- tail(as.numeric(d_spread), n_test)

accuracy(pred_out, reali_out)
```

```
##                   ME     RMSE     MAE MPE MAPE
## Test set -0.1529412 2.218001 1.56549 100  100
```

```r
SS_res <- sum((pred_out-reali_out)^2, na.rm = TRUE)
SS_tot <- sum((test - mean(test, na.rm = TRUE))^2, na.rm = TRUE)
R2 <- 1 - SS_res / SS_tot
cat("Model R2:", round(R2, 2), "%")
```

```
## Model R2: 0.97 %
```

```r
sign_actual <- sign(as.numeric(test))
sign_fitted <- sign(as.numeric(pred_out))
hits <- (sign_actual == sign_fitted)
da <- mean(hits, na.rm = TRUE) * 100
cat("DA del modello:", round(da, 2), "%")
```

```
## DA del modello: 0 %
```

If we substitute m1 with m0, we can see that m1 perform better both insample and out-sample

4.4.1 Tomorrow forecast

Compute the actual tomoroww forecast, using only this ARMA model

```r
forecast_next_week <- forecast(refit, h = 1)

cat("\nForecast for differenced spread tomorrow:\n")
```

```
##
## Forecast for differenced spread tomorrow:
```

```r
print(forecast_next_week)
```

```
##      Point Forecast     Lo 80    Hi 80     Lo 95    Hi 95
## 1275              0 -5.006129 5.006129 -7.656214 7.656214
```

```r
# Convert to spread
last_spread <- tail(spread, 1)
fc_diff <- forecast_next_week$mean[1]
fc_spread <- last_spread + fc_diff
fc_spread_lo95 <- last_spread + forecast_next_week$lo[1, 2]
fc_spread_hi95 <- last_spread + forecast_next_week$up[1, 2]

cat("\nForecasted spread for tomorrow:\n")
```

```
##
## Forecasted spread for tomorrow:
```

```r
cat("Point forecast:", fc_spread, "\n")
```

```
## Point forecast: 79.3
```

```r
cat("95% CI:", fc_spread_lo95, "to", fc_spread_hi95, "\n")
```

```
## 95% CI: 71.64379 to 86.95621
```

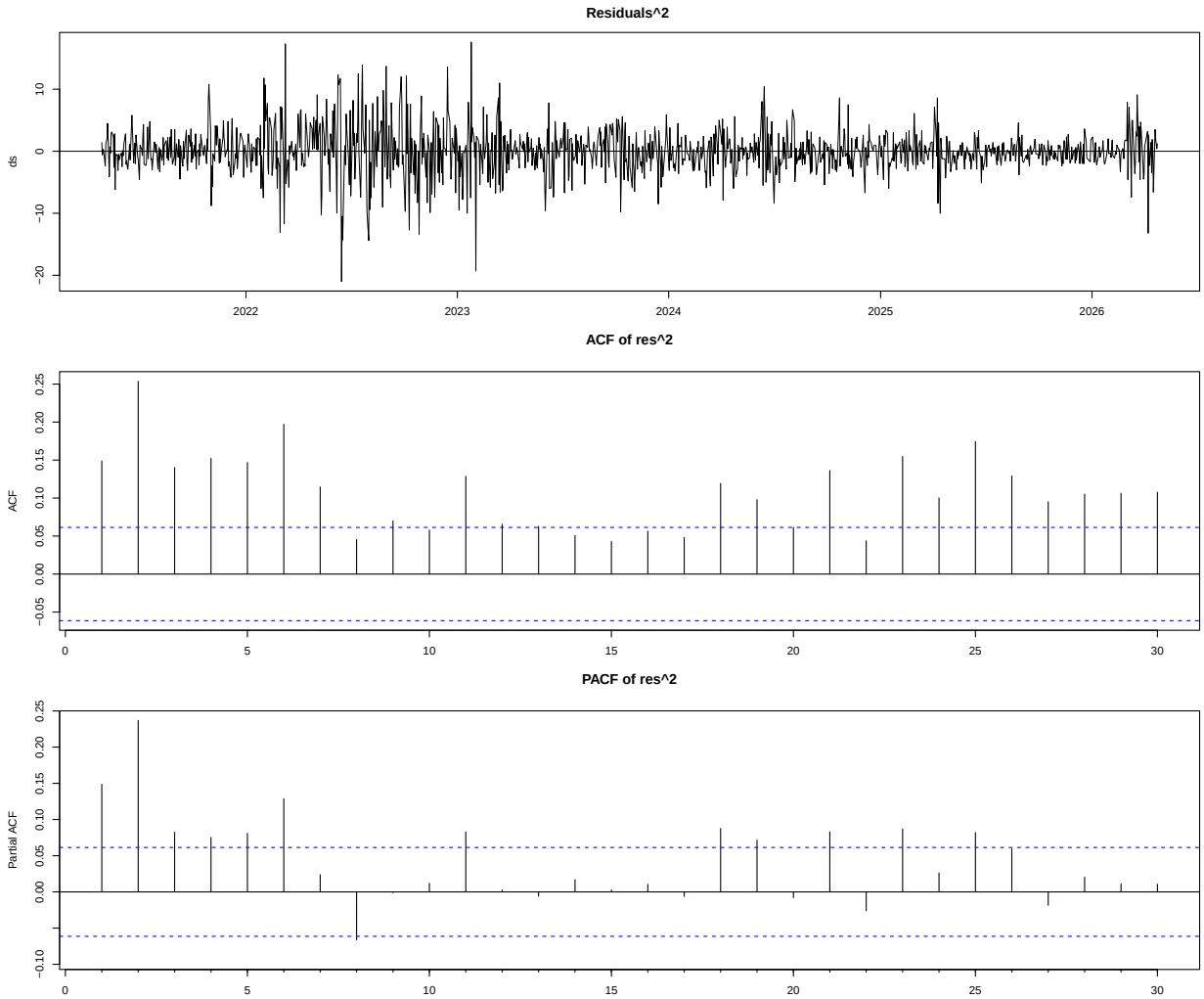
CONSIDERARE DI FARE UNA CROSS VALIDATION PER UNA ROBA PIU ACCURATA

5 Volatility Analysis

We restart from our model and test if the residuals need volatility modelling

```r
summary(m1)
```

```
## Series: d_train
## ARIMA(0,0,0) with zero mean
##
## sigma^2 = 15.26:  log likelihood = -2837.16
## AIC=5676.32   AICc=5676.33   BIC=5681.25
##
## Training set error measures:
##                       ME     RMSE      MAE MPE MAPE      MASE       ACF1
## Training set 0.01088235 3.906303 2.789902 100  100 0.6926156 0.02594926
```



Check for autocorrelation and ARCH effects on train data

```
DoF <- sum(m1$coef!=0)
# Ljung-Box on DS^2
lb_arima_res2 <- Box.test(res^2, lag = 16, type = "Ljung-Box", fitdf = DoF)
cat("\nLB test on ARIMA squared residuals (16 lags) p-value: ", round(lb_arima_res2$p.value, 4), "\n")
```

```
##
## LB test on ARIMA squared residuals (16 lags) p-value:  0
```

```
if (lb_arima_res2$p.value < 0.05) cat("Volatility clustering confirmed\n") else cat("No volatility clus
```

```
## Volatility clustering confirmed
```

```
# ARCH test
arch_test <- ArchTest(as.numeric(res), lags = 16)
cat(sprintf("ARCH Test (16 lags) p-value: %.4f\n", arch_test$p.value))
```

```
## ARCH Test (16 lags) p-value: 0.0000
```

```r
if (arch_test$p.value < 0.05) cat("ARCH effects confirmed - GARCH modelling needed\n") else cat("No ARCH
```

```
## ARCH effects confirmed - GARCH modelling needed
```

We clearly need a specification model for volatility

5.1 Best ARMA-GARCH

We try to find the best ARMA-GARCH fitting different combinations of models, then decide considering a precise IC, kinda like an auto_arima but hand-builded.

Create a function that simulates 1 iteration of a for cycle, then instead of a for cycle use parrallel (mclapply) It will run the function over n times using n CPU simultanealy.

Can change the critera or modify the parameters, if too much combinations the code will run for ages.

We will keep the max possible GARCH orders to 1, that's cause vast majority of the time a GARCH(1,1) is perfect for financial series. We could add more GARCH model other than `sGARCH`and `eGARCH` but since those are the most used we keep only them, also other models have additional parameter than can increase the computational time. We also capped max ARMA order to 3 to prevent the AIC from picking up noise (it does, tried to run multiple times and it gives back models that do not pass diagnostic tests).

The ugarch solvers (`gosolnp`) use random generate starting point, to have reproducibility of results we need to impose a seed whenever a optimization is done.

NOTE: in `dists` you choose the distribution you suppose the std residuals will follow, `norm`= normal, `std`= t, `sstd`= skewed t. I removed `norm`and cause the function would wrongly choose `norm` over the other 2: you can see if is the distribution fit the residuals in the printed residual plot or in the "Adjusted Pearson Goodness-of-Fit Test" at the end of the model output, when `norm` the fit is worse. The distribution that better fit the residuals is `sstd` (even if not perfect).

The reason is that `sstd` and `std` have more parameters than `norm` (number of parameter: `sstd` > `std` > `norm`) and since the train data is "small" this penalize them more than it should in the eye of AIC/BIC. `norm` will end up with better IC than `sstd` but then you have the wrong distribution for the residuals and/or fail the diagnostic tests.

```r
start_time <- proc.time() # to see for how much the code run

best_garch <- function(y,
                       ar_lags = 0:3, ma_lags = 0:3,
                       garch_p = 1:1,  garch_q = 1:1,
                       models  = c("sGARCH", "sGARCH-M"),
                       dists   = c("std", "sstd"),
                       criteria = "bic", timeout = 60) {

  combos <- expand.grid(
    ar_lag  = ar_lags,
    ma_lag  = ma_lags,
    garch_p = garch_p,
    garch_q = garch_q,
    model   = models,
    dist    = dists,
    stringsAsFactors = FALSE
  )
```

```r
cat("Total combinations to fit:", nrow(combos), "\n")
num_cores <- detectCores() - 1
cat("Running on", num_cores, "cores\n")

fit_one <- function(i) {
  arp   <- combos$ar_lag[i]
  map   <- combos$ma_lag[i]
  gp    <- combos$garch_p[i]
  gq    <- combos$garch_q[i]
  model <- combos$model[i]
  dist  <- combos$dist[i]

  archm <- model == "sGARCH-M"
  true_model <- ifelse(archm, "sGARCH", model)

  spec <- ugarchspec(
    variance.model = list(model = true_model, garchOrder = c(gp, gq)),
    mean.model     = list(armaOrder = c(arp, map), include.mean = TRUE,
                          archm = archm),
    distribution.model = dist
  )
  fit <- tryCatch({
    withTimeout({
      suppressWarnings(
        ugarchfit(spec = spec, data = y, solver = "hybrid",
        solver.control = list(n.restarts = 5, rseed = 42, trace = 0))
      )
    },
    timeout   = timeout,
    onTimeout = "silent")
  },
  error   = function(e) NULL)
  # Store only not null + right class
  if (!is.null(fit) && inherits(fit, "uGARCHfit") && fit@fit$convergence == 0) {
    ic <- infocriteria(fit)
    return(data.frame(
      ar_p    = arp,
      ma_q    = map,
      garch_p = gp,
      garch_q = gq,
      model   = model,
      dist    = dist,
      aic     = as.numeric(round(ic["Akaike", ] * length(y), 2)),
      bic     = as.numeric(round(ic["Bayes",  ] * length(y), 2)),
      likelihood = as.numeric(round(likelihood(fit), 2)),
      stringsAsFactors = FALSE
    ))
  }
  return(NULL)
}

# Gestione Cluster (Windows richiede export esplicito)
os_name <- Sys.info()["sysname"]
```

```r
  if (os_name == "Windows") {
    cl <- makeCluster(num_cores)
    clusterSetRNGStream(cl, iseed = 42)
    clusterExport(cl, varlist = c("y", "combos", "fit_one"), envir = environment())
    clusterEvalQ(cl, { library(rugarch); library(R.utils) })
    results_list <- parLapply(cl, seq_len(nrow(combos)), fit_one)
    stopCluster(cl)
  } else {
    results_list <- mclapply(seq_len(nrow(combos)), fit_one, mc.cores = num_cores)
  }
  # Keep only non NULL model
  successful <- Filter(Negate(is.null), results_list)
  if (length(successful) == 0) return(NULL)
  cat("Successful fits:", length(successful), "/", nrow(combos), "\n")
  cat("Models ordered by: ", criteria, "\n")
  results_df <- bind_rows(successful) |> arrange(.data[[criteria]])
  return(results_df)
}

results <- best_garch(as.numeric(d_train))
```

```
## Total combinations to fit: 64
## Running on 11 cores
## Successful fits: 64 / 64
## Models ordered by:  bic
```

```r
cat("\nTop models by BIC:\n")
```

```
##
## Top models by BIC:
```

```r
print(head(results))
```

```
##   ar_p ma_q garch_p garch_q    model dist     aic     bic likelihood
## 1    0    0       1       1   sGARCH  std 5397.97 5422.61   -2693.99
## 2    1    0       1       1   sGARCH  std 5398.40 5427.97   -2693.20
## 3    0    1       1       1   sGARCH  std 5398.42 5427.99   -2693.21
## 4    0    0       1       1 sGARCH-M  std 5399.95 5429.51   -2693.97
## 5    0    0       1       1   sGARCH sstd 5399.97 5429.54   -2693.99
## 6    1    1       1       1   sGARCH  std 5399.99 5434.48   -2692.99
```

```r
end_time <- proc.time()
execution_time <- end_time - start_time
cat("\n Total execution time: ", execution_time[3], "seconds")
```

```
##
##  Total execution time:  8.41 seconds
```

Extract the best model.

```r
arma_best_order <- c(results$ar_p[1], results$ma_q[1])
garch_best_order  <- c(results$garch_p[1], results$garch_q[1])
best_dist  <- results$dist[1]
best_mod  <- results$model[1]

cat("\nBest ARMA-GARCH model orders:\nARMA:  ", arma_best_order, "\nGARCH: ", garch_best_order,
    "\nModel: ", best_mod, "\nDistribution: ", best_dist, "\n")
```

```
##
## Best ARMA-GARCH model orders:
## ARMA:   0 0
## GARCH:  1 1
## Model:  sGARCH
## Distribution:  std
```

```r
garch_spec <- ugarchspec(variance.model = list(model = best_mod, garchOrder = garch_best_order),
                         mean.model = list(armaOrder =arma_best_order, include.mean = TRUE, archm = grep
                         distribution.model = best_dist)

set.seed(42)
garch_fit <- ugarchfit(spec = garch_spec, data = d_train, solver = "hybrid",
solver.control = list(n.restarts = 5, rseed = 42, trace = 0)) # for reproducibility
```

- best model with dtrain(4 anni)(max garch par=3) = ARMA-GARCH(4,1)-(3,3),distribution=std (RANK=AIC) agg.03_04
- best model with dtrain(4 anni)(max garch par=1) = ARMA-GARCH(4,2)-(1,1),distribution=sstd (RANK=AIC) agg.03_04

- best model with dtrain(4 anni)(max garch par=1) = ARMA-GARCH(3,2)-(1,1),distribution=norm (RANK=BIC) agg.03_04
- best model with dtrain(4 anni)(max garch par=1) = ARMA-GARCH-M(2,2)-(1,1),distribution=std (RANK=AIC) agg.10_04
- best model with dtrain(4 anni)(max garch par=1) = ARMA-GARCH-M(0,0)-(1,1),distribution=std (RANK=BIC) agg.10_04 -best model with dtrain(4 anni)(max garch par=5) = ARMA-GARCH(0,0)-(1,1),distribution=std (RANK=BIC) agg.17_04 -best model with dtrain(4 anni)(max garch par=5) = ARMA-GARCH(0,0)-(1,1),distribution=std (RANK=BIC) agg.20_04

5.2 Check the GARCH positivity of coefficients

```r
# ============================================================
# GARCH POSITIVITY & STATIONARITY CHECK (rugarch)
# ============================================================
check_garch_positivity <- function(fit, model_name = "GARCH fit") {

  cf    <- fit@fit$coef
  # omega is always named "omega" in rugarch
  omega <- cf["omega"]
  # alpha coefficients: alpha1, alpha2, ...
  alphas <- cf[grepl("^alpha[0-9]+", names(cf))]
  # beta coefficients: beta1, beta2, ...
```

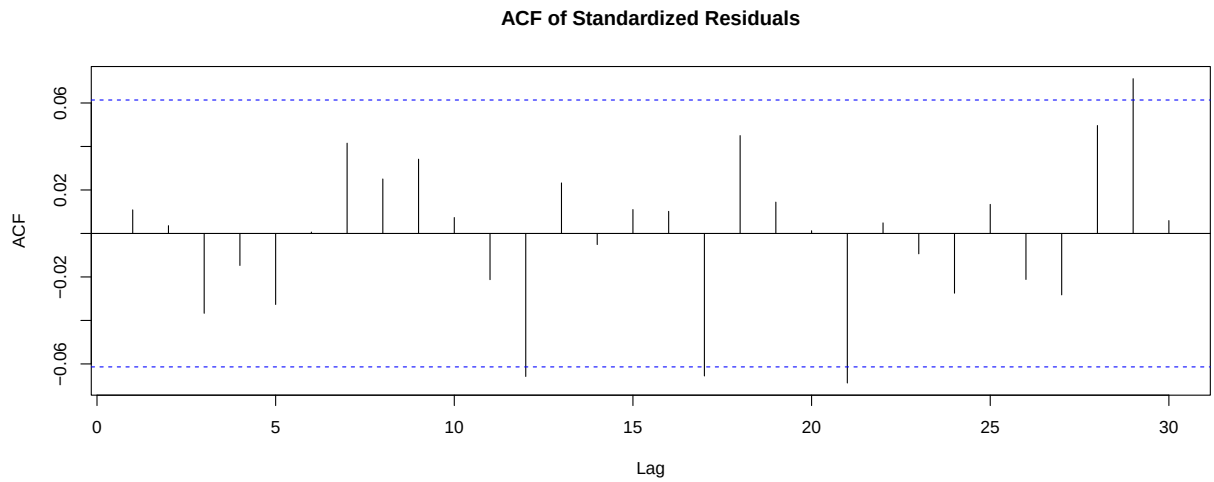
```r
  betas  <- cf[grepl("^beta[0-9]+",  names(cf))]

  persistence <- sum(alphas) + sum(betas)

  positivity_ok   <- (omega > 0) & all(alphas >= 0) & all(betas >= 0)
  stationarity_ok <- persistence < 1

  cat("\n==============================================================\n")
  cat("GARCH POSITIVITY & STATIONARITY CHECK:", model_name, "\n")
  cat("==============================================================\n")
  cat(sprintf("  omega  = %+.6f  > 0?  %s\n",
              omega,  ifelse(omega > 0,      "YES  ", "NO   ")))

  for (nm in names(alphas)) {
    cat(sprintf("  %-7s= %+.6f  >= 0? %s\n",
                nm, alphas[nm], ifelse(alphas[nm] >= 0, "YES  ", "NO   ")))
  }
  for (nm in names(betas)) {
    cat(sprintf("  %-7s= %+.6f  >= 0? %s\n",
                nm, betas[nm],  ifelse(betas[nm]  >= 0, "YES  ", "NO   ")))
  }

  cat(sprintf("\n  Persistence (sum  + sum  ) = %.6f  < 1? %s\n",
              persistence, ifelse(stationarity_ok, "YES   (covariance stationary)", "NO    (IGARCH or ex

  if (positivity_ok & stationarity_ok) {
    uncond_var <- as.numeric(omega) / (1 - persistence)
    cat(sprintf("  Unconditional variance       = %.6f\n", uncond_var))
    cat(sprintf("  Unconditional vol (daily)    = %.4f bps\n", sqrt(uncond_var)))
    cat("  --> All positivity conditions SATISFIED  \n")
  } else {
    if (!positivity_ok)   cat("  --> WARNING: Positivity condition VIOLATED  \n")
    if (!stationarity_ok) cat("  --> WARNING: Stationarity condition VIOLATED (persistence >= 1)  \n")
  }
  cat("==============================================================\n\n")

  invisible(list(
    omega         = omega,
    alphas        = alphas,
    betas         = betas,
    persistence   = persistence,
    positivity_ok = positivity_ok,
    stationarity_ok = stationarity_ok
  ))
}
```

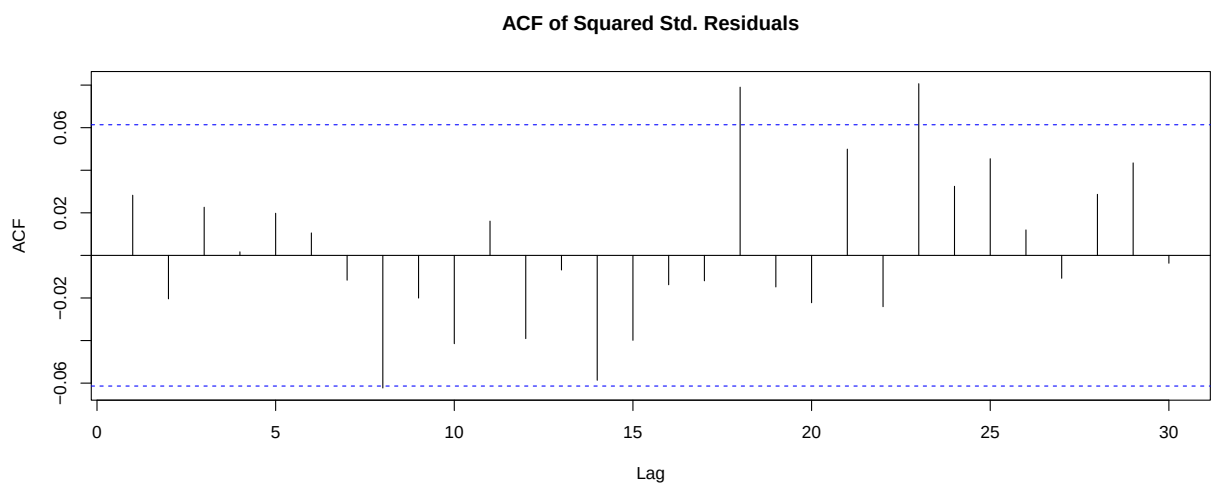
```r
# --- Positivity check on the training GARCH fit ---
check_garch_positivity(garch_fit, model_name = "ARMA-GARCH (training fit)")
```

```
##
## ==============================================================
## GARCH POSITIVITY & STATIONARITY CHECK: ARMA-GARCH (training fit)
## ==============================================================
```

```
##   omega  = +0.632398  > 0?  YES
##   alpha1 = +0.190795  >= 0? YES
##   beta1  = +0.779505  >= 0? YES
##
##   Persistence (sum   + sum  ) = 0.970300  < 1? YES   (covariance stationary)
##   Unconditional variance       = 21.292949
##   Unconditional vol (daily)    = 4.6144 bps
##   --> All positivity conditions SATISFIED
## ============================================================
```

```
par(mfrow = c(2, 1))
Acf(garch_fit@fit$z, main = "ACF of Standardized Residuals")
Acf(garch_fit@fit$z^2,lag.max=30,main = "ACF of Squared Std. Residuals")
```






```
par(mfrow = c(1, 1))

#Ljung-Box test
Box.test(garch_fit@fit$z^2, lag = 20, type = "Ljung-Box", fitdf = 2)
```

```
##
```

```
##  Box-Ljung test
##
## data:  garch_fit@fit$z^2
## X-squared = 23.312, df = 18, p-value = 0.1789
```

5.3 Best model analysis

Variance parameters

I coefficienti stimati descrivono la struttura della volatilità e la sua evoluzione temporale:

- ω **(Omega) = +0.638728**: Rappresenta la costante della varianza. Essendo positiva, assicura un livello minimo di volatilità ("rumore di fondo") sempre presente nel mercato.
- α_1 **(Alpha) = +0.195306**: Misura la reattività immediata. Indica che il **18%%** di uno shock inatteso (news, eventi politici) si traduce istantaneamente in nuova volatilità per il giorno successivo.
- β_1 **(Beta) = +0.775521**: Indica la persistenza. Suggerisce che il **78%** della volatilità odierna viene trascinata per inerzia al giorno seguente, riflettendo una memoria lunga degli shock finanziari.

Stability diagnostics

L'affidabilità del modello è confermata dal superamento dei test di stabilità matematica:

- **Persistenza** $(\alpha+\beta)$ **= 0.970827**: Il valore è inferiore a 1, garantendo che il modello sia **stazionario**. Lo spread tende quindi a riassorbire gli shock e a convergere verso la sua media nel lungo periodo.
- **Volatility Clustering**: La prossimità del valore a 1 conferma la tendenza della volatilità a manifestarsi a "blocchi" (periodi di alta tensione seguiti da periodi di calma).

Long-run indicators

- **Varianza Incondizionata**: 21.34
- **Volatilità Incondizionata (Daily)**: **4.61 bps**

Interpretazione: In condizioni di equilibrio, lo spread oscilla mediamente di circa 4.9 punti base al giorno. Variazioni significativamente superiori indicano una fase di stress che eccede la norma statistica del periodo analizzato.

5.4 Diagnostic

```r
show(garch_fit)
```

```
##
## *---------------------------------*
## *          GARCH Model Fit        *
## *---------------------------------*
##
## Conditional Variance Dynamics
## -----------------------------------
## GARCH Model  : sGARCH(1,1)
## Mean Model   : ARFIMA(0,0,0)
## Distribution : std
```

```
##
## Optimal Parameters
## ------------------------------------
##         Estimate  Std. Error  t value Pr(>|t|)
## mu     -0.064183    0.088137 -0.72822 0.466477
## omega   0.632398    0.235185  2.68894 0.007168
## alpha1  0.190795    0.043997  4.33655 0.000014
## beta1   0.779505    0.046833 16.64435 0.000000
## shape   7.737562    1.875982  4.12454 0.000037
##
## Robust Standard Errors:
##         Estimate  Std. Error  t value Pr(>|t|)
## mu     -0.064183    0.078710 -0.81544 0.414823
## omega   0.632398    0.321079  1.96960 0.048884
## alpha1  0.190795    0.060036  3.17803 0.001483
## beta1   0.779505    0.070642 11.03464 0.000000
## shape   7.737562    1.731373  4.46903 0.000008
##
## LogLikelihood : -2693.986
##
## Information Criteria
## ------------------------------------
##
## Akaike       5.2921
## Bayes        5.3163
## Shibata      5.2921
## Hannan-Quinn 5.3013
##
## Weighted Ljung-Box Test on Standardized Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                     0.1203  0.7287
## Lag[2*(p+q)+(p+q)-1][2]    0.1269  0.9012
## Lag[4*(p+q)+(p+q)-1][5]    1.2678  0.7967
## d.o.f=0
## H0 : No serial correlation
##
## Weighted Ljung-Box Test on Standardized Squared Residuals
## ------------------------------------
##                         statistic p-value
## Lag[1]                     0.8171  0.3660
## Lag[2*(p+q)+(p+q)-1][5]    1.5545  0.7265
## Lag[4*(p+q)+(p+q)-1][9]    2.8610  0.7817
## d.o.f=2
##
## Weighted ARCH LM Tests
## ------------------------------------
##             Statistic Shape Scale P-Value
## ARCH Lag[3]    0.5222 0.500 2.000  0.4699
## ARCH Lag[5]    0.7643 1.440 1.667  0.8039
## ARCH Lag[7]    0.9343 2.315 1.543  0.9240
##
## Nyblom stability test
## ------------------------------------
```

```
## Joint Statistic:  1.0111
## Individual Statistics:
## mu     0.33596
## omega  0.14287
## alpha1 0.30667
## beta1  0.39436
## shape  0.06151
##
## Asymptotic Critical Values (10% 5% 1%)
## Joint Statistic:          1.28 1.47 1.88
## Individual Statistic:     0.35 0.47 0.75
##
## Sign Bias Test
## ------------------------------------
##                    t-value   prob sig
## Sign Bias          0.32421 0.7458
## Negative Sign Bias 0.52932 0.5967
## Positive Sign Bias 0.00254 0.9980
## Joint Effect       1.01870 0.7967
##
##
## Adjusted Pearson Goodness-of-Fit Test:
## ------------------------------------
##   group statistic p-value(g-1)
## 1    20     19.88       0.4017
## 2    30     26.53       0.5971
## 3    40     37.02       0.5605
## 4    50     46.96       0.5562
##
##
## Elapsed time : 0.07245898
```

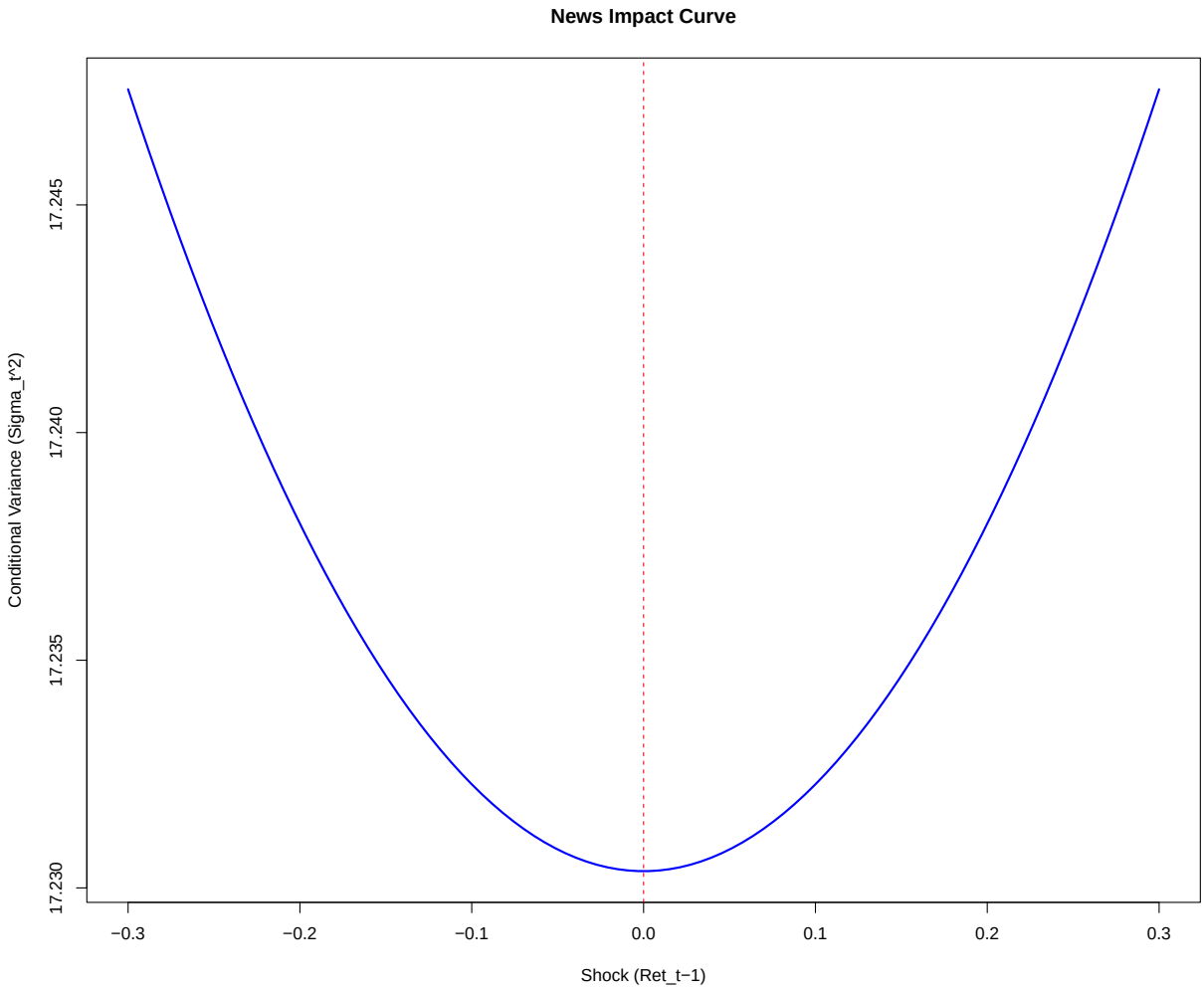
```r
infocriteria(garch_fit)*length(d_train)
```

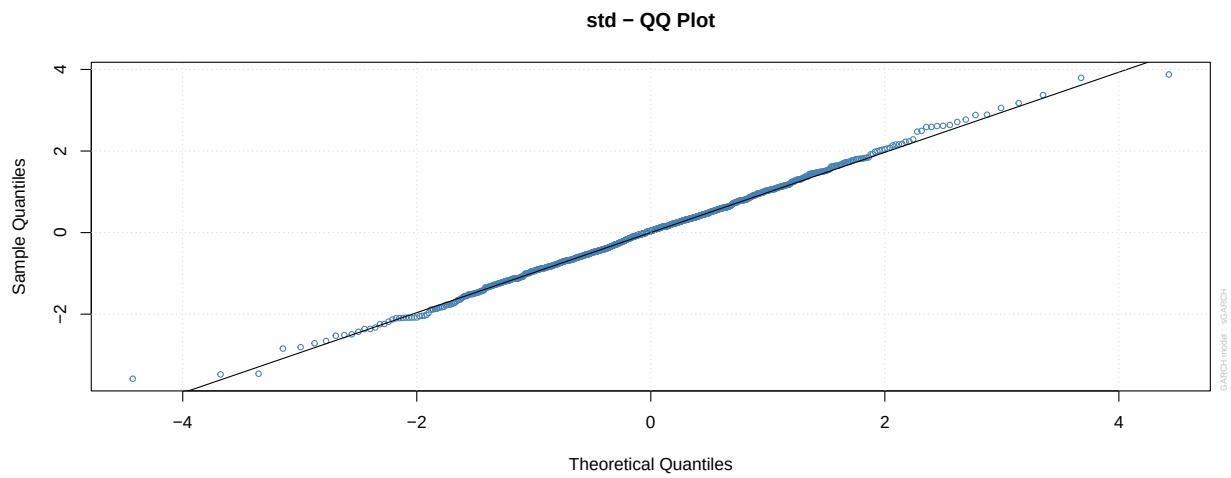
```
##
## Akaike       5397.973
## Bayes        5422.610
## Shibata      5397.924
## Hannan-Quinn 5407.328
```

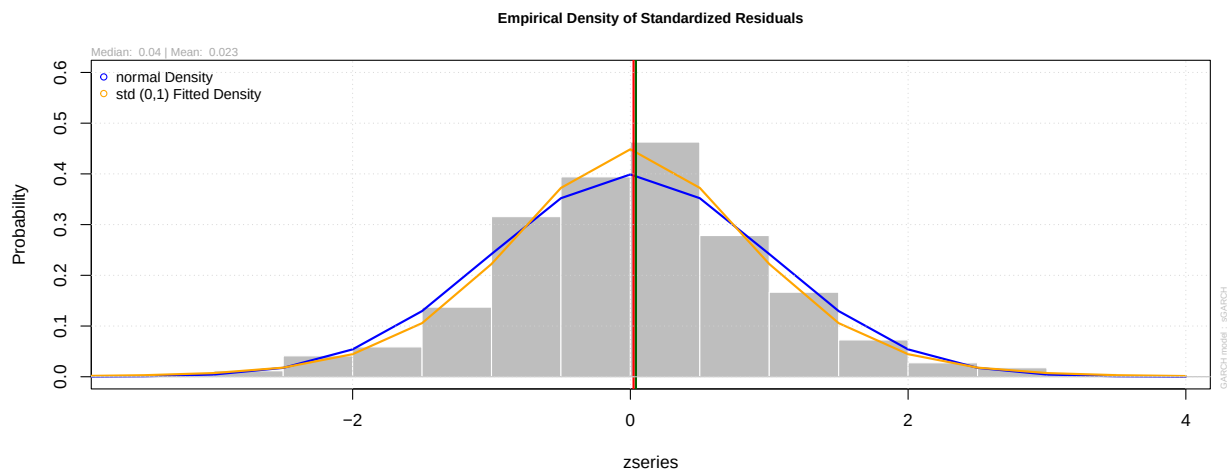
In the model output look at:

- "Weighted Ljung-Box Test on Standardized Residuals"
- "Weighted Ljung-Box Test on Standardized Squared Residuals"
- "Weighted ARCH LM Tests"
- "Adjusted Pearson Goodness-of-Fit Test" –> HO: residuals fit the supposed distribution
- "Sign Bias Test" –> HO: no sign bias in the standardized residuals

TO a visulization of the possible asymmetri effect of news we plot the NIC:


News Impact Curve
Conditional Variance (Sigma_t^2)
17.230
17.235
17.240
17.245
−0.3
−0.2
−0.1
0.0
0.1
0.2
0.3
Shock (Ret_t−1)





Assymetry test (Engle e Ng)

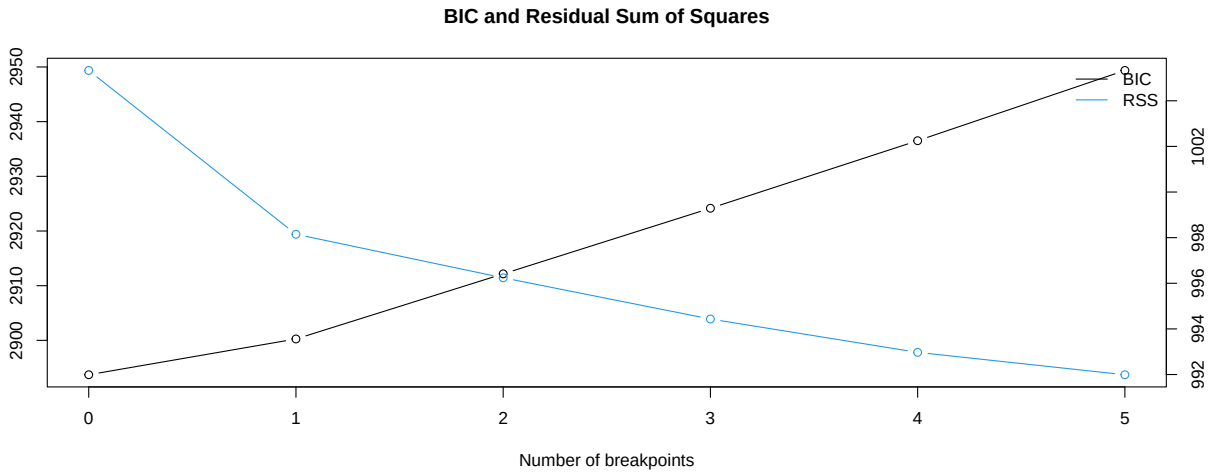
From the model output, the Sign Bias Test results are as follows: all p-values are well above the 0.05 threshold, indicating that we fail to reject the null hypothesis of no sign bias in the standardized residuals. The diagnostic tests confirm that the sGARCH specification with a Student-t distribution is sufficient to capture the volatility dynamics of the BTP-Bund spread. There is no significant evidence that positive and negative spread movements generate asymmetric volatility responses, suggesting that an asymmetric specification such as eGARCH is not warranted by the data.

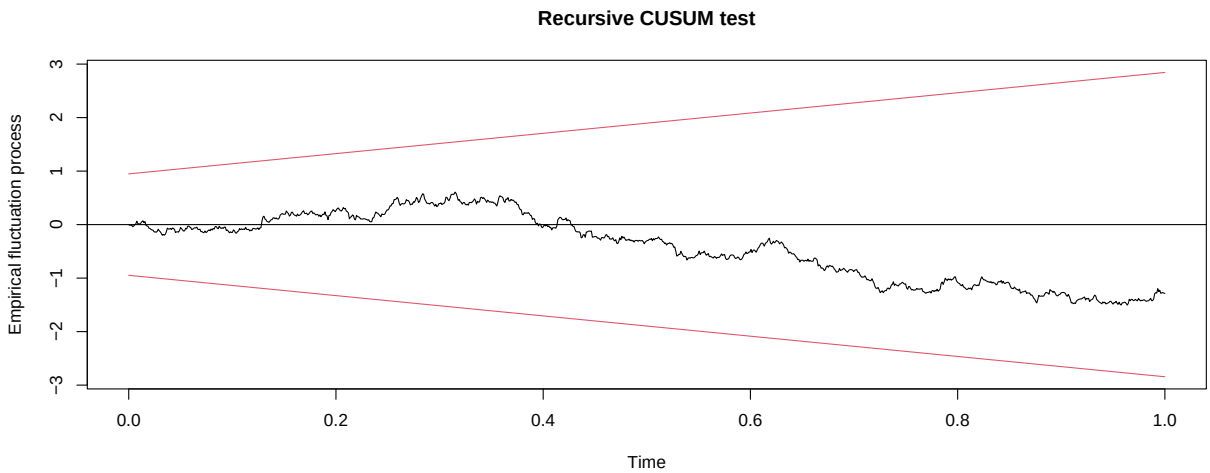
We also test if there's any breakpoint inside the residuals. We use:

- Bau-Perron test: tests for multiple unknow structural breaks in the mean, it find the best breaks location that minimizes the RSS, then selects the number of breaks minimizing BIC. The number of breaks with lowest BIC is the correct one.
- CUSUM: tests for parameter instability by tracking the cumulative sum of recursive residuals, a deviation of this sum from 0 means instability, H_0 : stable parameters throught the sample.

```
##
##   Optimal (m+1)-segment partition:
##
## Call:
```

```
## breakpoints.formula(formula = resid ~ 1, h = 0.15, data = bp_resid_df)
##
## Breakpoints at observation number:
##
## m = 1       366
## m = 2       366         740
## m = 3       366     631 785
## m = 4       322 478 631 785
## m = 5   166 322 478 631 785
##
## Corresponding to breakdates:
##
## m = 1                     0.358823529411765
## m = 2                     0.358823529411765
## m = 3                     0.358823529411765                   0.618627450980392
## m = 4                     0.315686274509804 0.468627450980392 0.618627450980392
## m = 5   0.162745098039216 0.315686274509804 0.468627450980392 0.618627450980392
##
## m = 1
## m = 2   0.725490196078431
## m = 3   0.769607843137255
## m = 4   0.769607843137255
## m = 5   0.769607843137255
##
## Fit:
##
## m   0      1      2      3      4      5
## RSS 1005.3  998.1  996.2  994.4  993.0  992.0
## BIC 2893.7 2900.3 2912.2 2924.2 2936.5 2949.4
```






```
##
##  Recursive CUSUM test
##
## data:  cusum_resid
## S = 0.53185, p-value = 0.5549
```

Checking the RSS table we can see that we have the optimal BIC at 0 breaks -> the residuals do not need breakpoint, adding any break will make the model worse. Also the RSS remain almost equal with differents possibles breaks: adding breaks will not explain anything more.

We fail to reject the null in the CUSUM test since the line in plot stay inside the confidence intervals -> parameters are stable.

Combined both tests tell us that there's no evidence of structural breaks in the standardize residuals, any possible change presents in the raw series is explained by the model.

5.4.1 Comment

The estimated model pass all the tests: there's no mean or volatility to model and the residual fit the supposed distribution (t-skewed) and pass the tests. Our data can be modelled with this ARMA-GARCH model, next we run a validation process for its forecasts and collect some metrics.

6 Validation

Now the validation phase, we take the best estimated ARMA-GARCH and validate through the test data. At every iteration we refit the model (re-estimate the parameter) but the structure of the model remain the same: the goal is to calculate how good the model is at forecasting to then forecast the real future value.

Then we compare the forecast with a Random Walk model to test it our model is actually useful.

- Forecast on test data using rolling forecast (window increases on 1 week every iteration)
- Refit the model every 10 iteration (10 days) to keep model useful, if every day it sometinme doesn't converge
- Perform the rolling estimation and forecast
- If window.size too small the model will fail to converge
- window.size = 252 -> last year of data

```
start_time <- proc.time()

c1 <- makeCluster(detectCores() - 1)

window_size = 252

set.seed(42)

roll <- ugarchroll(spec = garch_spec,
                   data = d_spread,
                   n.ahead = 1,
                   forecast.length = n_test,
                   refit.every = 10,
                   refit.window = "moving",
                   window.size = window_size,
                   solver = "hybrid",
                   solver.control = list(n.restarts = 5, rseed = 42, trace = 0),
                   calculate.VaR = TRUE,
                   VaR.alpha = c(0.01, 0.05, 0.95, 0.99), # calculate VaR using the GARCH model
                   cluster = c1)

stopCluster(c1)
# In case of non-convergence problems, re-run with other solvers
if (length(roll@model$nonconverged) > 0) {
  roll <- resume(roll, solver = "gosolnp")
}
if (length(roll@model$nonconverged) > 0) {
  roll <- resume(roll, solver = "nloptr")
}
if (length(roll@model$nonconverged) > 0) {
  roll <- resume(roll, solver = "solnp")
}

# Extract density predictions and realized spread
roll_df <- as.data.frame(roll)

# Extract theoretical quantiles
quants <- as.data.frame(roll, which = "VaR")
```

```r
# Build the forecast_df for better reading
dfc_df <- data.frame(
  pred_mu  = roll_df$Mu,
  pred_var = roll_df$Sigma^2,
  pred_vol = roll_df$Sigma,
  ci_q05  = quants[, "alpha(5%)"],
  ci_q95  = quants[, "alpha(95%)"],
  ci_q01  = quants[, "alpha(1%)"],
  ci_q99  = quants[, "alpha(99%)"],
  actual   = roll_df$Realized
)
rownames(dfc_df) <- rownames(roll_df)

end_time <- proc.time()
execution_time <- end_time - start_time
cat("\n Total execution time: ", execution_time[3], "seconds\n")
```

```
##
##  Total execution time:  2.17 seconds
```

=== VERSIONE PER WINDOWS + CORREZIONE (HO MESSO ANCHE I QUANTILI SUPERIORI PER IL CALCOLO DELLA DISTRIBUTION COVERAGE NEL CHUNCK SUCCESSIVO)

NOTE: OCCHIO A "ROLL.RDS", SE SI CAMBIA UN QUALCOSA NEL CODICE CHE PRECEDE (TIPO MODEL SPECIFICATION) "RICARICA" LA VECCHIA CACHE CON I VECCHI PARAMETRI ANDREBBE AGGIUNTO UN CHECK PER VEDERE LA CACHE E COERENTE CON I NUOVI DATI O AGGIORNARE LA CACHE OGNI VOLTA (?) ===

I test di copertura incondizionata (Kupiec Test) mostrano un comportamento divergente del modello ai due livelli di confidenza analizzati.

–> RIFLESSIONE SUI RIOSULTATI DEI VAR

6.1 Forecast evaluation

First we convert back to spread to have more readable metrics

```r
get_spreads <- function() {
  fc_spread <- numeric(n_test)
  lower_05 <- numeric(n_test)
  upper_95 <- numeric(n_test)
  lower_01 <- numeric(n_test)
  upper_99 <- numeric(n_test)

  for (i in 1:n_test) {
    # precedent spread value
    prev_spread <- as.numeric(spread[n_train + i - 1])

    fc <- dfc_df$pred_mu[i]
    fc_spread[i] <- prev_spread + fc

    ci_lower_05 <- dfc_df$ci_q05[i]
    ci_upper_95 <- dfc_df$ci_q95[i]
```

```r
    ci_lower_01 <- dfc_df$ci_q01[i]
    ci_upper_99 <- dfc_df$ci_q99[i]

    lower_05[i] <- prev_spread + ci_lower_05
    upper_95[i] <- prev_spread + ci_upper_95
    lower_01[i] <- prev_spread + ci_lower_01
    upper_99[i] <- prev_spread + ci_upper_99
  }

  act_spread <- as.numeric(spread[(n_train + 1):(n_train + n_test)])

  df <- data.frame(
    act_spread = act_spread,
    fc_spread = fc_spread,
    ci_q05 = lower_05,
    ci_q95 = upper_95,
    ci_q01 = lower_01,
    ci_q99 = upper_99
  )
  rownames(df) <- rownames(dfc_df)
  return(df)
}

fc_df <- get_spreads()
cat("Forecast spread vs actual spread:\n")
```

```
## Forecast spread vs actual spread:
```

```r
print(tail(fc_df))
```

```
##            act_spread fc_spread   ci_q05   ci_q95   ci_q01   ci_q99
## 2026-04-17       71.0  77.34546 72.58853 82.10239 70.07290 84.61802
## 2026-04-20       74.5  70.74410 64.06002 77.42818 60.42075 81.06746
## 2026-04-21       76.5  74.24410 67.77012 80.71809 64.24524 84.24297
## 2026-04-22       77.7  76.24410 70.30739 82.18082 67.07503 85.41317
## 2026-04-23       78.1  77.44410 72.09309 82.79511 69.17963 85.70857
## 2026-04-24       79.3  77.84410 73.07849 82.60971 70.48377 85.20444
```

Now the evaluation metrics on forecasted spread:

```r
errors <- fc_df$act_spread - fc_df$fc_spread

mae  <- mean(abs(as.numeric(errors)), na.rm = TRUE)
mse  <- mean(as.numeric(errors)^2, na.rm = TRUE)
rmse <- sqrt(mse)
mape <- mean(abs(as.numeric(errors) / as.numeric(dfc_df$actual)), na.rm = TRUE)
dir_acc <- mean(sign(as.numeric(dfc_df$actual)) == sign(as.numeric(dfc_df$pred_mu)), na.rm = TRUE)

cat("\nMean evaluation metrics:\n")
```

```
##
## Mean evaluation metrics:
```

```r
cat(sprintf("MAE: %.4f | MSE: %.4f | RMSE: %.4f | MAPE: %.4f | Dir. Acc: %.2f%%\n",
            mae, mse, rmse, mape, dir_acc * 100))
```

```
## MAE: 1.5593 | MSE: 4.9223 | RMSE: 2.2186 | MAPE: Inf | Dir. Acc: 54.12%
```

```r
# --- Distribution check  ---
coverage_90 <- mean(dfc_df$actual >= dfc_df$ci_q05 &
                    dfc_df$actual <= dfc_df$ci_q95, na.rm = TRUE)
coverage_98 <- mean(dfc_df$actual >= dfc_df$ci_q01 &
                    dfc_df$actual <= dfc_df$ci_q99, na.rm = TRUE)

cat("\nDistribution evaluation, % of forecast that fall into that interval:\n")
```

```
##
## Distribution evaluation, % of forecast that fall into that interval:
```

```r
cat(sprintf("90%% coverage: %.4f\n", coverage_90))
```

```
## 90% coverage: 0.9490
```

```r
cat("\nDistribution evaluation, % of forecast that fall into that interval:\n")
```

```
##
## Distribution evaluation, % of forecast that fall into that interval:
```

```r
cat(sprintf("98%% coverage: %.4f\n", coverage_98))
```

```
## 98% coverage: 0.9882
```

Compare those metrics with a Random Walk model (tomorrow's value = today's value). To see if our model is actually useful.

```r
spread_rw <- spread[n_train:(n_train + n_test - 1)] |> as.numeric()

fc_df$spread_rw <- spread_rw

mae_rw   <- mean(abs(spread_rw - fc_df$act_spread))
rmse_rw  <- sqrt(mean((spread_rw - fc_df$act_spread)^2))
mape_rw  <- mean(abs((spread_rw - fc_df$act_spread) / fc_df$act_spread))
dir_rw   <- mean(sign(diff(spread_rw)) == sign(diff(fc_df$act_spread)))

cat("\nEvalutation metrics, Model vs Random Walk:\n")
```

```
##
## Evalutation metrics, Model vs Random Walk:
```

```r
cat(sprintf("\nModel  | MAE: %.4f | RMSE: %.4f | MAPE: %.2f%% | Dir: %.2f%%\n",
            mae, rmse, mape*100, dir_acc*100))
```

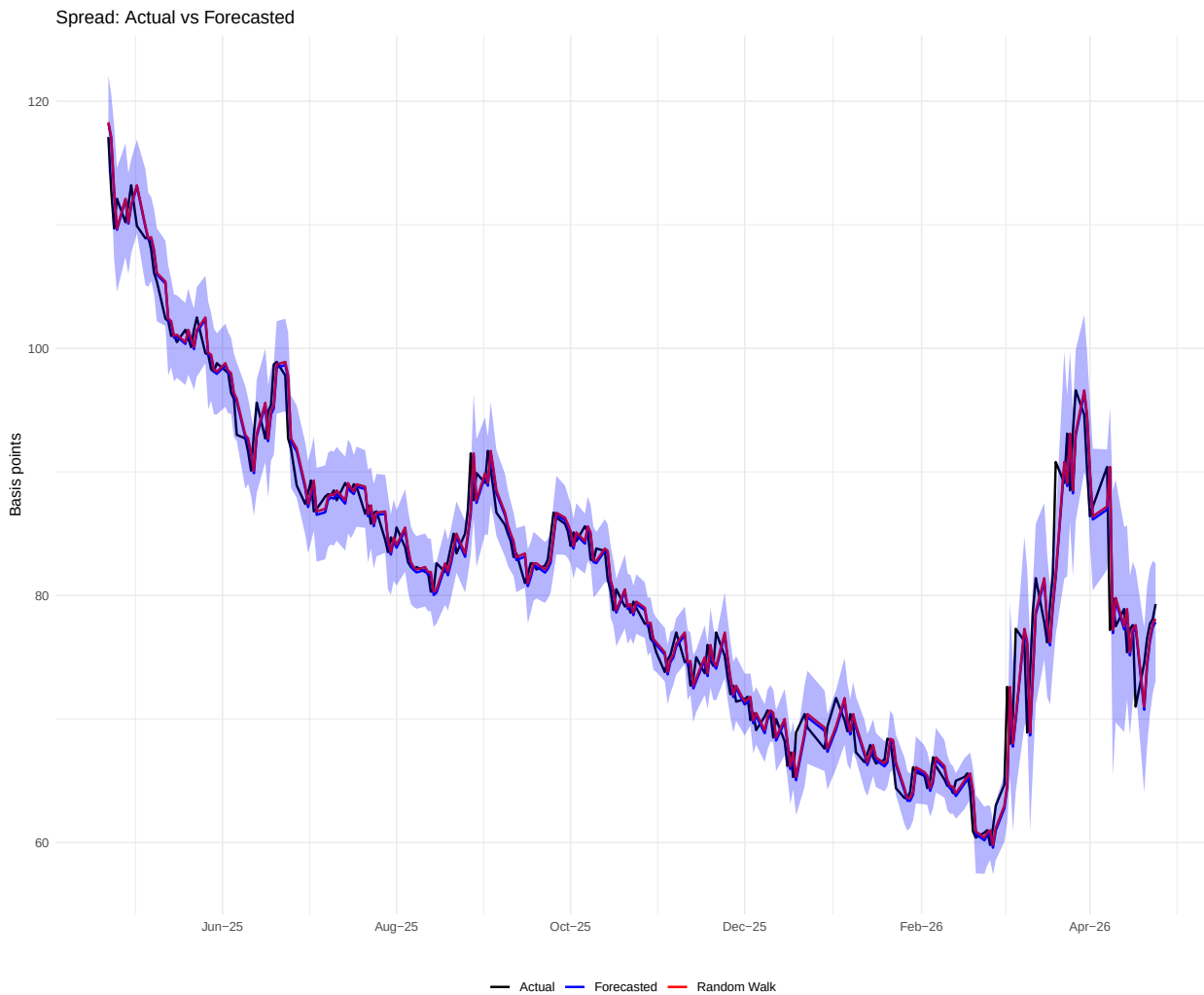
```
##
## Model  | MAE: 1.5593 | RMSE: 2.2186 | MAPE: Inf% | Dir: 54.12%
```

```r
cat(sprintf("RW    | MAE: %.4f | RMSE: %.4f | MAPE: %.2f%% | Dir: %.2f%%\n",
            mae_rw, rmse_rw, mape_rw*100, dir_rw*100))
```

```
## RW    | MAE: 1.5655 | RMSE: 2.2180 | MAPE: 1.94% | Dir: 47.64%
```

The metrics are similar,our model kind of act like a Random Walk.

Positive note: the accury direction is better.



7 Forecast

Forecast future value for next day, use in the same structure of the validated model.

Use the last `window_size` data (the same as validation phase)

```r
data_fit = tail(d_spread, window_size) # size used to validate
#last_date  <- last(index(data_fit))
last_date <- tail(index(data_fit), 1)
```

```r
set.seed(42)
future_fit <- ugarchfit(spec = garch_spec, data = data_fit, solver = "hybrid",
solver.control = list(n.restarts = 5, rseed = 42, trace = 0)
)

future_fc  <- ugarchforecast(future_fit, n.ahead = 1)

q_05 <- quantile(future_fc, probs = 0.05)
q_95 <- quantile(future_fc, probs = 0.95)

future_fc <- data.frame(
  d_spread = future_fc  |> fitted()  |> as.numeric(),
  vol = future_fc |> sigma()  |> as.numeric(),
  q_05 = q_05 |> as.numeric(),
  q_95 = q_95 |> as.numeric()
)

spread_conversion <- function(value, q_05, q_95, data){
    last_spread <- as.numeric(tail(data, 1))
    last_date <- data |> index() |> tail(1)

    spread <- last_spread + value
    q_05 <- last_spread + q_05
    q_95 <- last_spread + q_95

    df <- data.frame(
     spread = spread,
     q_05 = q_05,
     q_95 = q_95)

}

future_spread <- spread_conversion(future_fc$d_spread, future_fc$q_05, future_fc$q_95, spread)

cat("\nForecasted values:\n")
```

```
##
## Forecasted values:
```

```r
cat("\nSpread on last date: ", as.character(last_date), ": ", tail(spread, 1))
```

```
##
## Spread on last date:  2026-04-24 :  79.3
```

```r
cat("\nSpread forecasted for tomorrow:", as.character(last_date + 1), ":\n")
```

```
##
## Spread forecasted for tomorrow: 2026-04-25 :
```

```r
print(future_spread)
```

```
##     spread     q_05     q_95
## 1 79.06228 74.96571 83.15884
```

7.1 Simulation Forecast (10,000 paths)

Simulating 10,000 paths accounts for the GARCH shock uncertainty, not just the ARMA mean.

```
simulation <- ugarchsim(
  future_fit,
  n.sim       = 1,
  m.sim       = 10000,
  startMethod = "sample"
)

mean_sim <- mean(fitted(simulation))
vol_sim  <- mean(sigma(simulation))
q_90     <- quantile(fitted(simulation), probs = c(0.05, 0.95))

cat(sprintf("Mean DS:  %.4f\n", mean_sim))
```

```
## Mean DS:  -0.2145
```

```
cat(sprintf("Mean Vol: %.4f\n", vol_sim))
```

```
## Mean Vol: 2.5400
```

```
cat(sprintf("Quantiles 5%% | 95%%: %.4f | %.4f\n", q_90[1], q_90[2]))
```

```
## Quantiles 5% | 95%: -4.3077 | 3.8268
```

```
sim_spread <- spread_conversion(mean_sim, unname(q_90[1]), unname(q_90[2]), spread)
cat("Spread forecast (simulation) for tomorrow:", as.character(last_date + 1), ":\n")
```

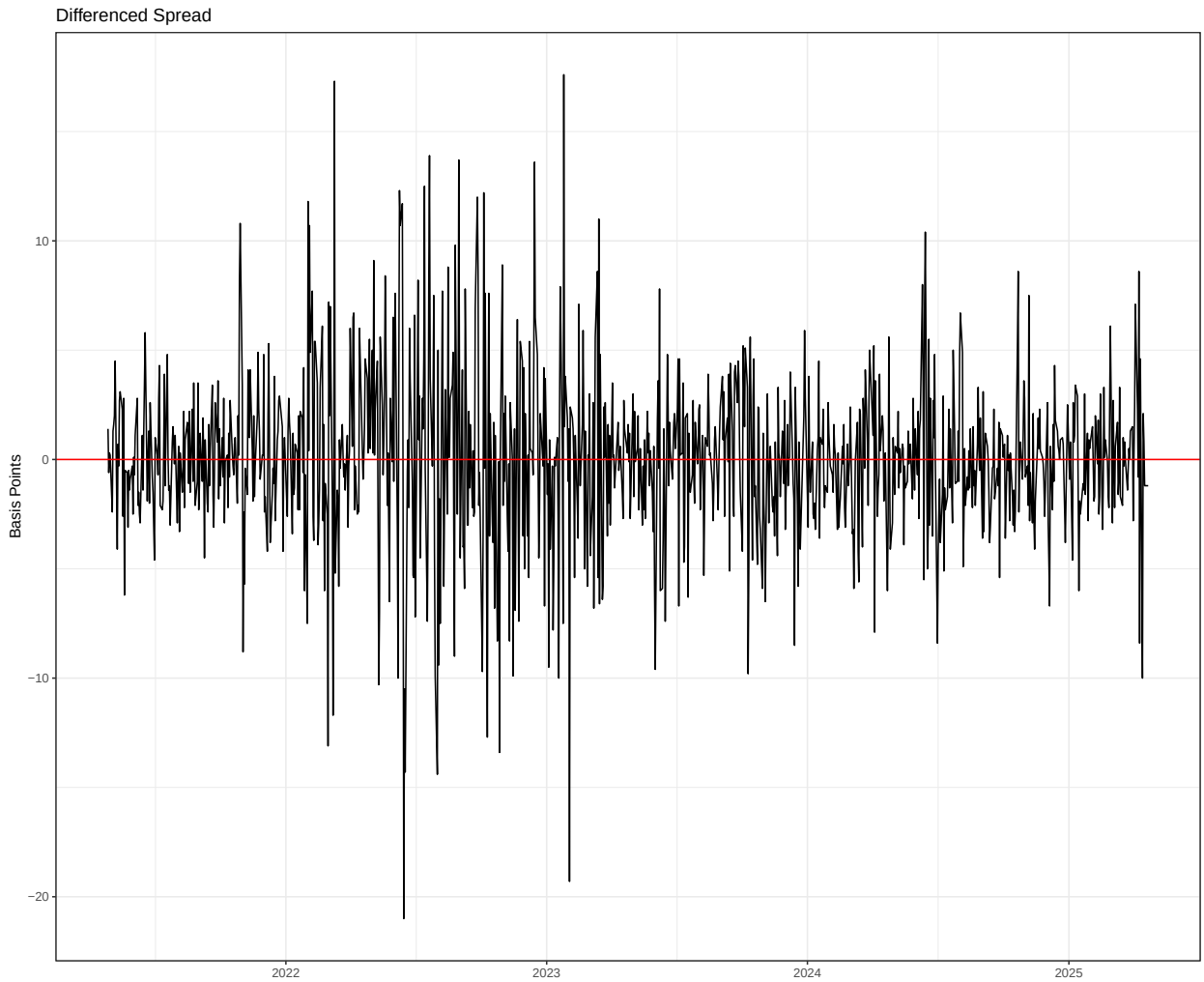
```
## Spread forecast (simulation) for tomorrow: 2026-04-25 :
```

```
print(sim_spread)
```

```
##     spread     q_05     q_95
## 1 79.08549 74.99233 83.12677
```

Se i risultati di sim_spread sono molto simili a quelli di future_spread (ottenuti con forecast), significa che il tuo modello è stabile. Se sono molto diversi, potrebbe esserci molta incertezza o asimmetria nella distribuzione dei rendimenti.

MULTIVARIATE MODEL



Multicollinearity Test

We now test for multicollinearity, so now we load the other series, and we make them stationary

```
# ==========================================
# PHASE 1: EXCEL DATA ACQUISITION & ALIGNMENT
# ==========================================

# Step 1: Load required libraries
#install.packages("readxl")
getwd()
```

```
## [1] "C:/Users/Jacopo/Downloads"
```

```
library(readxl)
library(xts)
```

```r
library(zoo)
# 1. Convertiamo il tuo oggetto xts 'train' in un data.frame per il join
df_spread <- data.frame(Data = index(spread), core_data = as.numeric(spread))
colnames(df_spread)[2] <- "Spread"

# 2. Unione progressiva (Left Join) partendo dal TRAIN
# Questo garantisce che terrai solo ed esattamente le date del tuo train attuale
df_multi <- df_spread %>%

# left_join(read_excel("vstoxx_24_04.xlsx") %>% dplyr::select(Data = 1, Vstoxx = 2) %>%
#             mutate(Data = as.Date(substr(as.character(Data), 1, 10))),
#             by = "Data") %>%

  left_join(read_excel("eurusd_24_04.xlsx") %>% dplyr::select(Data = 1, EURUSD = 2) %>%
             mutate(Data = as.Date(substr(as.character(Data), 1, 10))),
             by = "Data") %>%
# left_join(read_excel("FTSEMIB_17_04.xlsx") %>% select(Data = 1, Price_MIB = 2), by = "Data") %>%
# left_join(read_excel("CDSIT10Y_17_04.xlsx") %>% select(Data = 1, CDS10_IT = 2), by = "Data") %>%
  left_join(read_excel("euribor_3M_24_04.xlsx") %>% dplyr::select(Data = 1, EURIBOR3 = 2) %>%
             mutate(Data = as.Date(substr(as.character(Data), 1, 10))),
             by = "Data") %>%
# left_join(read_excel("EURBANKINDX_17_04.xlsx") %>% select(Data = 1, EUROABANKS = 2), by = "Data") %>%
# left_join(read_excel("ITBANKINDX_17_04.xlsx") %>% select(Data = 1, ITABANKS = 2), by = "Data") %>%
# left_join(read_excel("STOXX® Europe 600 Automobiles & Parts_17_04.xlsx") %>% select(Data = 1,
# Auto_Index = 2), by = "Data")%>%
# Ordinamento cronologico
  arrange(Data) %>%

  # 3. Creazione delle variabili trasformate (Rendimenti e Gap)
  mutate(across(-Data, as.numeric)) %>% # Trasforma tutto in numerico tranne la data
#   mutate(
    # Rendimenti logaritmici
#   ret_mib    = c(NA, diff(log(Price_MIB))),
#   ret_auto   = c(NA, diff(log(Auto_Index))),
   #itabanks vs eurobanks
#   bankratio= ITABANKS/ EUROABANKS,
#   ) %>%

    dplyr::select(
    Data,
    Spread,
    #Vstoxx,
    EURUSD,
    EURIBOR3
  )

# 2. Gestione NA con Interpolazione Lineare
# na.approx riempie i buchi (es. weekend o festività diverse tra mercati)
# senza eliminare l'intera riga.
df_multi <- df_multi %>%
  arrange(Data) %>%
  mutate(across(-Data, ~ na.approx(. , na.rm = FALSE))) %>%
  # Riempiamo eventuali NA rimasti all'inizio o alla fine con il valore più vicino
```

```r
  mutate(across(-Data, ~ na.locf(., na.rm = FALSE))) %>%
  mutate(across(-Data, ~ na.locf(., fromLast = TRUE, na.rm = FALSE)))

# Verifica finale
print(paste("Dataset creato con", nrow(df_multi), "osservazioni e", ncol(df_multi), "variabili."))
```

```
## [1] "Dataset creato con 1275 osservazioni e 4 variabili."
```

```r
sum(is.na(df_multi))
```

```
## [1] 0
```

```r
# 3. Test di Stazionarietà (ADF Test)
# Creiamo una funzione per testare ogni variabile
check_stationarity <- function(series) {
  test <- adf.test(series, alternative = "stationary")
  return(test$p.value)
}

print("--- Risultati Test ADF (Livelli) ---")
```

```
## [1] "--- Risultati Test ADF (Livelli) ---"
```

```r
variabili <- names(df_multi)[names(df_multi) != "Data"]
stazionarieta_risultati <- sapply(df_multi[variabili], check_stationarity)
```

```
## Warning in adf.test(series, alternative = "stationary"): p-value greater than
## printed p-value
```

```r
print(stazionarieta_risultati)
```

```
##    Spread    EURUSD  EURIBOR3
## 0.5405225 0.4993900 0.9900000
```

```r
# 4. Trasformazione in Differenze Prime (solo dove necessario)
# Se p-value > 0.05, la serie non è stazionaria e va differenziata
df_final_staz <- df_multi


for (var in variabili) {
  if (stazionarieta_risultati[var] > 0.01) {
    message(paste("Variabile", var, "non stazionaria. Calcolo differenze prime..."))
    # Sovrascriviamo con la differenza o creiamo nuova colonna
    # Qui creiamo 'd_NomeVariabile' per chiarezza nel Granger
    df_final_staz[[paste0("d_", var)]] <- c(NA, diff(df_multi[[var]]))
  } else {
    message(paste("Variabile", var, "già stazionaria."))
    df_final_staz[[paste0("d_", var)]] <- df_multi[[var]]
  }
}
```

```
## Variabile Spread non stazionaria. Calcolo differenze prime...

## Variabile EURUSD non stazionaria. Calcolo differenze prime...

## Variabile EURIBOR3 non stazionaria. Calcolo differenze prime...
```

```r
# Puliamo i nuovi NA generati dal comando diff()
df_final_staz <- df_final_staz[-1, ]
sum(is.na(df_final_staz))
```

```
## [1] 0
```

```r
df_final_staz<- df_final_staz %>%
  dplyr::select(Data, starts_with("d_"))

# 3. Create the Training set (First 80%)
n_rows <- nrow(df_final_staz)
train_var <- df_final_staz %>% slice(1:n_train)

# 4. Create the Testing set (Remaining 20%)
test_var <- df_final_staz %>% slice((n_train + 1):n_rows)
# Check the results
cat("Total observations:", n_rows, "\n")
```

```
## Total observations: 1274
```

```r
cat("Training set rows:", nrow(train_var), "(from", min(train_var$Data), "to", max(train_var$Data), ")\n
```

```
## Training set rows: 1020 (from 18744 to 20200 )
```

```r
cat("Testing set rows:", nrow(test_var), "(from", min(test_var$Data), "to", max(test_var$Data))
```

```
## Testing set rows: 254 (from 20201 to 20567
```

Now we should have a matrix with the d_spread value and the vstoxx index value, with matching data. We also have the train and test matrices.

```r
# 5. Granger Test con stampa estesa
target <- "d_Spread"
colonne_test <- names(train_var)[sapply(train_var, is.numeric)]
colonne_test <- colonne_test[colonne_test != target]

risultati_granger <- data.frame(Variabile = character(), P_Value = numeric())

cat("\n--- DETTAGLI TEST DI GRANGER (Lag = 5) ---\n")
```

```
##
## --- DETTAGLI TEST DI GRANGER (Lag = 5) ---
```

```r
for (var in colonne_test) {
  # Eseguiamo il test
  g_test <- grangertest(as.formula(paste(target, "~", var)), order = 5, data = train_var)

# Estraiamo il p-value
  p_val <- g_test$`Pr(>F)`[2]

  # Salviamo i risultati
  risultati_granger <- rbind(risultati_granger, data.frame(Variabile = var, P_Value = p_val))

  # Stampiamo l'esito a video con un po' di formattazione
  cat(paste0("\nTest: ", var, " causality on ", target, "\n"))
  print(g_test)
  cat("--------------------------------------------\n")
}
```

```
##
## Test: d_EURUSD causality on d_Spread
## Granger causality test
##
## Model 1: d_Spread ~ Lags(d_Spread, 1:5) + Lags(d_EURUSD, 1:5)
## Model 2: d_Spread ~ Lags(d_Spread, 1:5)
##   Res.Df Df      F Pr(>F)
## 1   1004
## 2   1009 -5 1.2775 0.2712
## --------------------------------------------
##
## Test: d_EURIBOR3 causality on d_Spread
## Granger causality test
##
## Model 1: d_Spread ~ Lags(d_Spread, 1:5) + Lags(d_EURIBOR3, 1:5)
## Model 2: d_Spread ~ Lags(d_Spread, 1:5)
##   Res.Df Df      F Pr(>F)
## 1   1004
## 2   1009 -5 2.7908 0.0164 *
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
## --------------------------------------------
```

```r
# 6. Analisi CCF: Lag Predittivi e Intervalli di Confidenza
target <- "d_Spread"
variabili_ccf <- names(train_var)[sapply(train_var, is.numeric)]
variabili_ccf <- variabili_ccf [colonne_test != target]
# Calcolo del numero di osservazioni e della soglia IC (95%)
n_obs <- nrow(train_var)
soglia_ic <- 1.96 / sqrt(n_obs)

cat("\n=========================================================\n")
```

```
##
## ========================================================
```

```r
cat("   ANALISI DETTAGLIATA LAG PREDITTIVI (X precede Spread)   \n")
```

```
##    ANALISI DETTAGLIATA LAG PREDITTIVI (X precede Spread)
```

```r
cat("==========================================================\n")
```

```
## ==========================================================
```

```r
cat(paste("Numero osservazioni:", n_obs, "\n"))
```

```
## Numero osservazioni: 1020
```

```r
cat(paste("Soglia IC (95%): +/-", round(soglia_ic, 4), "\n"))
```

```
## Soglia IC (95%): +/- 0.0614
```

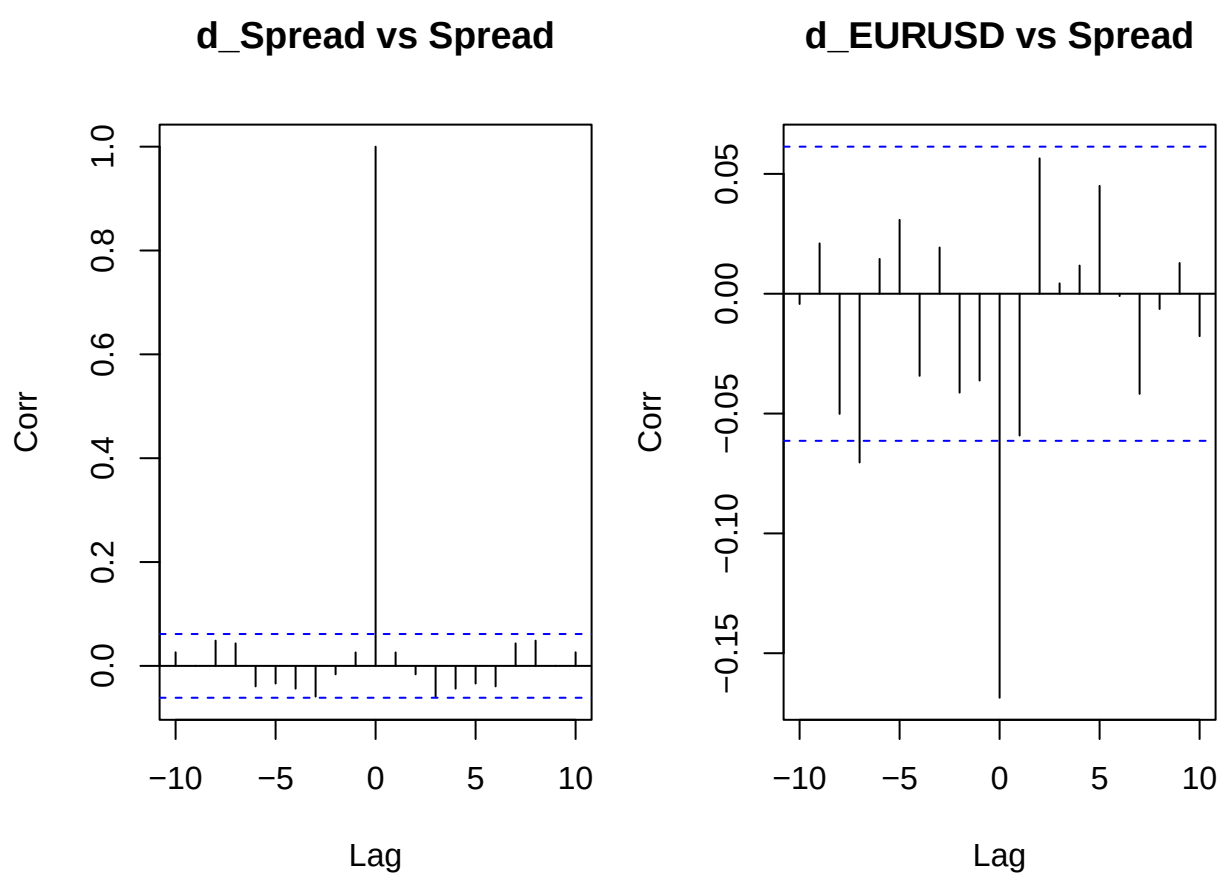
```r
cat("Nota: Un lag k < 0 indica che X al tempo t-|k| correla con Spread al tempo t\n")
```

```
## Nota: Un lag k < 0 indica che X al tempo t-|k| correla con Spread al tempo t
```

```r
# Tabella riassuntiva per i risultati
final_ccf_summary <- data.frame(Variabile = character(),
                                Lag = numeric(),
                                Correlazione = numeric())

# Setup layout: 1 grafico per riga (o puoi fare par(mfrow=c(2,2)) fuori dal ciclo)
# Setup layout: 3 righe e 2 colonne
par(mfrow = c(1, 2), mar = c(4, 4, 3, 1))

for (var in variabili_ccf) {
  # 1. Calcolo della CCF
  ccf_result <- ccf(train_var[[var]], train_var[[target]], lag.max = 10, plot = FALSE)
  # 3. Generazione del PLOT
  # Usiamo il risultato già calcolato per fare il plot
  plot(ccf_result,
       main = paste(var, "vs Spread"),
       ylab = "Corr", xlab = "Lag",
       ci.col = "blue")
  # Estrazione dati
  lags <- ccf_result$lag[,1,1]
  corrs <- ccf_result$acf[,1,1]

  # Filtriamo: solo lag < 0 (X predice Spread)
  idx_p <- which(lags < 0)
  lags_p <- lags[idx_p]
  corrs_p <- corrs[idx_p]

  # Verifichiamo quali superano l'Intervallo di Confidenza
  sig_idx <- which(abs(corrs_p) >= soglia_ic)
```

```r
  if (length(sig_idx) > 0) {
    cat(paste0("\n>>> Variabile: ", var, "\n"))

    # 2. Creazione della tabella dei lag significativi
    df_temp <- data.frame(
      Lag = lags_p[sig_idx],
      Correlazione = round(corrs_p[sig_idx], 4)
    ) %>% arrange(Lag)

    print(df_temp)

    # Salvataggio nel riassunto globale
    df_temp$Variabile <- var
    final_ccf_summary <- rbind(final_ccf_summary, df_temp)

  } else {
    cat(paste0("\nVariabile: ", var, " - Nessun lag predittivo significativo.\n"))
  }
}
```

```
##
## Variabile: d_Spread - Nessun lag predittivo significativo.
```




```
##
```

```
## >>> Variabile: d_EURUSD
##   Lag Correlazione
## 1  -7      -0.0704
```

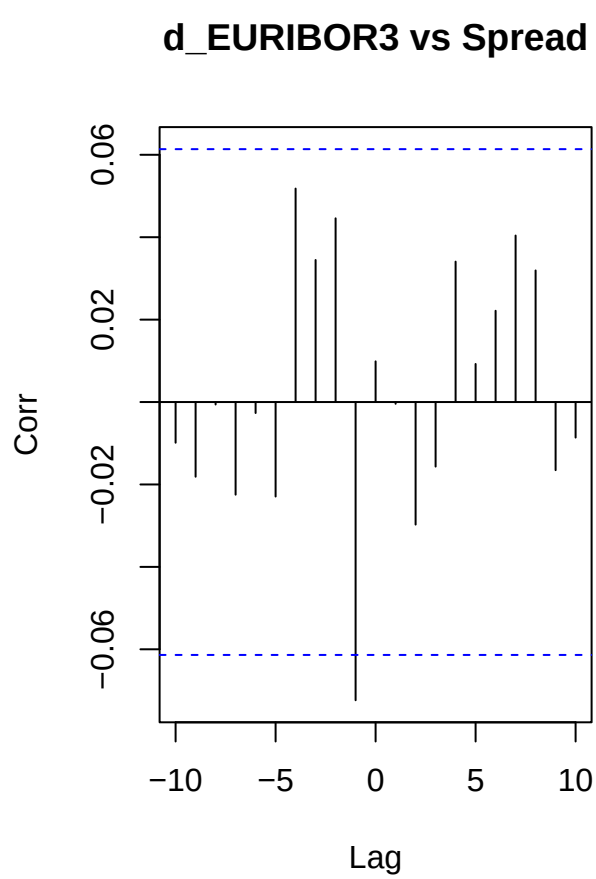
```
##
## >>> Variabile: d_EURIBOR3
##   Lag Correlazione
## 1  -1      -0.0724
```

```r
# 8. Selezione finale delle variabili basata su CCF
# Teniamo solo le variabili che hanno almeno un lag predittivo (escluso lo 0 se cerchi solo causalità)
variabili_da_usare_ccf <- unique(final_ccf_summary$Variabile)

cat("\n--- Selezione Finale basata su CCF ---\n")
```

```
##
## --- Selezione Finale basata su CCF ---
```

```r
print(variabili_da_usare_ccf)
```

```
## [1] "d_EURUSD"   "d_EURIBOR3"
```



MQ Test

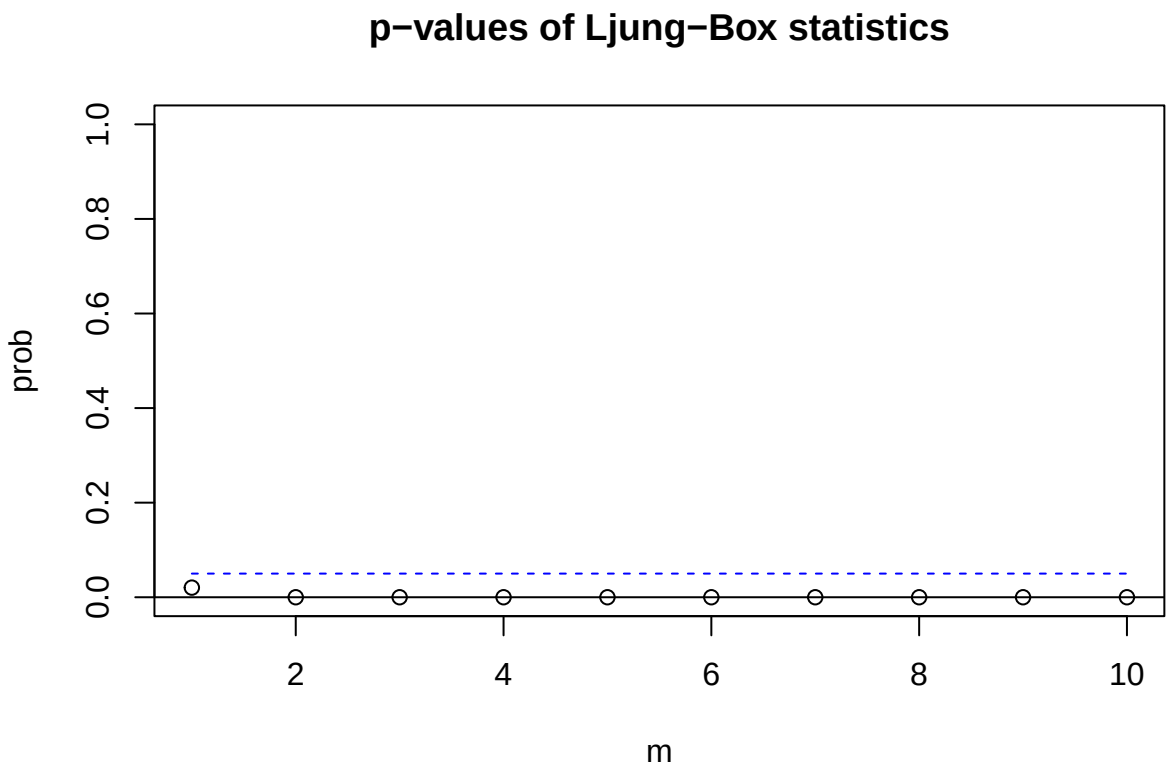
We have both series stationary, we now run the qm test to see if we need a VAR model.

```r
library(MTS)
library(vars)
```

```
##
## Attaching package: 'vars'

## The following object is masked from 'package:MTS':
##
##     VAR
```

```r
# MQ test
mq(train_var[, -1], lag = 10)
```

```
## Ljung-Box Statistics:
##         m       Q(m)     df    p-value
##  [1,]   1.0      19.6     9.0     0.02
##  [2,]   2.0      59.5    18.0     0.00
##  [3,]   3.0      85.9    27.0     0.00
##  [4,]   4.0     100.7    36.0     0.00
##  [5,]   5.0     127.3    45.0     0.00
##  [6,]   6.0     135.3    54.0     0.00
##  [7,]   7.0     156.4    63.0     0.00
##  [8,]   8.0     182.9    72.0     0.00
##  [9,]   9.0     197.2    81.0     0.00
## [10,]  10.0     220.7    90.0     0.00
```




```
# We have all lags in which p-value is less than 0.05 ( equal to 0), so we can estimate a VAR model.
```

VAR Model Estimation

```
library(vars)
# 1. Determine optimal VAR lag length
lag_selection <- VARselect(train_var[, -1], lag.max = 10, type = "const")
lag_selection
```

```
## $selection
## AIC(n)  HQ(n)  SC(n) FPE(n)
##      5      2      1      5
##
## $criteria
##                        1                 2                 3                 4
## AIC(n) -15.4234914140027 -15.4448556850197 -15.4494034579794 -15.4478611495774
## HQ(n)  -15.4012953548650 -15.4060125815287 -15.3939133101351 -15.3757239573798
## SC(n)  -15.3650632285393 -15.3426063604588 -15.3033329943210 -15.2579695468214
## FPE(n)   0.0000002002916   0.0000001960581   0.0000001951687   0.0000001954704
##                        5                 6                 7                 8
## AIC(n) -15.4513815786496 -15.4384137882970 -15.435468867039 -15.4321723532106
## HQ(n)  -15.3625973420987 -15.3329825073928 -15.313390541782 -15.2934469835998
## SC(n)  -15.2176688367960 -15.1608799073459 -15.114113846991 -15.0669961940644
```

```
## FPE(n)   0.0000001947842   0.0000001973276   0.000000197911   0.0000001985665
##                        9                10
## AIC(n) -15.4227178458266 -15.4152258930042
## HQ(n)  -15.2673454318625 -15.2432064346868
## SC(n)  -15.0137205475829 -14.9624074556629
## FPE(n)   0.0000002004552   0.0000002019657
```

```r
# We have optimal lag = 1 chosen by all the models

# Fit the model
var_bic <-vars::VAR(train_var[, -1], p = 2, type = "const")
summary(var_bic)
```

```
##
## VAR Estimation Results:
## =========================
## Endogenous variables: d_Spread, d_EURUSD, d_EURIBOR3
## Deterministic variables: const
## Sample size: 1018
## Log Likelihood: 3553.658
## Roots of the characteristic polynomial:
## 0.4369 0.3533 0.2532 0.2532 0.2342 0.2342
## Call:
## vars::VAR(y = train_var[, -1], p = 2, type = "const")
##
##
## Estimation results for equation d_Spread:
## =========================================
## d_Spread = d_Spread.l1 + d_EURUSD.l1 + d_EURIBOR3.l1 + d_Spread.l2 + d_EURUSD.l2 + d_EURIBOR3.l2 + c
##
##                Estimate Std. Error t value Pr(>|t|)
## d_Spread.l1     0.02301    0.03180   0.724   0.4694
## d_EURUSD.l1   -30.29865   23.70688  -1.278   0.2015
## d_EURIBOR3.l1 -13.94782    5.58282  -2.498   0.0126 *
## d_Spread.l2    -0.02677    0.03177  -0.843   0.3996
## d_EURUSD.l2   -31.18748   23.65352  -1.319   0.1876
## d_EURIBOR3.l2   9.51334    5.62521   1.691   0.0911 .
## const           0.01728    0.12393   0.139   0.8891
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 3.899 on 1011 degrees of freedom
## Multiple R-Squared: 0.0123,  Adjusted R-squared: 0.006435
## F-statistic: 2.098 on 6 and 1011 DF,  p-value: 0.05106
##
##
## Estimation results for equation d_EURUSD:
## =========================================
## d_EURUSD = d_Spread.l1 + d_EURUSD.l1 + d_EURIBOR3.l1 + d_Spread.l2 + d_EURUSD.l2 + d_EURIBOR3.l2 + c
##
##                  Estimate  Std. Error t value Pr(>|t|)
## d_Spread.l1   -0.00008631  0.00004270  -2.021   0.0435 *
## d_EURUSD.l1    0.00388083  0.03183262   0.122   0.9030
```

```
## d_EURIBOR3.l1  0.01560495  0.00749638   2.082   0.0376 *
## d_Spread.l2    0.00006721  0.00004266   1.576   0.1154
## d_EURUSD.l2   -0.05106705  0.03176097  -1.608   0.1082
## d_EURIBOR3.l2 -0.00548994  0.00755330  -0.727   0.4675
## const         -0.00009889  0.00016641  -0.594   0.5525
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.005236 on 1011 degrees of freedom
## Multiple R-Squared: 0.01383, Adjusted R-squared: 0.007978
## F-statistic: 2.363 on 6 and 1011 DF,  p-value: 0.02845
##
##
## Estimation results for equation d_EURIBOR3:
## ===========================================
## d_EURIBOR3 = d_Spread.l1 + d_EURUSD.l1 + d_EURIBOR3.l1 + d_Spread.l2 + d_EURUSD.l2 + d_EURIBOR3.l2 +
##
##                 Estimate Std. Error t value    Pr(>|t|)
## d_Spread.l1    0.0000680  0.0001769   0.384     0.70072
## d_EURUSD.l1   -0.0203526  0.1318547  -0.154     0.87736
## d_EURIBOR3.l1  0.0625102  0.0310509   2.013     0.04436 *
## d_Spread.l2   -0.0001468  0.0001767  -0.831     0.40631
## d_EURUSD.l2    0.1420859  0.1315579   1.080     0.28039
## d_EURIBOR3.l2  0.1643157  0.0312867   5.252 0.000000183 ***
## const          0.0020715  0.0006893   3.005     0.00272 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.02169 on 1011 degrees of freedom
## Multiple R-Squared: 0.03361, Adjusted R-squared: 0.02787
## F-statistic:  5.86 on 6 and 1011 DF,  p-value: 0.000005082
##
##
##
## Covariance matrix of residuals:
##             d_Spread     d_EURUSD   d_EURIBOR3
## d_Spread   15.203362 -0.003357494  0.000652969
## d_EURUSD   -0.003357  0.000027412 -0.000005083
## d_EURIBOR3  0.000653 -0.000005083  0.000470307
##
## Correlation matrix of residuals:
##             d_Spread d_EURUSD d_EURIBOR3
## d_Spread    1.000000 -0.16447   0.007722
## d_EURUSD   -0.164467  1.00000  -0.044766
## d_EURIBOR3  0.007722 -0.04477   1.000000
```

```r
var_bic_mts <- MTS::VAR(train_var[, -1], p = 2)#equivalent with MTs package
```

```
## Constant term:
## Estimates:  0.01728062 -0.00009888765 0.002071492
## Std.Error:  0.1239324 0.0001664113 0.0006892966
## AR coefficient matrix
```

```
## AR( 1 )-matrix
##            [,1]      [,2]     [,3]
## [1,]  0.0230128 -30.29865 -13.9478
## [2,] -0.0000863   0.00388   0.0156
## [3,]  0.0000680  -0.02035   0.0625
## standard error
##           [,1]    [,2]   [,3]
## [1,] 0.0318010 23.7069 5.5828
## [2,] 0.0000427  0.0318 0.0075
## [3,] 0.0001769  0.1319 0.0311
## AR( 2 )-matrix
##            [,1]     [,2]     [,3]
## [1,] -0.0267700 -31.1875  9.51334
## [2,]  0.0000672  -0.0511 -0.00549
## [3,] -0.0001468   0.1421  0.16432
## standard error
##           [,1]    [,2]    [,3]
## [1,] 0.0317675 23.6535 5.62521
## [2,] 0.0000427  0.0318 0.00755
## [3,] 0.0001767  0.1316 0.03129
##
## Residuals cov-mtx:
##              [,1]            [,2]            [,3]
## [1,] 15.098820173 -0.003334406907  0.000648478995
## [2,] -0.003334407  0.000027223180 -0.000005047912
## [3,]  0.000648479 -0.000005047912  0.000467073548
##
## det(SSE) =  0.0000001864175
## AIC =  -15.45998
## BIC =  -15.37303
## HQ  =  -15.42696
```

```r
var_aic<-vars::VAR(train_var[, -1], p = 3, type = "const")
summary(var_aic)
```

```
##
## VAR Estimation Results:
## =========================
## Endogenous variables: d_Spread, d_EURUSD, d_EURIBOR3
## Deterministic variables: const
## Sample size: 1017
## Log Likelihood: 3560.052
## Roots of the characteristic polynomial:
## 0.594 0.4252 0.4252 0.4133 0.4133 0.326 0.2508 0.2508 0.08928
## Call:
## vars::VAR(y = train_var[, -1], p = 3, type = "const")
##
##
## Estimation results for equation d_Spread:
## =========================================
## d_Spread = d_Spread.l1 + d_EURUSD.l1 + d_EURIBOR3.l1 + d_Spread.l2 + d_EURUSD.l2 + d_EURIBOR3.l2 + d
##
##                Estimate Std. Error t value Pr(>|t|)
## d_Spread.l1     0.02064    0.03185   0.648  0.51709
```

```
## d_EURUSD.l1   -27.22177   23.76506  -1.145  0.25229
## d_EURIBOR3.l1 -15.62137    5.65567  -2.762  0.00585 **
## d_Spread.l2    -0.02207    0.03184  -0.693  0.48842
## d_EURUSD.l2   -35.76982   23.70903  -1.509  0.13169
## d_EURIBOR3.l2   8.87263    5.63337   1.575  0.11557
## d_Spread.l3    -0.06001    0.03179  -1.887  0.05939 .
## d_EURUSD.l3     8.83653   23.76322   0.372  0.71008
## d_EURIBOR3.l3   8.37575    5.70662   1.468  0.14249
## const           0.00133    0.12449   0.011  0.99148
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 3.895 on 1007 degrees of freedom
## Multiple R-Squared: 0.01818, Adjusted R-squared: 0.00941
## F-statistic: 2.072 on 9 and 1007 DF,  p-value: 0.02935
##
##
## Estimation results for equation d_EURUSD:
## =========================================
## d_EURUSD = d_Spread.l1 + d_EURUSD.l1 + d_EURIBOR3.l1 + d_Spread.l2 + d_EURUSD.l2 + d_EURIBOR3.l2 + d_
##
##                   Estimate   Std. Error t value Pr(>|t|)
## d_Spread.l1   -0.000089536  0.000042874  -2.088   0.0370 *
## d_EURUSD.l1    0.002874883  0.031988100   0.090   0.9284
## d_EURIBOR3.l1  0.014643785  0.007612614   1.924   0.0547 .
## d_Spread.l2    0.000068845  0.000042855   1.606   0.1085
## d_EURUSD.l2   -0.052795073  0.031912692  -1.654   0.0984 .
## d_EURIBOR3.l2 -0.005913133  0.007582600  -0.780   0.4357
## d_Spread.l3   -0.000003898  0.000042794  -0.091   0.9274
## d_EURUSD.l3   -0.018586766  0.031985624  -0.581   0.5613
## d_EURIBOR3.l3  0.006647004  0.007681192   0.865   0.3870
## const         -0.000114552  0.000167568  -0.684   0.4944
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.005243 on 1007 degrees of freedom
## Multiple R-Squared: 0.01495, Adjusted R-squared: 0.006145
## F-statistic: 1.698 on 9 and 1007 DF,  p-value: 0.08506
##
##
## Estimation results for equation d_EURIBOR3:
## ===========================================
## d_EURIBOR3 = d_Spread.l1 + d_EURUSD.l1 + d_EURIBOR3.l1 + d_Spread.l2 + d_EURUSD.l2 + d_EURIBOR3.l2 +
##
##                  Estimate  Std. Error t value    Pr(>|t|)
## d_Spread.l1    0.00002303  0.00017643   0.131    0.896173
## d_EURUSD.l1   -0.02700339  0.13163098  -0.205    0.837501
## d_EURIBOR3.l1  0.04577222  0.03132589   1.461    0.144283
## d_Spread.l2   -0.00011320  0.00017635  -0.642    0.521065
## d_EURUSD.l2    0.11132434  0.13132068   0.848    0.396790
## d_EURIBOR3.l2  0.15712065  0.03120239   5.036 0.000000564 ***
## d_Spread.l3   -0.00011387  0.00017610  -0.647    0.517997
```

```
## d_EURUSD.l3   -0.18346677  0.13162079  -1.394    0.163654
## d_EURIBOR3.l3  0.10888197  0.03160809   3.445    0.000595 ***
## const          0.00182474  0.00068954   2.646    0.008264 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.02157 on 1007 degrees of freedom
## Multiple R-Squared: 0.04727, Adjusted R-squared: 0.03875
## F-statistic: 5.551 on 9 and 1007 DF,  p-value: 0.000000171
##
##
##
## Covariance matrix of residuals:
##             d_Spread     d_EURUSD   d_EURIBOR3
## d_Spread   15.172684 -0.003391637  0.000217979
## d_EURUSD   -0.003392  0.000027489 -0.000005549
## d_EURIBOR3  0.000218 -0.000005549  0.000465480
##
## Correlation matrix of residuals:
##             d_Spread d_EURUSD d_EURIBOR3
## d_Spread    1.000000 -0.16607   0.002594
## d_EURUSD   -0.166072  1.00000  -0.049057
## d_EURIBOR3  0.002594 -0.04906   1.000000
```

```r
var_aic_mts <- MTS::VAR(train_var[, -1], p = 3)#equivalent with MTS package
```

```
## Constant term:
## Estimates:  0.001330417 -0.0001145522 0.001824745
## Std.Error:  0.1244922 0.0001675682 0.0006895428
## AR coefficient matrix
## AR( 1 )-matrix
##            [,1]      [,2]     [,3]
## [1,]  0.0206424 -27.22177 -15.6214
## [2,] -0.0000895   0.00287   0.0146
## [3,]  0.0000230  -0.02700   0.0458
## standard error
##           [,1]   [,2]    [,3]
## [1,] 0.0318526 23.765 5.65567
## [2,] 0.0000429  0.032 0.00761
## [3,] 0.0001764  0.132 0.03133
## AR( 2 )-matrix
##            [,1]     [,2]     [,3]
## [1,] -0.0220667 -35.7698  8.87263
## [2,]  0.0000688  -0.0528 -0.00591
## [3,] -0.0001132   0.1113  0.15712
## standard error
##           [,1]    [,2]    [,3]
## [1,] 0.0318386 23.7090 5.63337
## [2,] 0.0000429  0.0319 0.00758
## [3,] 0.0001763  0.1313 0.03120
## AR( 3 )-matrix
##            [,1]    [,2]    [,3]
## [1,] -0.0600075  8.8365 8.37575
```

```
## [2,] -0.0000039 -0.0186 0.00665
## [3,] -0.0001139 -0.1835 0.10888
## standard error
##           [,1]   [,2]    [,3]
## [1,] 0.0317928 23.763 5.70662
## [2,] 0.0000428  0.032 0.00768
## [3,] 0.0001761  0.132 0.03161
##
## Residuals cov-mtx:
##              [,1]            [,2]            [,3]
## [1,] 15.023492945 -0.003358287797  0.000215835974
## [2,] -0.003358288  0.000027218869 -0.000005494696
## [3,]  0.000215836 -0.000005494696  0.000460902893
##
## det(SSE) =  0.0000001828286
## AIC =  -15.46178
## BIC =  -15.33134
## HQ  =  -15.41225
```

BIC suggests 2 lags. #All coefficients ar significant except the one of spread itself

```r
var_restricted_bic = vars::restrict(var_bic, method = "ser", thresh = 1.64)
summary(var_restricted_bic)
```

```
##
## VAR Estimation Results:
## =========================
## Endogenous variables: d_Spread, d_EURUSD, d_EURIBOR3
## Deterministic variables: const
## Sample size: 1018
## Log Likelihood: 3548.061
## Roots of the characteristic polynomial:
## 0.4343 0.3711     0     0     0     0
## Call:
## vars::VAR(y = train_var[, -1], p = 2, type = "const")
##
##
## Estimation results for equation d_Spread:
## =========================================
## d_Spread = d_EURIBOR3.l1 + d_EURIBOR3.l2
##
##               Estimate Std. Error t value Pr(>|t|)
## d_EURIBOR3.l1  -13.449      5.540  -2.428   0.0154 *
## d_EURIBOR3.l2    9.130      5.556   1.643   0.1006
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 3.898 on 1016 degrees of freedom
## Multiple R-Squared: 0.007755,    Adjusted R-squared: 0.005801
## F-statistic:  3.97 on 2 and 1016 DF,  p-value: 0.01916
##
##
```

```
## Estimation results for equation d_EURUSD:
## =========================================
## d_EURUSD = d_Spread.l1 + d_EURIBOR3.l1 + d_Spread.l2
##
##                  Estimate  Std. Error t value Pr(>|t|)
## d_Spread.l1   -0.00008273  0.00004198  -1.971   0.0490 *
## d_EURIBOR3.l1  0.01450737  0.00740824   1.958   0.0505 .
## d_Spread.l2    0.00007799  0.00004197   1.858   0.0634 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.005234 on 1015 degrees of freedom
## Multiple R-Squared: 0.01063, Adjusted R-squared: 0.00771
## F-statistic: 3.636 on 3 and 1015 DF,  p-value: 0.01253
##
##
## Estimation results for equation d_EURIBOR3:
## ===========================================
## d_EURIBOR3 = d_EURIBOR3.l1 + d_EURIBOR3.l2 + const
##
##                Estimate Std. Error t value    Pr(>|t|)
## d_EURIBOR3.l1 0.0631992  0.0309833   2.040     0.04163 *
## d_EURIBOR3.l2 0.1611889  0.0310814   5.186 0.000000259 ***
## const         0.0020688  0.0006886   3.004     0.00273 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.02167 on 1015 degrees of freedom
## Multiple R-Squared: 0.04551, Adjusted R-squared: 0.04269
## F-statistic: 16.13 on 3 and 1015 DF,  p-value: 0.0000000002999
##
##
##
## Covariance matrix of residuals:
##              d_Spread     d_EURUSD   d_EURIBOR3
## d_Spread   15.2729351 -0.003318821  0.000627130
## d_EURUSD   -0.0033188  0.000027494 -0.000005283
## d_EURIBOR3  0.0006271 -0.000005283  0.000471401
##
## Correlation matrix of residuals:
##             d_Spread d_EURUSD d_EURIBOR3
## d_Spread    1.000000 -0.16196   0.007391
## d_EURUSD   -0.161960  1.00000  -0.046407
## d_EURIBOR3  0.007391 -0.04641   1.000000
```

```
var_ref_bic<- refVAR(var_bic_mts, thres = 1.64)
```

```
## Constant term:
## Estimates:  0 0 0.002068847
## Std.Error:  0 0 0.0006886136
## AR coefficient matrix
## AR( 1 )-matrix
```

```
##            [,1] [,2]     [,3]
## [1,]  0.0000000    0 -13.4490
## [2,] -0.0000827    0   0.0145
## [3,]  0.0000000    0   0.0632
## standard error
##          [,1] [,2]    [,3]
## [1,] 0.000000    0 5.53995
## [2,] 0.000042    0 0.00741
## [3,] 0.000000    0 0.03098
## AR( 2 )-matrix
##          [,1] [,2]  [,3]
## [1,] 0.000000    0 9.130
## [2,] 0.000078    0 0.000
## [3,] 0.000000    0 0.161
## standard error
##          [,1] [,2]   [,3]
## [1,] 0.000000    0 5.5556
## [2,] 0.000042    0 0.0000
## [3,] 0.000000    0 0.0311
##
## Residuals cov-mtx:
##               [,1]            [,2]            [,3]
## [1,] 15.1683620990 -0.003298282629  0.000622817453
## [2,] -0.0032982826  0.000027316156 -0.000005246794
## [3,]  0.0006228175 -0.000005246794  0.000468160029
##
## det(SSE) =  0.0000001884785
## AIC =  -15.47056
## BIC =  -15.43674
## HQ  =  -15.45772
```

```r
var_restricted_aic = vars::restrict(var_aic, method = "ser", thresh = 1.64)
summary(var_restricted_aic)
```

```
##
## VAR Estimation Results:
## =========================
## Endogenous variables: d_Spread, d_EURUSD, d_EURIBOR3
## Deterministic variables: const
## Sample size: 1017
## Log Likelihood: 3549.655
## Roots of the characteristic polynomial:
##   0.6 0.4504 0.4504 0.3941 0.3941 0.3941     0     0     0
## Call:
## vars::VAR(y = train_var[, -1], p = 3, type = "const")
##
##
## Estimation results for equation d_Spread:
## =========================================
## d_Spread = d_EURIBOR3.l1 + d_Spread.l3
##
##                Estimate Std. Error t value Pr(>|t|)
## d_EURIBOR3.l1 -12.94527    5.51943  -2.345   0.0192 *
## d_Spread.l3    -0.06122    0.03126  -1.958   0.0505 .
```

```
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 3.898 on 1015 degrees of freedom
## Multiple R-Squared: 0.008862,    Adjusted R-squared: 0.006909
## F-statistic: 4.537 on 2 and 1015 DF,  p-value: 0.01092
##
##
## Estimation results for equation d_EURUSD:
## =========================================
## d_EURUSD = d_Spread.l1 + d_EURIBOR3.l1 + d_Spread.l2
##
##                  Estimate  Std. Error t value Pr(>|t|)
## d_Spread.l1   -0.00008275  0.00004200  -1.970   0.0491 *
## d_EURIBOR3.l1  0.01450877  0.00741184   1.958   0.0506 .
## d_Spread.l2    0.00007805  0.00004200   1.859   0.0634 .
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.005237 on 1014 degrees of freedom
## Multiple R-Squared: 0.01064, Adjusted R-squared: 0.007714
## F-statistic: 3.635 on 3 and 1014 DF,  p-value: 0.01255
##
##
## Estimation results for equation d_EURIBOR3:
## ===========================================
## d_EURIBOR3 = d_EURIBOR3.l2 + d_EURIBOR3.l3 + const
##
##                Estimate Std. Error t value    Pr(>|t|)
## d_EURIBOR3.l2 0.1571228  0.0309174   5.082 0.000000444 ***
## d_EURIBOR3.l3 0.1217444  0.0309454   3.934 0.000089171 ***
## const         0.0019180  0.0006858   2.797     0.00526 **
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
##
## Residual standard error: 0.02156 on 1014 degrees of freedom
## Multiple R-Squared: 0.05604, Adjusted R-squared: 0.05325
## F-statistic: 20.07 on 3 and 1014 DF,  p-value: 0.000000000001222
##
##
##
## Covariance matrix of residuals:
##              d_Spread     d_EURUSD   d_EURIBOR3
## d_Spread   15.3147976 -0.003348314  0.000069296
## d_EURUSD   -0.0033483  0.000027602 -0.000005632
## d_EURIBOR3  0.0000693 -0.000005632  0.000468048
##
## Correlation matrix of residuals:
##              d_Spread d_EURUSD d_EURIBOR3
## d_Spread    1.0000000 -0.16285  0.0008185
## d_EURUSD   -0.1628534  1.00000 -0.0495511
```

```
## d_EURIBOR3  0.0008185 -0.04955  1.0000000
```

```r
var_ref_aic <- refVAR(var_aic_mts, thres = 1.64)
```

```
## Constant term:
## Estimates:  0 0 0.001918001
## Std.Error:  0 0 0.0006858248
## AR coefficient matrix
## AR( 1 )-matrix
##            [,1] [,2]     [,3]
## [1,]  0.0000000    0 -12.9453
## [2,] -0.0000828    0   0.0145
## [3,]  0.0000000    0   0.0000
## standard error
##          [,1] [,2]    [,3]
## [1,] 0.000000    0 5.51943
## [2,] 0.000042    0 0.00741
## [3,] 0.000000    0 0.00000
## AR( 2 )-matrix
##           [,1] [,2]  [,3]
## [1,] 0.0000000    0 0.000
## [2,] 0.0000781    0 0.000
## [3,] 0.0000000    0 0.157
## standard error
##          [,1] [,2]   [,3]
## [1,] 0.000000    0 0.0000
## [2,] 0.000042    0 0.0000
## [3,] 0.000000    0 0.0309
## AR( 3 )-matrix
##         [,1] [,2]  [,3]
## [1,] -0.0612    0 0.000
## [2,]  0.0000    0 0.000
## [3,]  0.0000    0 0.122
## standard error
##        [,1] [,2]   [,3]
## [1,] 0.0313    0 0.0000
## [2,] 0.0000    0 0.0000
## [3,] 0.0000    0 0.0309
##
## Residuals cov-mtx:
##                [,1]            [,2]            [,3]
## [1,] 15.16625103402 -0.003320242681  0.000068614339
## [2,] -0.00332024268  0.000027342570 -0.000005576743
## [3,]  0.00006861434 -0.000005576743  0.000463445378
##
## det(SSE) =  0.0000001866052
## AIC =  -15.48055
## BIC =  -15.44673
## HQ  =  -15.4677
```

```r
count_coeff = function(model){
  names = names(model$varresult)
  n = c()
```

```r
  for (name in names){
    n = c(n, length(model$varresult[[name]]$coefficients))
  }
  sum(n)
}

cat("\nUnrestricted model bic coeff: ", count_coeff(var_bic),
    " Restricted model bic coeff: ", count_coeff(var_restricted_bic), "\n")
```

```
##
## Unrestricted model bic coeff:  21  Restricted model bic coeff:  8
```

```r
cat("\nUr model SC: ", BIC(var_bic), "R model SC: ", BIC(var_bic))
```

```
##
## Ur model SC:  -6961.878 R model SC:  -6961.878
```

```r
cat("\nUr model AIC: ", AIC(var_bic), "R model AIC: ", AIC(var_bic), "\n")
```

```
##
## Ur model AIC:  -7065.315 R model AIC:  -7065.315
```

```r
cat("\nUnrestricted model aic coeff: ", count_coeff(var_aic),
    " Restricted model aic coeff: ", count_coeff(var_restricted_aic), "\n")
```

```
##
## Unrestricted model aic coeff:  30  Restricted model aic coeff:  8
```

```r
cat("\nUr model SC: ", BIC(var_aic), "R model SC: ", BIC(var_aic))
```

```
##
## Ur model SC:  -6912.366 R model SC:  -6912.366
```

```r
cat("\nUr model AIC: ", AIC(var_aic), "R model AIC: ", AIC(var_aic))
```

```
##
## Ur model AIC:  -7060.104 R model AIC:  -7060.104
```

```r
## VAR Diagnostics
```

```r
# ==========================================
# PHASE 4: VAR DIAGNOSTICS
# ==========================================

# 1. Test for leftover Serial Correlation (Portmanteau Test) # equivalente dell'MQ per pacchetto vars
# We test up to 10 lags to be safe. We want p-value > 0.05
serial_test <- serial.test(var_bic, lags.pt = 10, type = "PT.asymptotic")
cat("\n--- Serial Correlation Test ---\n")
```

```
##
## --- Serial Correlation Test ---
```

```r
print(serial_test)
```

```
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object var_bic
## Chi-squared = 110.45, df = 72, p-value = 0.002417
```

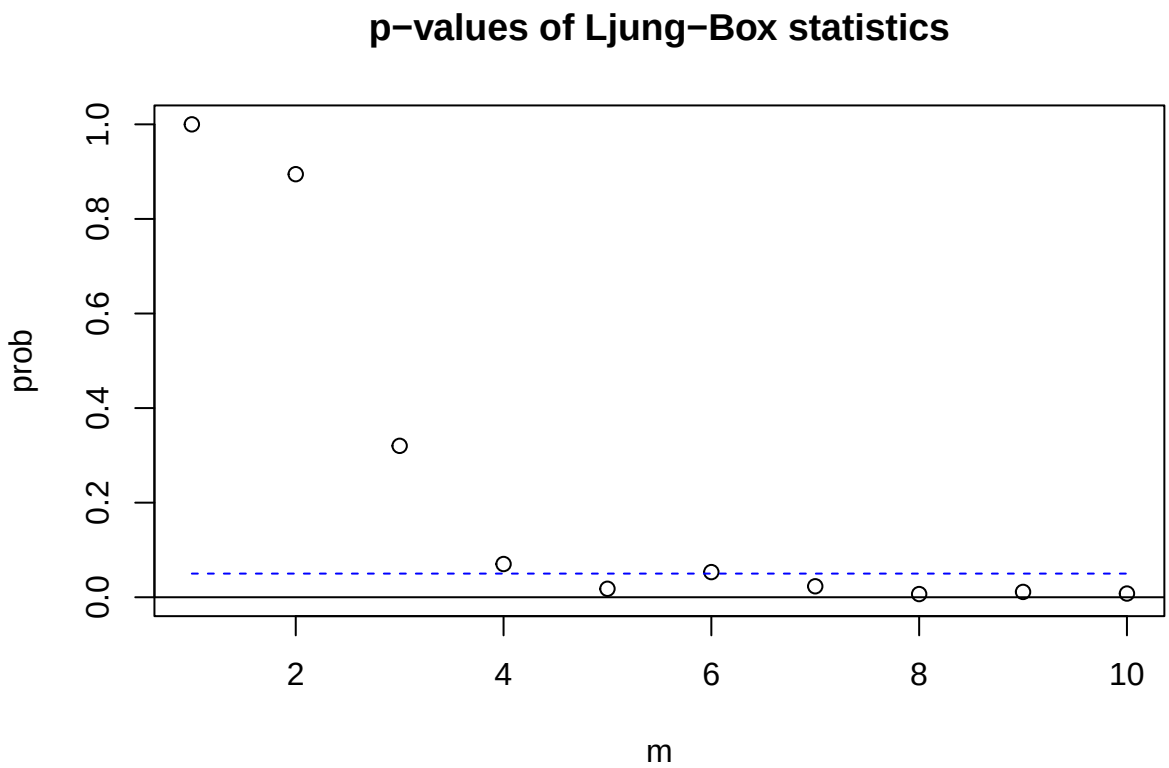
```
## $serial
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object var_bic
## Chi-squared = 110.45, df = 72, p-value = 0.002417
```

```r
cat("\n--- MQ test ---\n")
```

```
##
## --- MQ test ---
```

```r
# Calcoliamo i residui del modello raffinato
res_refined <- var_ref_bic$residuals
# Eseguiamo il test MQ fino al lag 10
mq_test <- mq(res_refined, lag = 10,adj=TRUE)
```

```
## Ljung-Box Statistics:
##          m       Q(m)     df    p-value
##  [1,]   1.00      2.72    8.00     1.00
##  [2,]   2.00     10.21   17.00     0.89
##  [3,]   3.00     28.80   26.00     0.32
##  [4,]   4.00     48.02   35.00     0.07
##  [5,]   5.00     65.89   44.00     0.02
##  [6,]   6.00     70.62   53.00     0.05
##  [7,]   7.00     86.10   62.00     0.02
##  [8,]   8.00    103.84   71.00     0.01
##  [9,]   9.00    111.61   80.00     0.01
## [10,]  10.00    124.46   89.00     0.01
```




```r
# 2. Test for ARCH Effects (Volatility Clustering)
# We want p-value < 0.05 to justify using DCC-GARCH next
arch_test <- arch.test(var_bic, lags.multi = 5, multivariate.only = TRUE)
cat("\n--- ARCH Effects Test ---\n")
```

```
##
## --- ARCH Effects Test ---
```

```r
print(arch_test)
```

```
##
##  ARCH (multivariate)
##
## data:  Residuals of VAR object var_bic
## Chi-squared = 818.04, df = 180, p-value < 0.00000000000000022

## $arch.mul
##
##  ARCH (multivariate)
##
## data:  Residuals of VAR object var_bic
## Chi-squared = 818.04, df = 180, p-value < 0.00000000000000022
```

```r
## We need the DCC-GARCH model, as there are arch effects (p-value is near 0)
```

Il modello con lag come suggerito dal bic non passa la diagnostica dì–> autocorrelazione residua

```r
# ==========================================
# PHASE 4: VAR DIAGNOSTICS
# ==========================================

# 1. Test for leftover Serial Correlation (Portmanteau Test) # equivalente dell'MQ per pacchetto vars
# We test up to 10 lags to be safe. We want p-value > 0.05
serial_test <- serial.test(var_aic, lags.pt = 10, type = "PT.asymptotic")
cat("\n--- Serial Correlation Test ---\n")
```

```
##
## --- Serial Correlation Test ---
```

```r
print(serial_test)
```

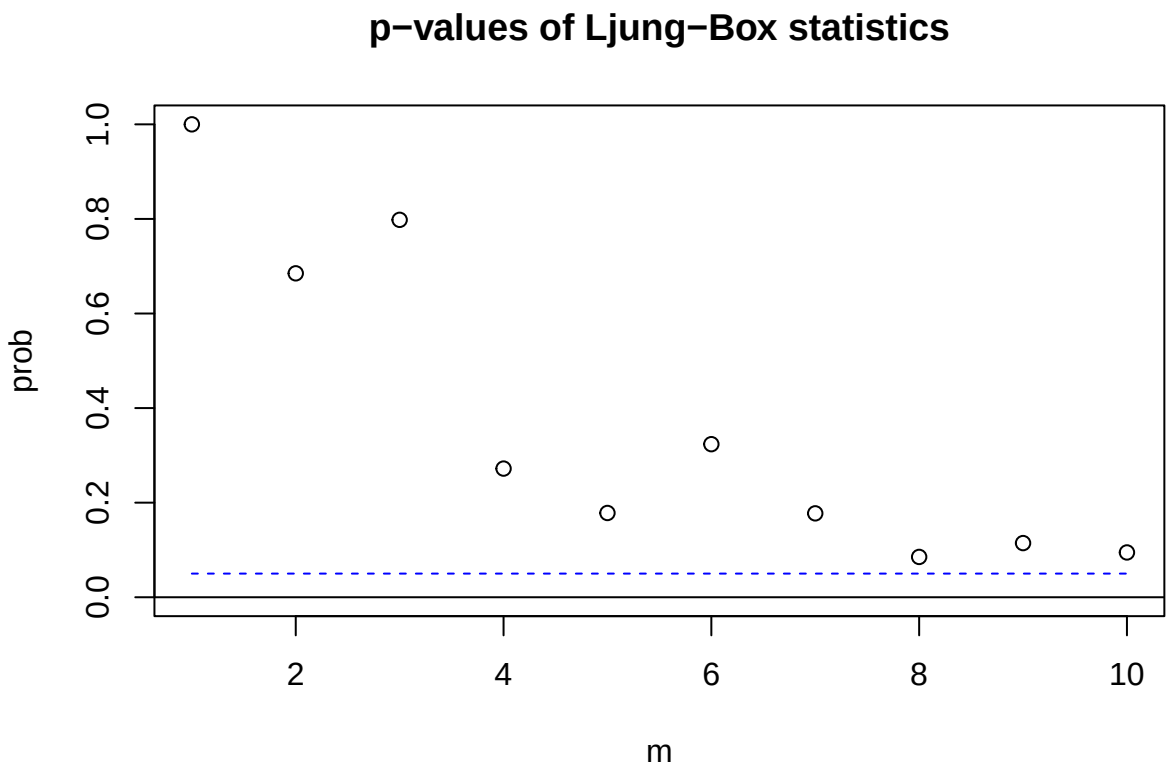
```
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object var_aic
## Chi-squared = 80.19, df = 63, p-value = 0.07092

## $serial
##
##  Portmanteau Test (asymptotic)
##
## data:  Residuals of VAR object var_aic
## Chi-squared = 80.19, df = 63, p-value = 0.07092
```

```r
cat("\n--- MQ test ---\n")
```

```
##
## --- MQ test ---
```

```r
# Calcoliamo i residui del modello raffinato
res_refined <- var_ref_aic$residuals
# Eseguiamo il test MQ fino al lag 10
mq_test <- mq(res_refined, lag = 10,adj=TRUE)
```

```
## Ljung-Box Statistics:
##          m       Q(m)     df    p-value
##  [1,]   1.0       4.7     8.0     1.00
##  [2,]   2.0      13.8    17.0     0.68
##  [3,]   3.0      19.9    26.0     0.80
##  [4,]   4.0      39.6    35.0     0.27
##  [5,]   5.0      52.5    44.0     0.18
##  [6,]   6.0      57.1    53.0     0.32
##  [7,]   7.0      72.2    62.0     0.18
##  [8,]   8.0      87.8    71.0     0.09
##  [9,]   9.0      95.5    80.0     0.11
## [10,]  10.0     106.9    89.0     0.09
```




```r
# 2. Test for ARCH Effects (Volatility Clustering)
# We want p-value < 0.05 to justify using DCC-GARCH next
arch_test <- arch.test(var_aic, lags.multi = 5, multivariate.only = TRUE)
cat("\n--- ARCH Effects Test ---\n")
```

```
##
## --- ARCH Effects Test ---
```

```r
print(arch_test)
```

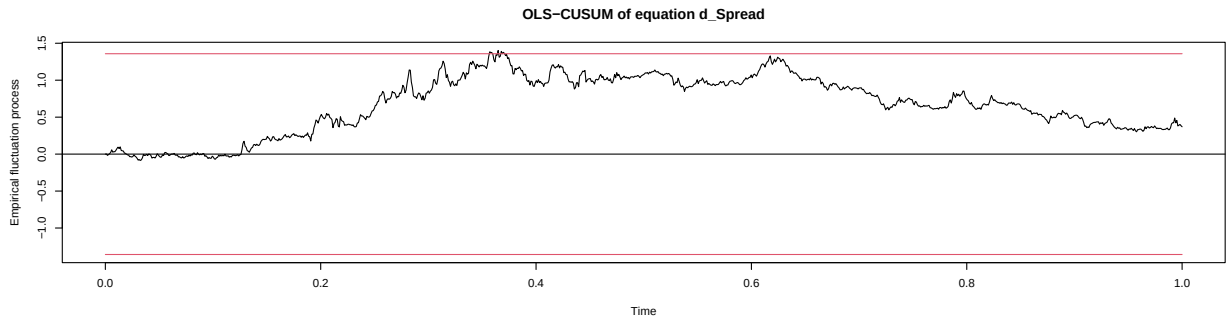
```
##
##  ARCH (multivariate)
##
## data:  Residuals of VAR object var_aic
## Chi-squared = 815.27, df = 180, p-value < 0.00000000000000022

## $arch.mul
##
##  ARCH (multivariate)
##
## data:  Residuals of VAR object var_aic
## Chi-squared = 815.27, df = 180, p-value < 0.00000000000000022
```

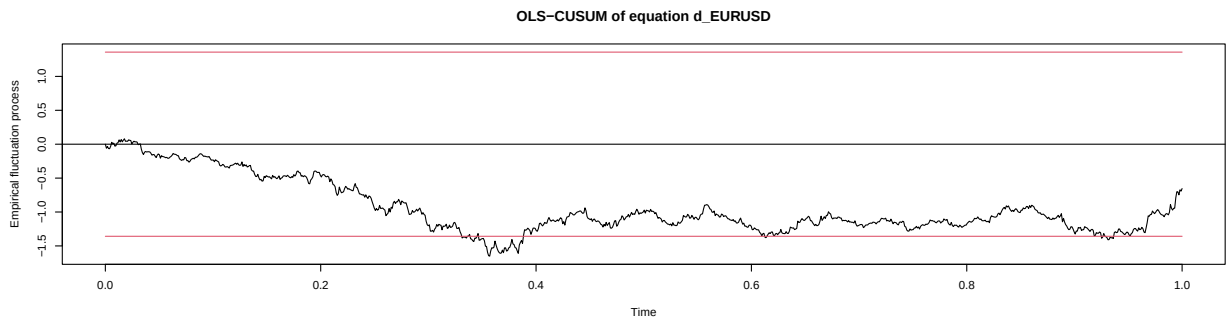
```r
## We need the DCC-GARCH model, as there are arch effects (p-value is near 0)
```

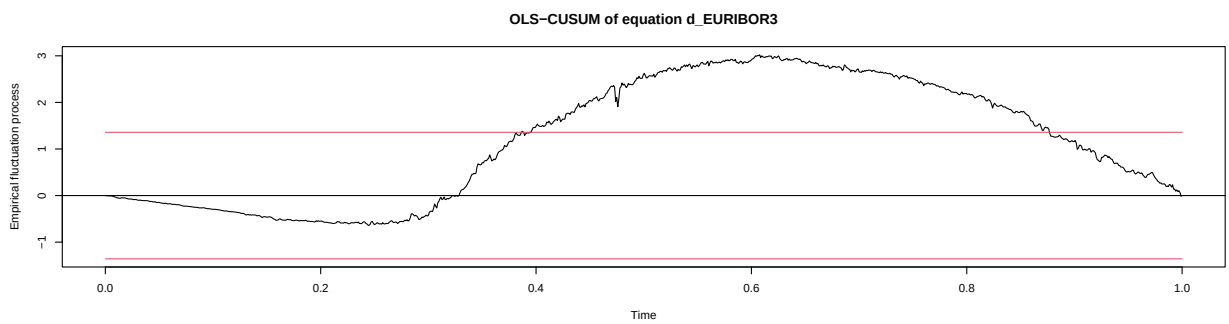
Stability test

We run CUSUM test to check for possible structural breaks

```
## $d_Spread
##
## Empirical Fluctuation Process: OLS-based CUSUM test
##
## Call: efp(formula = formula, data = data, type = type, h = h, dynamic = dynamic,
##     rescale = rescale)
##
##
## $d_EURUSD
##
## Empirical Fluctuation Process: OLS-based CUSUM test
##
## Call: efp(formula = formula, data = data, type = type, h = h, dynamic = dynamic,
##     rescale = rescale)
##
##
## $d_EURIBOR3
##
## Empirical Fluctuation Process: OLS-based CUSUM test
##
## Call: efp(formula = formula, data = data, type = type, h = h, dynamic = dynamic,
##     rescale = rescale)
```





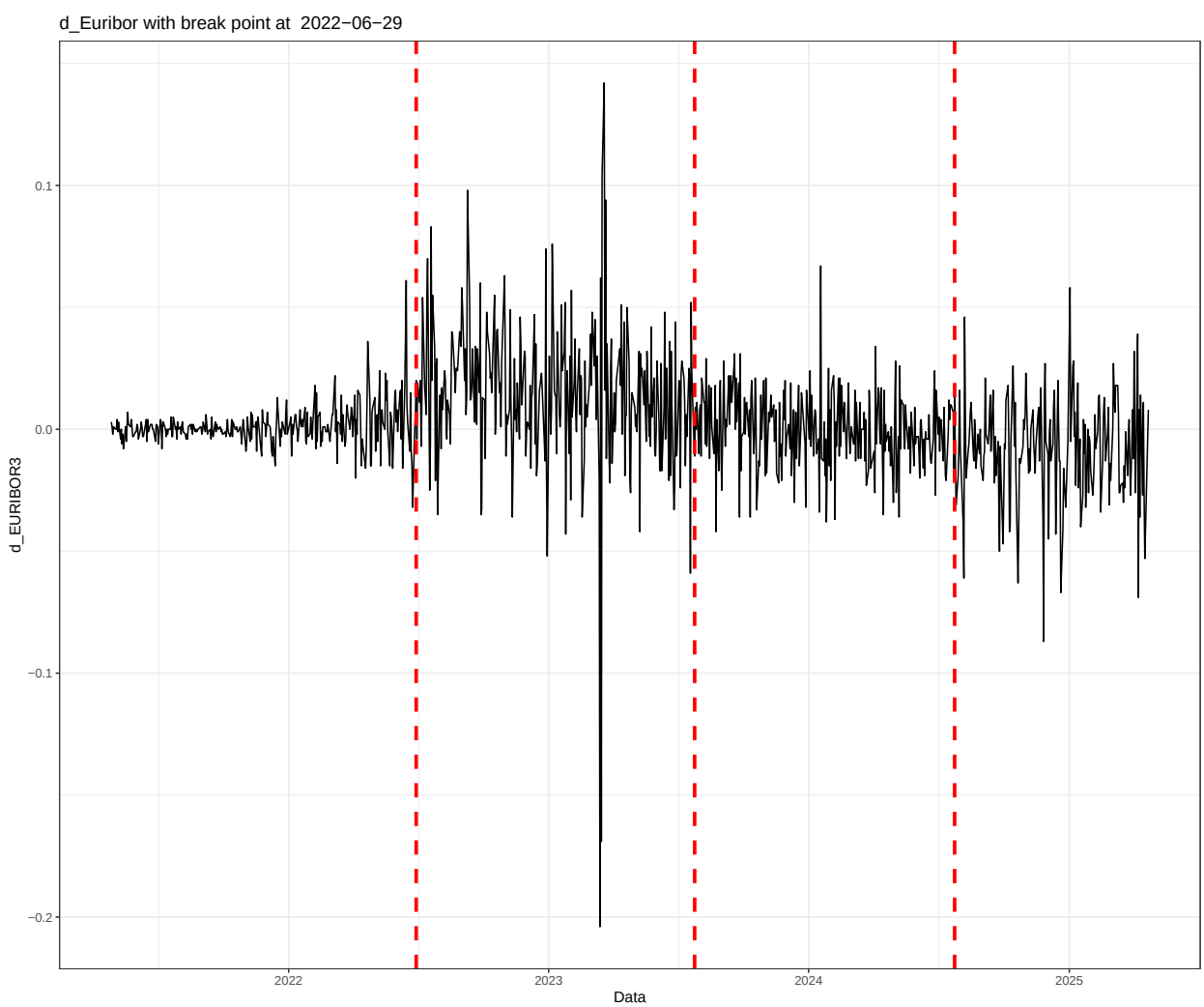


The `euribor` series residuals drifts outside the confidence intervals, sign that there's a structural break. We could try to solve it using a time dummy, but first we need to understand where the break occurs

```
bp_cds = breakpoints(residuals(var_restricted_aic)[,"d_EURIBOR3"] ~ 1)
summary(bp_cds)
```

```
##
##   Optimal (m+1)-segment partition:
##
## Call:
## breakpoints.formula(formula = residuals(var_restricted_aic)[,
##     "d_EURIBOR3"] ~ 1)
##
## Breakpoints at observation number:
##
## m = 1                636
## m = 2        297     618
## m = 3        297 509     757
## m = 4        297 478 636 827
## m = 5    153 305 478 636 827
```

```
##
## Corresponding to breakdates:
##
## m = 1                                                    0.625368731563422
## m = 2                    0.292035398230088               0.607669616519174
## m = 3                    0.292035398230088 0.500491642084562
## m = 4                    0.292035398230088 0.470009832841691 0.625368731563422
## m = 5   0.150442477876106 0.299901671583088 0.470009832841691 0.625368731563422
##
## m = 1
## m = 2
## m = 3   0.744346116027532
## m = 4   0.813176007866273
## m = 5   0.813176007866273
##
## Fit:
##
## m   0          1          2          3          4          5
## RSS     0.4713     0.4535     0.4417     0.4370     0.4350     0.4350
## BIC -4907.3578 -4932.7059 -4945.6494 -4942.6822 -4933.4725 -4919.6845
```



The RSS/BIC table tells that the model is at his best with 3 breakpoints. From the plot we can also see thhem Non so come si risolve questa cosa per cui rimandiamo

Normality Check if residuals are normal distributed, if not try to fit them to a t-distribution (more likely).

Use a KS test to test wheter they follow a t-distribution H_0 : serie follows supposed distribution

```r
# p_value < 0.05 -> non-normal distributed
normality.test(var_restricted_aic, multivariate.only = FALSE)
```

```
## $d_Spread
##
##  JB-Test (univariate)
##
## data:  Residual of d_Spread equation
## Chi-squared = 432.01, df = 2, p-value < 0.00000000000000022
##
##
## $d_EURUSD
##
##  JB-Test (univariate)
##
## data:  Residual of d_EURUSD equation
## Chi-squared = 112.85, df = 2, p-value < 0.00000000000000022
##
##
## $d_EURIBOR3
##
##  JB-Test (univariate)
##
## data:  Residual of d_EURIBOR3 equation
## Chi-squared = 9474.8, df = 2, p-value < 0.00000000000000022
##
##
## $JB
##
##  JB-Test (multivariate)
##
## data:  Residuals of VAR object var_restricted_aic
## Chi-squared = 10240, df = 6, p-value < 0.00000000000000022
##
##
## $Skewness
##
##  Skewness only (multivariate)
##
## data:  Residuals of VAR object var_restricted_aic
## Chi-squared = 50.381, df = 3, p-value = 0.00000000006629
##
##
## $Kurtosis
##
##  Kurtosis only (multivariate)
```

```
##
## data:  Residuals of VAR object var_restricted_aic
## Chi-squared = 10190, df = 3, p-value < 0.00000000000000022
```

```r
check_dist <- function(residual) {
  t = MASS::fitdistr(residuals, densfun = "t")
  n = MASS::fitdistr(residuals, densfun = "normal")
  aic = c(AIC(t), AIC(n))
  bic = c(BIC(t), BIC(n))
  df = data.frame(aic, bic, row.names = c("t", "n"))
  df
}
apply(residuals(var_restricted_aic), 2, check_dist)
```

```
## $d_Spread
##        aic      bic
## t 5544.752 5559.535
## n 5678.313 5688.168
##
## $d_EURUSD
##        aic      bic
## t 5544.752 5559.535
## n 5678.313 5688.168
##
## $d_EURIBOR3
##        aic      bic
## t 5544.752 5559.535
## n 5678.313 5688.168
```

We reject the null of the JB test for all the univariate series and the joint multivariate, thus the residuals do not follow a normal distribution.

The subsequent test, consisting in estimating a normal a t distribution on the residuals and then compute their IC, showed that a t-distribution is a better fit than the normal.

```r
vars::causality(var_restricted_aic, cause = c("d_EURUSD", "d_EURIBOR3"))
```

Causality

```
## $Granger
##
##  Granger causality H0: d_EURUSD d_EURIBOR3 do not Granger-cause d_Spread
##
## data:  VAR object var_restricted_aic
## F-Test = 1.1374, df1 = 6, df2 = 3021, p-value = 0.3377
##
##
## $Instant
##
##  H0: No instantaneous causality between: d_EURUSD d_EURIBOR3 and
```

```
##  d_Spread
##
## data:  VAR object var_restricted_aic
## Chi-squared = 26.387, df = 2, p-value = 0.000001863
```

```r
vars::causality(var_restricted_aic, cause = c("d_Spread", "d_EURIBOR3"))
```

```
## $Granger
##
##  Granger causality H0: d_Spread d_EURIBOR3 do not Granger-cause d_EURUSD
##
## data:  VAR object var_restricted_aic
## F-Test = 1.2412, df1 = 6, df2 = 3021, p-value = 0.2818
##
##
## $Instant
##
##  H0: No instantaneous causality between: d_Spread d_EURIBOR3 and
##  d_EURUSD
##
## data:  VAR object var_restricted_aic
## Chi-squared = 28.686, df = 2, p-value = 0.0000005901
```

```r
vars::causality(var_restricted_aic, cause = c("d_Spread", "d_EURUSD"))
```

```
## $Granger
##
##  Granger causality H0: d_Spread d_EURUSD do not Granger-cause d_EURIBOR3
##
## data:  VAR object var_restricted_aic
## F-Test = 0.93801, df1 = 6, df2 = 3021, p-value = 0.4663
##
##
## $Instant
##
##  H0: No instantaneous causality between: d_Spread d_EURUSD and
##  d_EURIBOR3
##
## data:  VAR object var_restricted_aic
## Chi-squared = 2.5447, df = 2, p-value = 0.2802
```

DCC-GARCH Model

```r
# ==========================================
# PHASE 5: THE DCC-GARCH MODEL
# ==========================================
library(rugarch)
library(rmgarch)
```

```
##
## Attaching package: 'rmgarch'
```

```
## The following objects are masked from 'package:dplyr':
##
##     first, last

## The following objects are masked from 'package:xts':
##
##     first, last
```

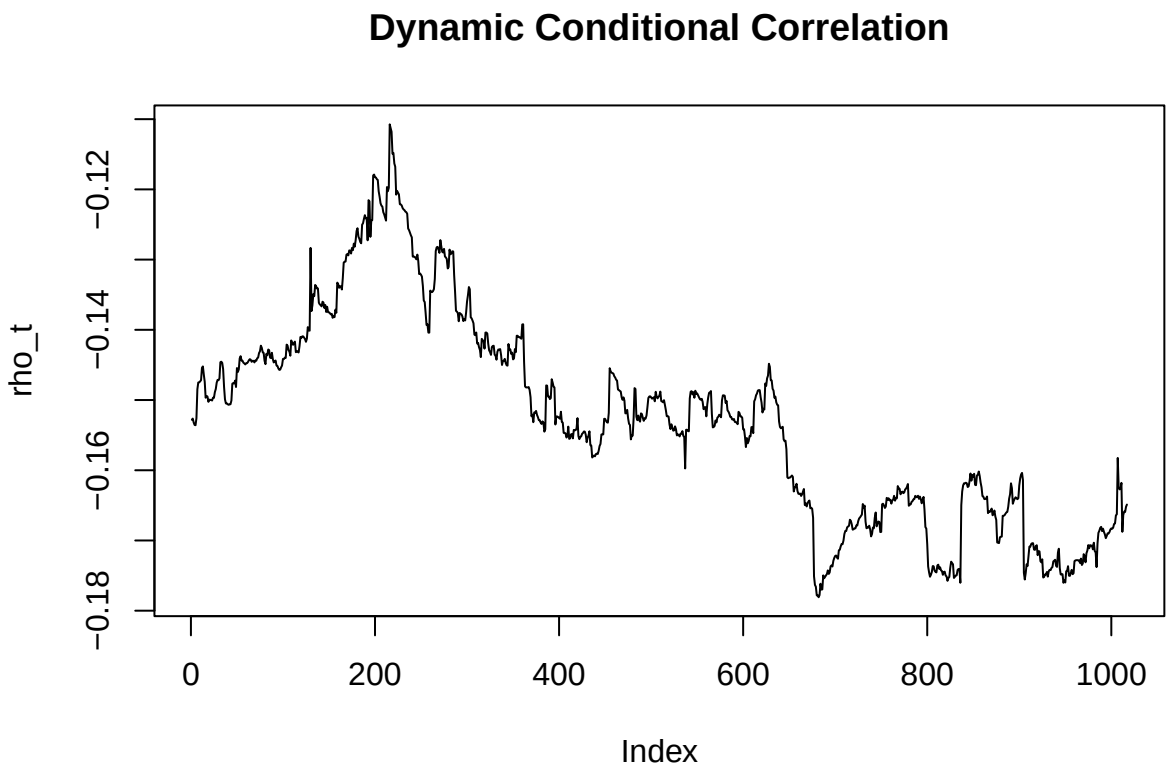
```r
# 1. Extract the residuals from your fitted VAR model
# The scale (* 100) is crucial! GARCH optimizers fail if the numbers are too tiny (e.g. 0.0001)
res_var <- resid(var_restricted_aic)*100

# Convert the residuals back into a Time Series (xts) object for plotting later
# We use the dates from diff_data, but remember the VAR loses the first 'p' rows
# So we align the dates by tailing the diff_data
# Assicurati che la colonna date sia nel formato corretto
dates_vector <- as.Date(train_var$Data)

# Crea l'oggetto xts usando le ultime 'n' date corrispondenti ai residui
var_residuals_xts <- xts(res_var,
                         order.by = tail(dates_vector, nrow(res_var)))
colnames(var_residuals_xts) <- c("Spread_Shock", "EURUSD_Shock","EURIBOR3_Shock")

# 2. Specify the Univariate GARCH Models
# Because the VAR already handled the mean, we set the ARMA order to (0,0) with NO mean!
uspec <- ugarchspec(
  mean.model = list(armaOrder = c(0, 0), include.mean = FALSE),
  variance.model = list(garchOrder = c(1, 1), model = "sGARCH"),
  distribution.model = "norm" # You can change to "std" (Student-t) if it fails to converge
)

# Replicate this specification for all of our variables
mspec <- multispec(replicate(3, uspec))

# 3. Specify the Multivariate DCC-GARCH model
# We set VAR = FALSE because we ALREADY did the VAR manually and are feeding it the residuals!
dcc_spec <- dccspec(
  uspec = mspec,
  VAR = FALSE,
  dccOrder = c(1, 1),
  distribution = "mvnorm"
)

# 4. Fit the DCC-GARCH model!
# We feed it our scaled VAR residuals
cat("\nFitting DCC-GARCH...\n")
```

```
##
## Fitting DCC-GARCH...
```

```r
fit_dcc <- dccfit(dcc_spec, data = var_residuals_xts)

# Print the final estimated model
show(fit_dcc)
```

```
##
## *---------------------------------*
## *          DCC GARCH Fit          *
## *---------------------------------*
##
## Distribution         :  mvnorm
## Model                :  DCC(1,1)
## No. Parameters       :  14
## [VAR GARCH DCC UncQ] : [0+9+2+3]
## No. Series           :  3
## No. Obs.             :  1017
## Log-Likelihood       :  -10063.03
## Av.Log-Likelihood    :  -9.89
##
## Optimal Parameters
## ------------------------------------
##                            Estimate  Std. Error   t value Pr(>|t|)
## [Spread_Shock].omega    4823.783126 2127.400834   2.26745 0.023363
## [Spread_Shock].alpha1      0.177619    0.049334   3.60035 0.000318
## [Spread_Shock].beta1       0.801576    0.051227  15.64747 0.000000
## [EURUSD_Shock].omega       0.002579    0.001514   1.70332 0.088508
## [EURUSD_Shock].alpha1      0.036030    0.006732   5.35184 0.000000
## [EURUSD_Shock].beta1       0.955859    0.005569 171.63640 0.000000
## [EURIBOR3_Shock].omega     0.007815    0.008254   0.94686 0.343710
## [EURIBOR3_Shock].alpha1    0.130689    0.027024   4.83605 0.000001
## [EURIBOR3_Shock].beta1     0.868311    0.037902  22.90916 0.000000
## [Joint]dcca1               0.001222    0.002498   0.48912 0.624758
## [Joint]dccb1               0.992054    0.004483 221.29416 0.000000
##
## Information Criteria
## ---------------------
##
## Akaike       19.817
## Bayes        19.885
## Shibata      19.817
## Hannan-Quinn 19.843
##
##
## Elapsed time : 0.8895259
```

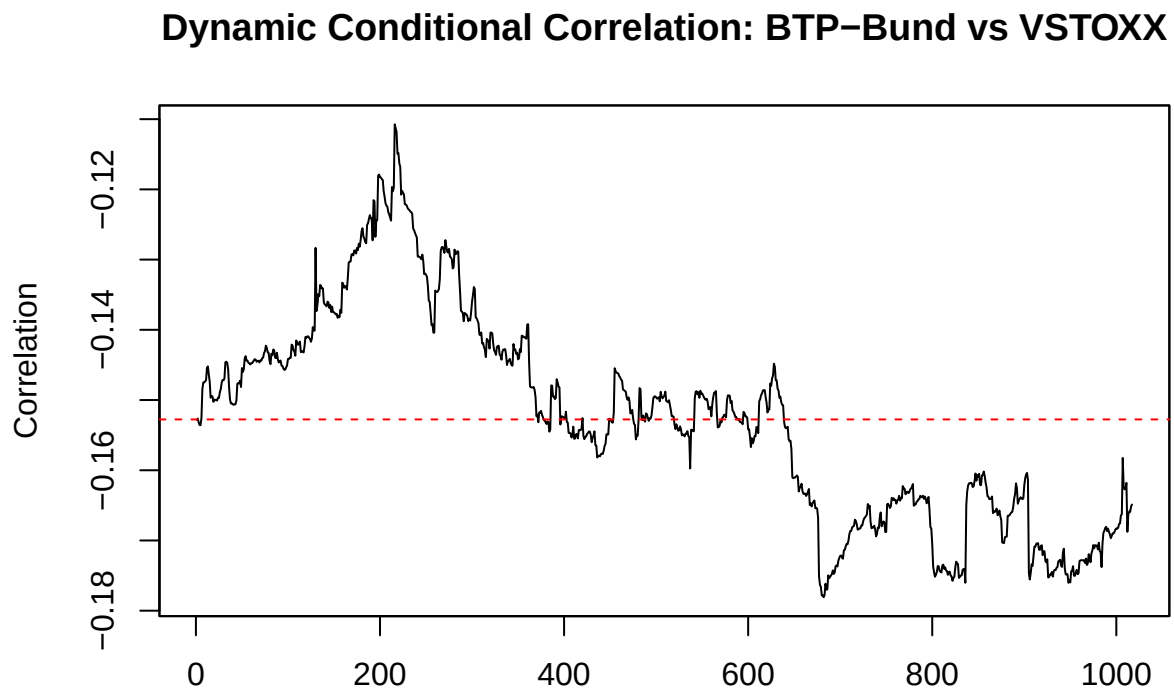
```r
# Dynamic conditional correlations over time
dcc_corr <- rcor(fit_dcc)        # array: 2x2xT
rho_t    <- dcc_corr[1,2,]       # time series of rho(BTP-Bund, VSTOXX)
plot(rho_t, type = "l", main = "Dynamic Conditional Correlation")
```




```r
# Conditional volatilities
vol <- sigma(fit_dcc)

# DCC parameters (alpha + beta, should sum < 1)
coef(fit_dcc)
```

```
##    [Spread_Shock].omega   [Spread_Shock].alpha1    [Spread_Shock].beta1
##          4823.783125850              0.177619192             0.801575783
##    [EURUSD_Shock].omega   [EURUSD_Shock].alpha1    [EURUSD_Shock].beta1
##             0.002579013              0.036029819             0.955859244
##  [EURIBOR3_Shock].omega [EURIBOR3_Shock].alpha1  [EURIBOR3_Shock].beta1
##             0.007815476              0.130688935             0.868311064
##            [Joint]dcca1             [Joint]dccb1
##             0.001221620              0.992053713
```

```r
library(xts)

rho_t2 <- rcor(fit_dcc)[1, 2, ]
resids <- residuals(var_restricted_aic)

plot(zoo(rho_t2, order.by = index(resids)), type = "l",
     main = "Dynamic Conditional Correlation: BTP-Bund vs VSTOXX",
     ylab = "Correlation", xlab = "")
abline(h = mean(rho_t2), col = "red", lty = 2)
```

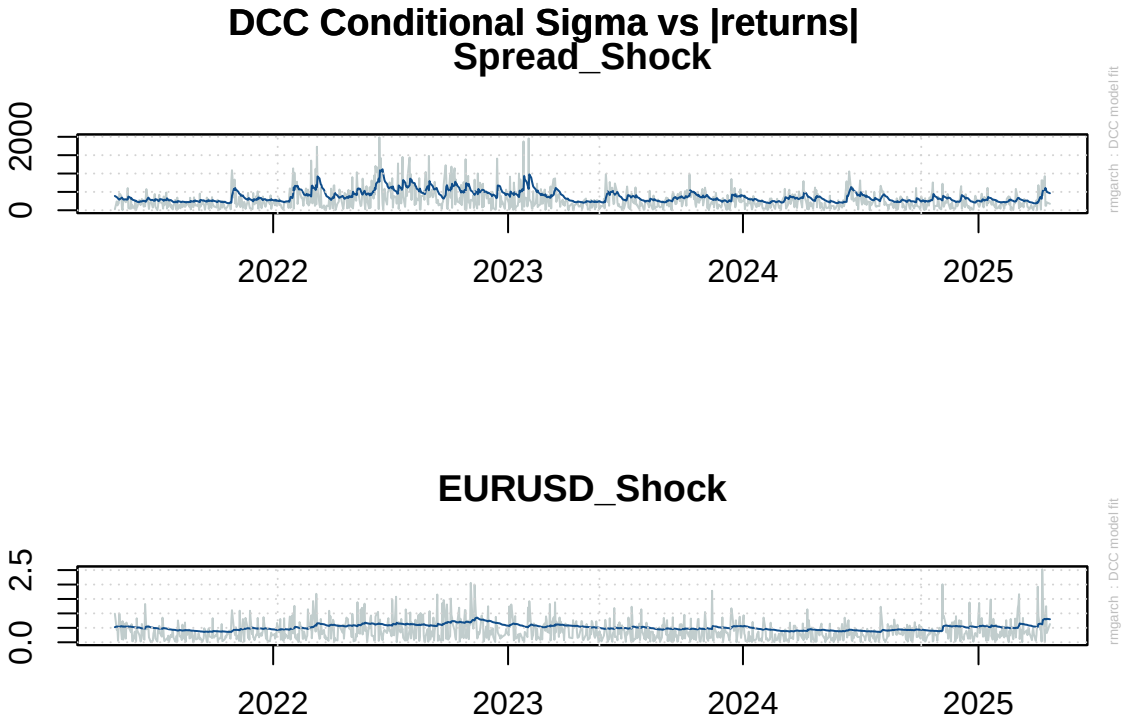


Model Diagnostics

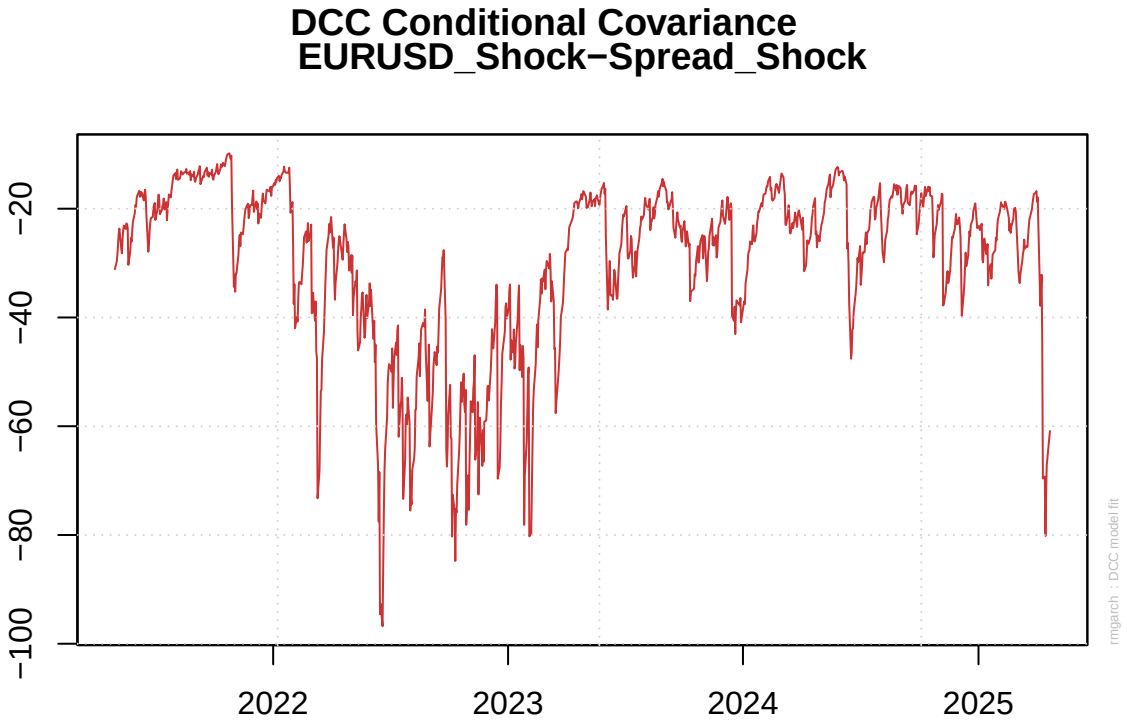
```
# 1. Model Diagnostics
# Standardized residuals should have no ARCH effects remaining
infocriteria(fit_dcc)   # AIC, BIC
```

```
##
## Akaike       19.81717
## Bayes        19.88496
## Shibata      19.81680
## Hannan-Quinn 19.84291
```

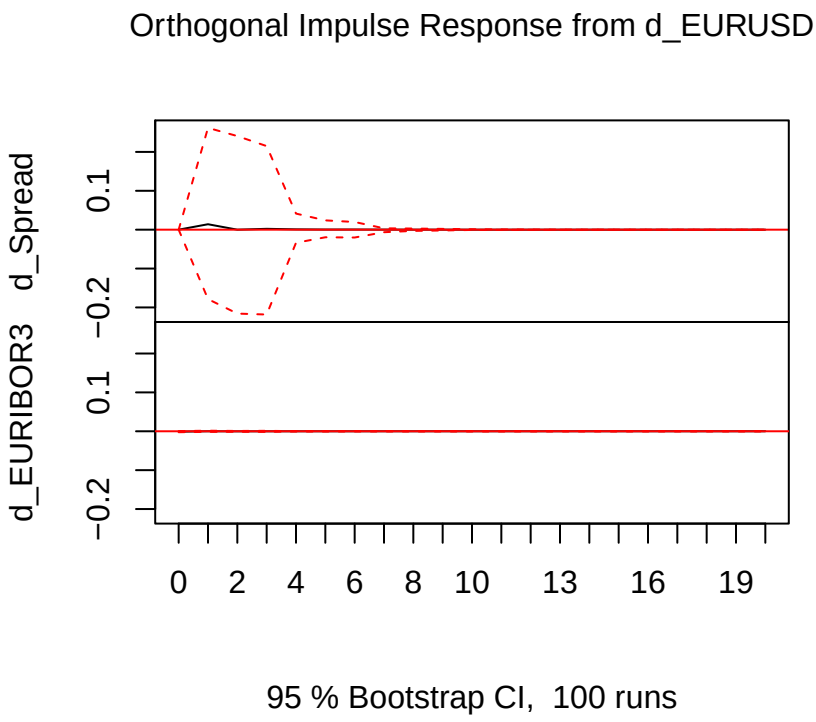
```
plot(fit_dcc, which=2)          # rmgarch gives several diagnostic plots
```



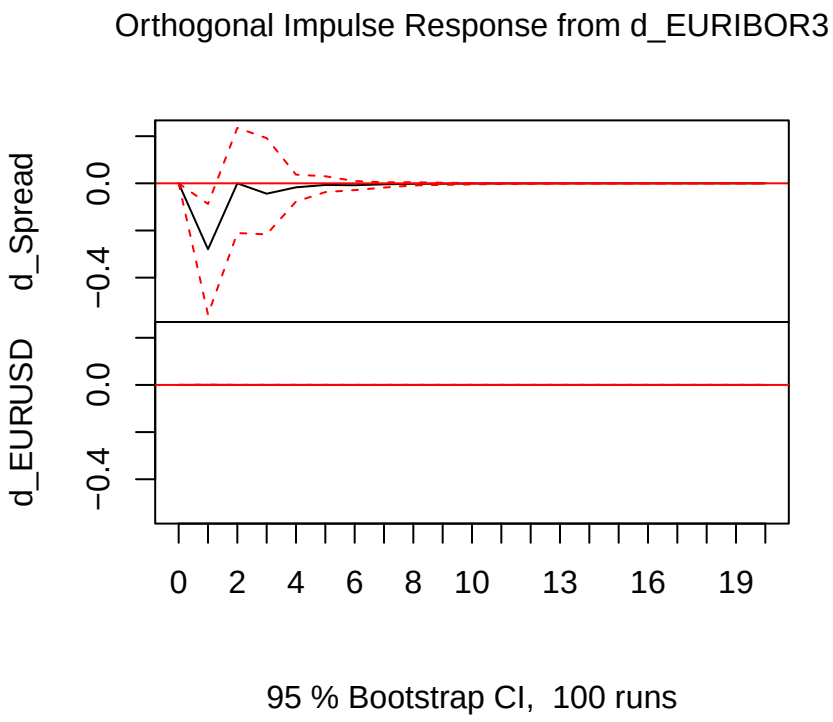

```
plot(fit_dcc, which=3)
```



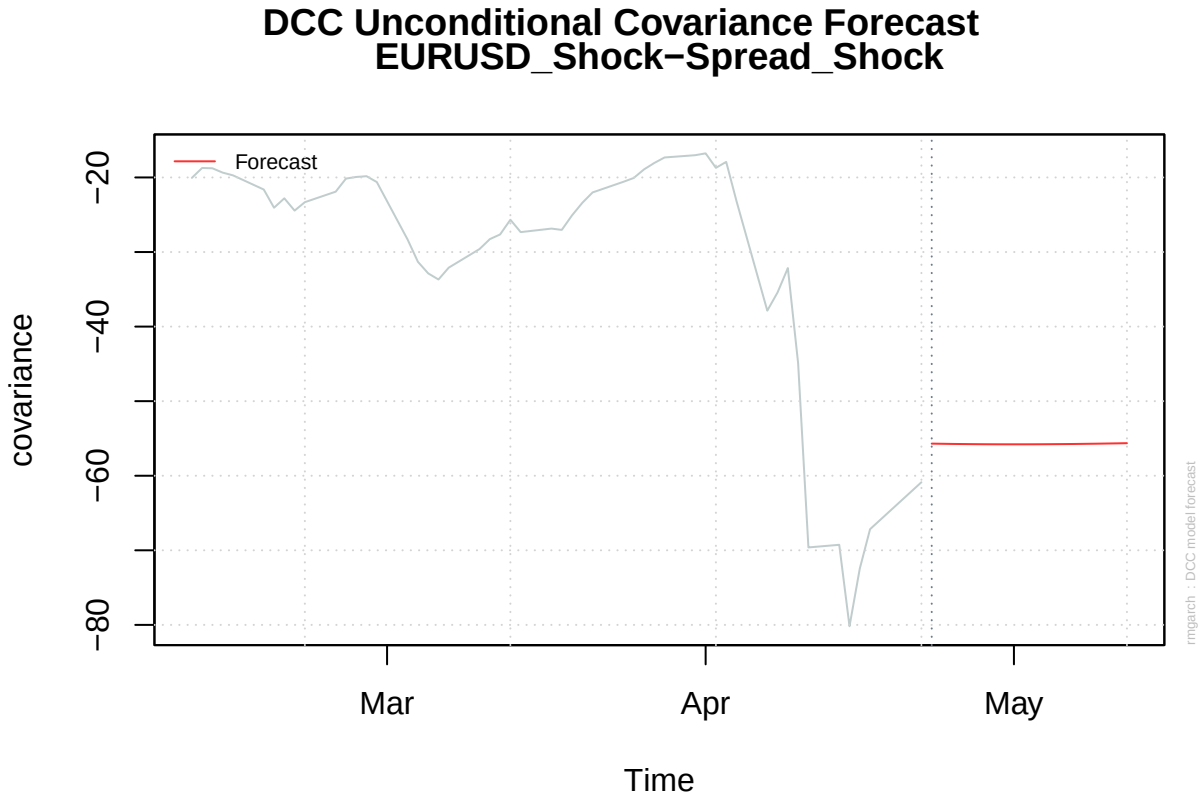

```
#colnames(var_fit$y)

# 2. VAR Impulse Response Functions (IRFs)
# This answers "how does the BTP-Bund spread respond to a VSTOXX shock?"
library(vars)
irf_result <- irf(var_restricted_aic, impulse = "d_EURUSD",
                  response = c("d_Spread", "d_EURIBOR3"),
                  n.ahead = 20, boot = TRUE)
plot(irf_result)
```




```
irf_result <- irf(var_restricted_aic, impulse = "d_EURIBOR3",
                  response = c("d_Spread", "d_EURUSD"),
                  n.ahead = 20, boot = TRUE)
plot(irf_result)
```




```r
# 3. DCC-GARCH Volatility Forecast
# A short-horizon forecast of conditional variances and correlations
dcc_forecast <- dccforecast(fit_dcc, n.ahead = 20)
plot(dcc_forecast, which = 3)
```




```r
# Extract forecasted correlation
rcor(dcc_forecast)
```

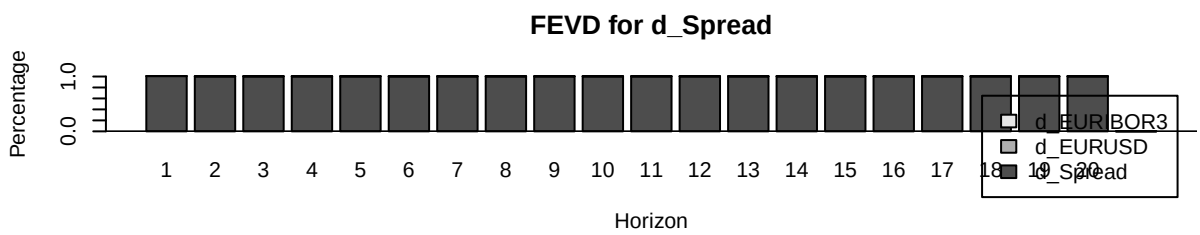
```
## $`2025-04-22`
## , , T+1
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16510079    -0.05087968
## EURUSD_Shock    -0.16510079   1.00000000    -0.03194427
## EURIBOR3_Shock  -0.05087968  -0.03194427     1.00000000
##
## , , T+2
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16502764    -0.05088257
## EURUSD_Shock    -0.16502764   1.00000000    -0.03195622
## EURIBOR3_Shock  -0.05088257  -0.03195622     1.00000000
##
## , , T+3
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16495498    -0.05088544
## EURUSD_Shock    -0.16495498   1.00000000    -0.03196809
## EURIBOR3_Shock  -0.05088544  -0.03196809     1.00000000
##
## , , T+4
```

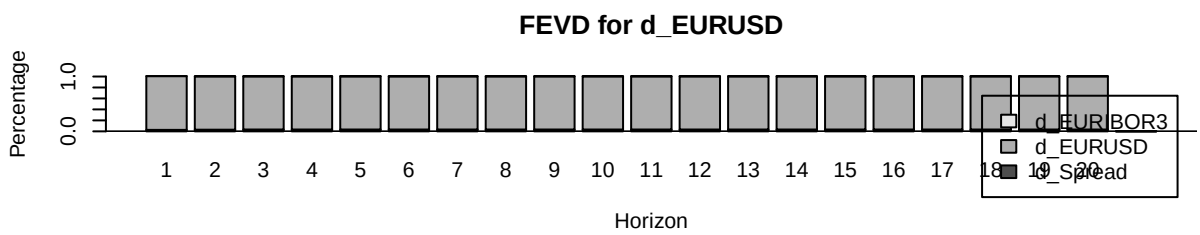
```
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16488281    -0.05088829
## EURUSD_Shock    -0.16488281   1.00000000    -0.03197989
## EURIBOR3_Shock  -0.05088829  -0.03197989     1.00000000
##
## , , T+5
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000   -0.1648111    -0.05089113
## EURUSD_Shock    -0.16481112    1.0000000    -0.03199160
## EURIBOR3_Shock  -0.05089113   -0.0319916     1.00000000
##
## , , T+6
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16473991    -0.05089394
## EURUSD_Shock    -0.16473991   1.00000000    -0.03200324
## EURIBOR3_Shock  -0.05089394  -0.03200324     1.00000000
##
## , , T+7
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000   -0.1646692    -0.05089673
## EURUSD_Shock    -0.16466919    1.0000000    -0.03201480
## EURIBOR3_Shock  -0.05089673   -0.0320148     1.00000000
##
## , , T+8
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16459893    -0.05089951
## EURUSD_Shock    -0.16459893   1.00000000    -0.03202628
## EURIBOR3_Shock  -0.05089951  -0.03202628     1.00000000
##
## , , T+9
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16452916    -0.05090227
## EURUSD_Shock    -0.16452916   1.00000000    -0.03203768
## EURIBOR3_Shock  -0.05090227  -0.03203768     1.00000000
##
## , , T+10
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16445985    -0.05090501
## EURUSD_Shock    -0.16445985   1.00000000    -0.03204901
## EURIBOR3_Shock  -0.05090501  -0.03204901     1.00000000
##
## , , T+11
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16439100    -0.05090773
## EURUSD_Shock    -0.16439100   1.00000000    -0.03206026
## EURIBOR3_Shock  -0.05090773  -0.03206026     1.00000000
```

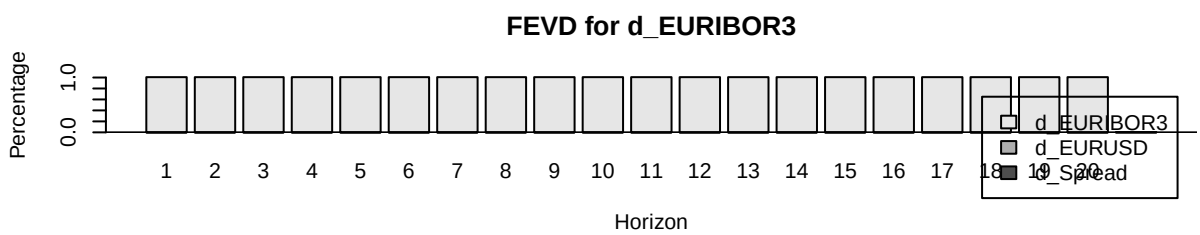
```
##
## , , T+12
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16432262    -0.05091043
## EURUSD_Shock    -0.16432262   1.00000000    -0.03207143
## EURIBOR3_Shock  -0.05091043  -0.03207143     1.00000000
##
## , , T+13
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16425470    -0.05091311
## EURUSD_Shock    -0.16425470   1.00000000    -0.03208253
## EURIBOR3_Shock  -0.05091311  -0.03208253     1.00000000
##
## , , T+14
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16418724    -0.05091578
## EURUSD_Shock    -0.16418724   1.00000000    -0.03209355
## EURIBOR3_Shock  -0.05091578  -0.03209355     1.00000000
##
## , , T+15
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16412023    -0.05091843
## EURUSD_Shock    -0.16412023   1.00000000    -0.03210451
## EURIBOR3_Shock  -0.05091843  -0.03210451     1.00000000
##
## , , T+16
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16405367    -0.05092106
## EURUSD_Shock    -0.16405367   1.00000000    -0.03211538
## EURIBOR3_Shock  -0.05092106  -0.03211538     1.00000000
##
## , , T+17
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16398756    -0.05092367
## EURUSD_Shock    -0.16398756   1.00000000    -0.03212619
## EURIBOR3_Shock  -0.05092367  -0.03212619     1.00000000
##
## , , T+18
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16392189    -0.05092627
## EURUSD_Shock    -0.16392189   1.00000000    -0.03213692
## EURIBOR3_Shock  -0.05092627  -0.03213692     1.00000000
##
## , , T+19
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock     1.00000000  -0.16385666    -0.05092884
```

```
## EURUSD_Shock    -0.16385666   1.00000000    -0.03214758
## EURIBOR3_Shock  -0.05092884  -0.03214758     1.00000000
##
## , , T+20
##
##                Spread_Shock EURUSD_Shock EURIBOR3_Shock
## Spread_Shock      1.0000000  -0.16379188    -0.05093140
## EURUSD_Shock     -0.1637919   1.00000000    -0.03215816
## EURIBOR3_Shock   -0.0509314  -0.03215816     1.00000000
```

```r
# 4. Variance Decomposition (FEVD)
# Tells you how much of the BTP-Bund spread variance is explained by VSTOXX shocks

fevd_result <- fevd(var_restricted_aic, n.ahead = 20)
plot(fevd_result)
```







Forecast

```r
# Re-fit with out.sample to enable rolling forecast
dcc_fit2 <- dccfit(dcc_spec, data = resids,
                   out.sample = 100)   # hold out last 100 obs

# 1. Unconditional n-ahead forecast
```

```r
dcc_fore <- dccforecast(dcc_fit2, n.ahead = 20)

# Forecasted correlation
rcor(dcc_fore)   # 2x2xT array
```

```
## $`1972-07-06`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17060913 -0.06756304
## d_EURUSD   -0.17060913  1.00000000 -0.03277547
## d_EURIBOR3 -0.06756304 -0.03277547  1.00000000
##
## , , T+2
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17046527 -0.06749227
## d_EURUSD   -0.17046527  1.00000000 -0.03278411
## d_EURIBOR3 -0.06749227 -0.03278411  1.00000000
##
## , , T+3
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17032229 -0.06742193
## d_EURUSD   -0.17032229  1.00000000 -0.03279271
## d_EURIBOR3 -0.06742193 -0.03279271  1.00000000
##
## , , T+4
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17018018 -0.06735203
## d_EURUSD   -0.17018018  1.00000000 -0.03280125
## d_EURIBOR3 -0.06735203 -0.03280125  1.00000000
##
## , , T+5
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17003896 -0.06728255
## d_EURUSD   -0.17003896  1.00000000 -0.03280975
## d_EURIBOR3 -0.06728255 -0.03280975  1.00000000
##
## , , T+6
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.16989860 -0.06721350
## d_EURUSD   -0.1698986  1.00000000 -0.03281818
## d_EURIBOR3 -0.0672135 -0.03281818  1.00000000
##
## , , T+7
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16975910 -0.06714488
## d_EURUSD   -0.16975910  1.00000000 -0.03282657
```

```
## d_EURIBOR3 -0.06714488 -0.03282657  1.00000000
##
## , , T+8
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16962047 -0.06707668
## d_EURUSD   -0.16962047  1.00000000 -0.03283491
## d_EURIBOR3 -0.06707668 -0.03283491  1.00000000
##
## , , T+9
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.16948268 -0.06700890
## d_EURUSD   -0.1694827  1.00000000 -0.03284319
## d_EURIBOR3 -0.0670089 -0.03284319  1.00000000
##
## , , T+10
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16934574 -0.06694153
## d_EURUSD   -0.16934574  1.00000000 -0.03285142
## d_EURIBOR3 -0.06694153 -0.03285142  1.00000000
##
## , , T+11
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16920965 -0.06687458
## d_EURUSD   -0.16920965  1.00000000 -0.03285961
## d_EURIBOR3 -0.06687458 -0.03285961  1.00000000
##
## , , T+12
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16907439 -0.06680804
## d_EURUSD   -0.16907439  1.00000000 -0.03286774
## d_EURIBOR3 -0.06680804 -0.03286774  1.00000000
##
## , , T+13
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16893996 -0.06674191
## d_EURUSD   -0.16893996  1.00000000 -0.03287582
## d_EURIBOR3 -0.06674191 -0.03287582  1.00000000
##
## , , T+14
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16880636 -0.06667619
## d_EURUSD   -0.16880636  1.00000000 -0.03288385
## d_EURIBOR3 -0.06667619 -0.03288385  1.00000000
##
## , , T+15
##
##               d_Spread    d_EURUSD  d_EURIBOR3
```

```
## d_Spread    1.00000000 -0.16867358 -0.06661087
## d_EURUSD   -0.16867358  1.00000000 -0.03289183
## d_EURIBOR3 -0.06661087 -0.03289183  1.00000000
##
## , , T+16
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16854162 -0.06654595
## d_EURUSD   -0.16854162  1.00000000 -0.03289977
## d_EURIBOR3 -0.06654595 -0.03289977  1.00000000
##
## , , T+17
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16841047 -0.06648143
## d_EURUSD   -0.16841047  1.00000000 -0.03290765
## d_EURIBOR3 -0.06648143 -0.03290765  1.00000000
##
## , , T+18
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16828012 -0.06641731
## d_EURUSD   -0.16828012  1.00000000 -0.03291549
## d_EURIBOR3 -0.06641731 -0.03291549  1.00000000
##
## , , T+19
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16815058 -0.06635359
## d_EURUSD   -0.16815058  1.00000000 -0.03292328
## d_EURIBOR3 -0.06635359 -0.03292328  1.00000000
##
## , , T+20
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16802184 -0.06629025
## d_EURUSD   -0.16802184  1.00000000 -0.03293102
## d_EURIBOR3 -0.06629025 -0.03293102  1.00000000
```

```r
# Forecasted volatilities
sigma(dcc_fore)
```

```
## , , 1972-07-06
##
##      d_Spread    d_EURUSD d_EURIBOR3
## T+1  2.402125 0.005436685 0.01894221
## T+2  2.475447 0.005433612 0.01895324
## T+3  2.545139 0.005430565 0.01896425
## T+4  2.611524 0.005427546 0.01897524
## T+5  2.674879 0.005424553 0.01898622
## T+6  2.735443 0.005421587 0.01899717
## T+7  2.793428 0.005418647 0.01900811
## T+8  2.849016 0.005415733 0.01901904
## T+9  2.902371 0.005412845 0.01902994
```

```
## T+10 2.953640 0.005409983 0.01904083
## T+11 3.002953 0.005407146 0.01905170
## T+12 3.050427 0.005404334 0.01906256
## T+13 3.096170 0.005401547 0.01907340
## T+14 3.140279 0.005398785 0.01908422
## T+15 3.182843 0.005396048 0.01909502
## T+16 3.223941 0.005393335 0.01910581
## T+17 3.263650 0.005390646 0.01911658
## T+18 3.302038 0.005387981 0.01912733
## T+19 3.339168 0.005385340 0.01913807
## T+20 3.375100 0.005382723 0.01914879
```

```r
# 2. Rolling 1-step-ahead forecast (more meaningful for evaluation)
dcc_roll <- dccforecast(dcc_fit2, n.ahead = 1, n.roll = 99)
rcor(dcc_roll)
```

```
## $`1972-07-06`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17060913 -0.06756304
## d_EURUSD   -0.17060913  1.00000000 -0.03277547
## d_EURIBOR3 -0.06756304 -0.03277547  1.00000000
##
##
## $`1972-07-07`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17043797 -0.06480901
## d_EURUSD   -0.17043797  1.00000000 -0.03189438
## d_EURIBOR3 -0.06480901 -0.03189438  1.00000000
##
##
## $`1972-07-08`
## , , T+1
##
##               d_Spread   d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.1722992 -0.06525536
## d_EURUSD   -0.17229919  1.0000000 -0.03081500
## d_EURIBOR3 -0.06525536 -0.0308150  1.00000000
##
##
## $`1972-07-09`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17157820 -0.06690543
## d_EURUSD   -0.17157820  1.00000000 -0.03123052
## d_EURIBOR3 -0.06690543 -0.03123052  1.00000000
##
##
## $`1972-07-10`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17161307 -0.06685344
## d_EURUSD   -0.17161307  1.00000000 -0.03122162
## d_EURIBOR3 -0.06685344 -0.03122162  1.00000000
##
##
## $`1972-07-11`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17348460 -0.06752643
## d_EURUSD   -0.17348460  1.00000000 -0.03009855
## d_EURIBOR3 -0.06752643 -0.03009855  1.00000000
##
##
## $`1972-07-12`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17373264 -0.06433316
## d_EURUSD   -0.17373264  1.00000000 -0.03074143
## d_EURIBOR3 -0.06433316 -0.03074143  1.00000000
##
##
## $`1972-07-13`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.17319111 -0.06390910
## d_EURUSD   -0.1731911  1.00000000 -0.03075861
## d_EURIBOR3 -0.0639091 -0.03075861  1.00000000
##
##
## $`1972-07-14`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17674452 -0.06456238
## d_EURUSD   -0.17674452  1.00000000 -0.03006821
## d_EURIBOR3 -0.06456238 -0.03006821  1.00000000
##
##
## $`1972-07-15`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17664546 -0.06455006
## d_EURUSD   -0.17664546  1.00000000 -0.02991147
## d_EURIBOR3 -0.06455006 -0.02991147  1.00000000
##
##
## $`1972-07-16`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17622726 -0.06414702
## d_EURUSD   -0.17622726  1.00000000 -0.02985649
## d_EURIBOR3 -0.06414702 -0.02985649  1.00000000
##
##
## $`1972-07-17`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17629136 -0.06391744
## d_EURUSD   -0.17629136  1.00000000 -0.03013838
## d_EURIBOR3 -0.06391744 -0.03013838  1.00000000
##
##
## $`1972-07-18`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17585068 -0.06407977
## d_EURUSD   -0.17585068  1.00000000 -0.03070235
## d_EURIBOR3 -0.06407977 -0.03070235  1.00000000
##
##
## $`1972-07-19`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17676947 -0.06432274
## d_EURUSD   -0.17676947  1.00000000 -0.03052794
## d_EURIBOR3 -0.06432274 -0.03052794  1.00000000
##
##
## $`1972-07-20`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17579126 -0.06579005
## d_EURUSD   -0.17579126  1.00000000 -0.03290306
## d_EURIBOR3 -0.06579005 -0.03290306  1.00000000
##
##
## $`1972-07-21`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17554871 -0.06552675
## d_EURUSD   -0.17554871  1.00000000 -0.03268655
## d_EURIBOR3 -0.06552675 -0.03268655  1.00000000
##
##
## $`1972-07-22`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17565009 -0.06530865
## d_EURUSD   -0.17565009  1.00000000 -0.03288099
## d_EURIBOR3 -0.06530865 -0.03288099  1.00000000
##
##
## $`1972-07-23`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17524521 -0.06538126
## d_EURUSD   -0.17524521  1.00000000 -0.03053037
## d_EURIBOR3 -0.06538126 -0.03053037  1.00000000
##
##
## $`1972-07-24`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17511055 -0.06526002
## d_EURUSD   -0.17511055  1.00000000 -0.03077268
## d_EURIBOR3 -0.06526002 -0.03077268  1.00000000
##
##
## $`1972-07-25`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17407871 -0.06616234
## d_EURUSD   -0.17407871  1.00000000 -0.03611634
## d_EURIBOR3 -0.06616234 -0.03611634  1.00000000
##
##
## $`1972-07-26`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17396024 -0.06615627
## d_EURUSD   -0.17396024  1.00000000 -0.03563892
## d_EURIBOR3 -0.06615627 -0.03563892  1.00000000
##
##
## $`1972-07-27`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17382829 -0.06611406
## d_EURUSD   -0.17382829  1.00000000 -0.03563414
## d_EURIBOR3 -0.06611406 -0.03563414  1.00000000
##
##
## $`1972-07-28`
## , , T+1
```

```
##
##               d_Spread   d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.1750071 -0.06416255
## d_EURUSD   -0.17500707  1.0000000 -0.03617850
## d_EURIBOR3 -0.06416255 -0.0361785  1.00000000
##
##
## $`1972-07-29`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17524954 -0.06404827
## d_EURUSD   -0.17524954  1.00000000 -0.03618927
## d_EURIBOR3 -0.06404827 -0.03618927  1.00000000
##
##
## $`1972-07-30`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17255165 -0.06495618
## d_EURUSD   -0.17255165  1.00000000 -0.04471823
## d_EURIBOR3 -0.06495618 -0.04471823  1.00000000
##
##
## $`1972-07-31`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17184147 -0.06493589
## d_EURUSD   -0.17184147  1.00000000 -0.04474756
## d_EURIBOR3 -0.06493589 -0.04474756  1.00000000
##
##
## $`1972-08-01`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17555252 -0.06602545
## d_EURUSD   -0.17555252  1.00000000 -0.04356752
## d_EURIBOR3 -0.06602545 -0.04356752  1.00000000
##
##
## $`1972-08-02`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17680522 -0.06495263
## d_EURUSD   -0.17680522  1.00000000 -0.04450336
## d_EURIBOR3 -0.06495263 -0.04450336  1.00000000
##
##
## $`1972-08-03`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17685624 -0.06505865
## d_EURUSD   -0.17685624  1.00000000 -0.04434041
## d_EURIBOR3 -0.06505865 -0.04434041  1.00000000
##
##
## $`1972-08-04`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.17686003 -0.06503670
## d_EURUSD   -0.1768600  1.00000000 -0.04427311
## d_EURIBOR3 -0.0650367 -0.04427311  1.00000000
##
##
## $`1972-08-05`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17855191 -0.06706304
## d_EURUSD   -0.17855191  1.00000000 -0.04259039
## d_EURIBOR3 -0.06706304 -0.04259039  1.00000000
##
##
## $`1972-08-06`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17814515 -0.06607997
## d_EURUSD   -0.17814515  1.00000000 -0.04248108
## d_EURIBOR3 -0.06607997 -0.04248108  1.00000000
##
##
## $`1972-08-07`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17860455 -0.06535974
## d_EURUSD   -0.17860455  1.00000000 -0.04393608
## d_EURIBOR3 -0.06535974 -0.04393608  1.00000000
##
##
## $`1972-08-08`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17653507 -0.06301519
## d_EURUSD   -0.17653507  1.00000000 -0.04360752
## d_EURIBOR3 -0.06301519 -0.04360752  1.00000000
##
##
## $`1972-08-09`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17647721 -0.06280285
## d_EURUSD   -0.17647721  1.00000000 -0.04363379
## d_EURIBOR3 -0.06280285 -0.04363379  1.00000000
##
##
## $`1972-08-10`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17566728 -0.06065815
## d_EURUSD   -0.17566728  1.00000000 -0.04243077
## d_EURIBOR3 -0.06065815 -0.04243077  1.00000000
##
##
## $`1972-08-11`
## , , T+1
##
##               d_Spread   d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.1775071 -0.05943269
## d_EURUSD   -0.17750712  1.0000000 -0.04668260
## d_EURIBOR3 -0.05943269 -0.0466826  1.00000000
##
##
## $`1972-08-12`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17759759 -0.05963505
## d_EURUSD   -0.17759759  1.00000000 -0.04647788
## d_EURIBOR3 -0.05963505 -0.04647788  1.00000000
##
##
## $`1972-08-13`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17709319 -0.05949498
## d_EURUSD   -0.17709319  1.00000000 -0.04638566
## d_EURIBOR3 -0.05949498 -0.04638566  1.00000000
##
##
## $`1972-08-14`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17656998 -0.05922894
## d_EURUSD   -0.17656998  1.00000000 -0.04633789
## d_EURIBOR3 -0.05922894 -0.04633789  1.00000000
##
##
## $`1972-08-15`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17743636 -0.05771613
## d_EURUSD   -0.17743636  1.00000000 -0.04969451
## d_EURIBOR3 -0.05771613 -0.04969451  1.00000000
##
##
## $`1972-08-16`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17730725 -0.05769501
## d_EURUSD   -0.17730725  1.00000000 -0.04967043
## d_EURIBOR3 -0.05769501 -0.04967043  1.00000000
##
##
## $`1972-08-17`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.17508117 -0.05592310
## d_EURUSD   -0.1750812  1.00000000 -0.04774807
## d_EURIBOR3 -0.0559231 -0.04774807  1.00000000
##
##
## $`1972-08-18`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17499149 -0.05590377
## d_EURUSD   -0.17499149  1.00000000 -0.04772005
## d_EURIBOR3 -0.05590377 -0.04772005  1.00000000
##
##
## $`1972-08-19`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17499963 -0.05593142
## d_EURUSD   -0.17499963  1.00000000 -0.04755109
## d_EURIBOR3 -0.05593142 -0.04755109  1.00000000
##
##
## $`1972-08-20`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17502716 -0.05615875
## d_EURUSD   -0.17502716  1.00000000 -0.04687255
## d_EURIBOR3 -0.05615875 -0.04687255  1.00000000
##
##
## $`1972-08-21`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17512211 -0.05736777
## d_EURUSD   -0.17512211  1.00000000 -0.04612601
## d_EURIBOR3 -0.05736777 -0.04612601  1.00000000
##
##
## $`1972-08-22`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17514502 -0.05701428
## d_EURUSD   -0.17514502  1.00000000 -0.04712448
## d_EURIBOR3 -0.05701428 -0.04712448  1.00000000
##
##
## $`1972-08-23`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17568739 -0.05635148
## d_EURUSD   -0.17568739  1.00000000 -0.04739251
## d_EURIBOR3 -0.05635148 -0.04739251  1.00000000
##
##
## $`1972-08-24`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17492584 -0.05696797
## d_EURUSD   -0.17492584  1.00000000 -0.04770837
## d_EURIBOR3 -0.05696797 -0.04770837  1.00000000
##
##
## $`1972-08-25`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17616311 -0.05732427
## d_EURUSD   -0.17616311  1.00000000 -0.04714315
## d_EURIBOR3 -0.05732427 -0.04714315  1.00000000
##
##
## $`1972-08-26`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.17592318 -0.05724960
## d_EURUSD   -0.1759232  1.00000000 -0.04705248
## d_EURIBOR3 -0.0572496 -0.04705248  1.00000000
##
##
## $`1972-08-27`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17423642 -0.05629381
## d_EURUSD   -0.17423642  1.00000000 -0.04580033
## d_EURIBOR3 -0.05629381 -0.04580033  1.00000000
##
##
## $`1972-08-28`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17466662 -0.05752255
## d_EURUSD   -0.17466662  1.00000000 -0.04518481
## d_EURIBOR3 -0.05752255 -0.04518481  1.00000000
##
##
## $`1972-08-29`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17584833 -0.05741181
## d_EURUSD   -0.17584833  1.00000000 -0.04521634
## d_EURIBOR3 -0.05741181 -0.04521634  1.00000000
##
##
## $`1972-08-30`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.17418370 -0.06298940
## d_EURUSD   -0.1741837  1.00000000 -0.04723532
## d_EURIBOR3 -0.0629894 -0.04723532  1.00000000
##
##
## $`1972-08-31`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17372969 -0.06092283
## d_EURUSD   -0.17372969  1.00000000 -0.04719189
## d_EURIBOR3 -0.06092283 -0.04719189  1.00000000
##
##
## $`1972-09-01`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17274186 -0.06125329
## d_EURUSD   -0.17274186  1.00000000 -0.04772269
## d_EURIBOR3 -0.06125329 -0.04772269  1.00000000
##
##
## $`1972-09-02`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17338580 -0.05922409
## d_EURUSD   -0.17338580  1.00000000 -0.04834448
## d_EURIBOR3 -0.05922409 -0.04834448  1.00000000
##
##
## $`1972-09-03`
## , , T+1
##
##              d_Spread   d_EURUSD d_EURIBOR3
## d_Spread    1.0000000 -0.1728168 -0.0591801
## d_EURUSD   -0.1728168  1.0000000 -0.0477168
## d_EURIBOR3 -0.0591801 -0.0477168  1.0000000
##
##
## $`1972-09-04`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17306198 -0.05961014
## d_EURUSD   -0.17306198  1.00000000 -0.04620555
## d_EURIBOR3 -0.05961014 -0.04620555  1.00000000
##
##
## $`1972-09-05`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17295745 -0.05956204
## d_EURUSD   -0.17295745  1.00000000 -0.04626953
## d_EURIBOR3 -0.05956204 -0.04626953  1.00000000
##
##
## $`1972-09-06`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.17373870 -0.06005810
## d_EURUSD   -0.1737387  1.00000000 -0.04545517
## d_EURIBOR3 -0.0600581 -0.04545517  1.00000000
##
##
## $`1972-09-07`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17253646 -0.05715654
## d_EURUSD   -0.17253646  1.00000000 -0.04322532
## d_EURIBOR3 -0.05715654 -0.04322532  1.00000000
##
##
## $`1972-09-08`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17411979 -0.05818879
## d_EURUSD   -0.17411979  1.00000000 -0.04069613
## d_EURIBOR3 -0.05818879 -0.04069613  1.00000000
##
##
## $`1972-09-09`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17445151 -0.06166302
## d_EURUSD   -0.17445151  1.00000000 -0.04014739
## d_EURIBOR3 -0.06166302 -0.04014739  1.00000000
##
##
## $`1972-09-10`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17729495 -0.06190481
## d_EURUSD   -0.17729495  1.00000000 -0.03917844
## d_EURIBOR3 -0.06190481 -0.03917844  1.00000000
##
##
## $`1972-09-11`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.17278151 -0.05944210
## d_EURUSD   -0.1727815  1.00000000 -0.03245344
## d_EURIBOR3 -0.0594421 -0.03245344  1.00000000
##
##
## $`1972-09-12`
## , , T+1
##
##              d_Spread   d_EURUSD d_EURIBOR3
## d_Spread    1.0000000 -0.1707573 -0.0592182
## d_EURUSD   -0.1707573  1.0000000 -0.0292149
## d_EURIBOR3 -0.0592182 -0.0292149  1.0000000
##
##
## $`1972-09-13`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17037717 -0.05920757
## d_EURUSD   -0.17037717  1.00000000 -0.02924736
## d_EURIBOR3 -0.05920757 -0.02924736  1.00000000
##
##
## $`1972-09-14`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16985030 -0.05888736
## d_EURUSD   -0.16985030  1.00000000 -0.02857254
## d_EURIBOR3 -0.05888736 -0.02857254  1.00000000
##
##
## $`1972-09-15`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16969703 -0.05797355
## d_EURUSD   -0.16969703  1.00000000 -0.02858961
## d_EURIBOR3 -0.05797355 -0.02858961  1.00000000
##
##
## $`1972-09-16`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17054099 -0.05794871
## d_EURUSD   -0.17054099  1.00000000 -0.02851674
## d_EURIBOR3 -0.05794871 -0.02851674  1.00000000
##
##
## $`1972-09-17`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17010876 -0.05674939
## d_EURUSD   -0.17010876  1.00000000 -0.02659704
## d_EURIBOR3 -0.05674939 -0.02659704  1.00000000
##
##
## $`1972-09-18`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.17074642 -0.05963320
## d_EURUSD   -0.1707464  1.00000000 -0.02537158
## d_EURIBOR3 -0.0596332 -0.02537158  1.00000000
##
##
## $`1972-09-19`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17125948 -0.05827149
## d_EURUSD   -0.17125948  1.00000000 -0.02624533
## d_EURIBOR3 -0.05827149 -0.02624533  1.00000000
##
##
## $`1972-09-20`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17202721 -0.05706139
## d_EURUSD   -0.17202721  1.00000000 -0.02706899
## d_EURIBOR3 -0.05706139 -0.02706899  1.00000000
##
##
## $`1972-09-21`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17165463 -0.05760496
## d_EURUSD   -0.17165463  1.00000000 -0.02782747
## d_EURIBOR3 -0.05760496 -0.02782747  1.00000000
##
##
## $`1972-09-22`
## , , T+1
##
##               d_Spread   d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.1711863 -0.05732406
## d_EURUSD   -0.17118630  1.0000000 -0.02738630
## d_EURIBOR3 -0.05732406 -0.0273863  1.00000000
##
##
## $`1972-09-23`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17126315 -0.05753702
## d_EURUSD   -0.17126315  1.00000000 -0.02621729
## d_EURIBOR3 -0.05753702 -0.02621729  1.00000000
##
##
## $`1972-09-24`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17102146 -0.05753289
## d_EURUSD   -0.17102146  1.00000000 -0.02636957
## d_EURIBOR3 -0.05753289 -0.02636957  1.00000000
##
##
## $`1972-09-25`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17058231 -0.05663457
## d_EURUSD   -0.17058231  1.00000000 -0.02606932
## d_EURIBOR3 -0.05663457 -0.02606932  1.00000000
##
##
## $`1972-09-26`
## , , T+1
```

```
##
##               d_Spread   d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.1704178 -0.05658612
## d_EURUSD   -0.17041776  1.0000000 -0.02608830
## d_EURIBOR3 -0.05658612 -0.0260883  1.00000000
##
##
## $`1972-09-27`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17051352 -0.05644844
## d_EURUSD   -0.17051352  1.00000000 -0.02642217
## d_EURIBOR3 -0.05644844 -0.02642217  1.00000000
##
##
## $`1972-09-28`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17033580 -0.05637464
## d_EURUSD   -0.17033580  1.00000000 -0.02755093
## d_EURIBOR3 -0.05637464 -0.02755093  1.00000000
##
##
## $`1972-09-29`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16969017 -0.05781265
## d_EURUSD   -0.16969017  1.00000000 -0.02893563
## d_EURIBOR3 -0.05781265 -0.02893563  1.00000000
##
##
## $`1972-09-30`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16964220 -0.05735784
## d_EURUSD   -0.16964220  1.00000000 -0.02901349
## d_EURIBOR3 -0.05735784 -0.02901349  1.00000000
##
##
## $`1972-10-01`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16815991 -0.05624027
## d_EURUSD   -0.16815991  1.00000000 -0.02869515
## d_EURIBOR3 -0.05624027 -0.02869515  1.00000000
##
##
## $`1972-10-02`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16796028 -0.05616893
## d_EURUSD   -0.16796028  1.00000000 -0.02517485
## d_EURIBOR3 -0.05616893 -0.02517485  1.00000000
##
##
## $`1972-10-03`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.15740037 -0.05693180
## d_EURUSD   -0.1574004  1.00000000 -0.02719814
## d_EURIBOR3 -0.0569318 -0.02719814  1.00000000
##
##
## $`1972-10-04`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.16301297 -0.06409080
## d_EURUSD   -0.1630130  1.00000000 -0.02330352
## d_EURIBOR3 -0.0640908 -0.02330352  1.00000000
##
##
## $`1972-10-05`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16345368 -0.06251754
## d_EURUSD   -0.16345368  1.00000000 -0.02497459
## d_EURIBOR3 -0.06251754 -0.02497459  1.00000000
##
##
## $`1972-10-06`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16337094 -0.06193396
## d_EURUSD   -0.16337094  1.00000000 -0.02774669
## d_EURIBOR3 -0.06193396 -0.02774669  1.00000000
##
##
## $`1972-10-07`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.16221313 -0.06117220
## d_EURUSD   -0.1622131  1.00000000 -0.02776224
## d_EURIBOR3 -0.0611722 -0.02776224  1.00000000
##
##
## $`1972-10-08`
## , , T+1
```

```
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.17173667 -0.05801012
## d_EURUSD   -0.17173667  1.00000000 -0.03375918
## d_EURIBOR3 -0.05801012 -0.03375918  1.00000000
##
##
## $`1972-10-09`
## , , T+1
##
##               d_Spread   d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.1684420 -0.05719956
## d_EURUSD   -0.16844203  1.0000000 -0.03175130
## d_EURIBOR3 -0.05719956 -0.0317513  1.00000000
##
##
## $`1972-10-10`
## , , T+1
##
##              d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.0000000 -0.16779916 -0.05470980
## d_EURUSD   -0.1677992  1.00000000 -0.03170414
## d_EURIBOR3 -0.0547098 -0.03170414  1.00000000
##
##
## $`1972-10-11`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16791420 -0.05455931
## d_EURUSD   -0.16791420  1.00000000 -0.03225098
## d_EURIBOR3 -0.05455931 -0.03225098  1.00000000
##
##
## $`1972-10-12`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16690815 -0.05483016
## d_EURUSD   -0.16690815  1.00000000 -0.03448649
## d_EURIBOR3 -0.05483016 -0.03448649  1.00000000
##
##
## $`1972-10-13`
## , , T+1
##
##               d_Spread    d_EURUSD  d_EURIBOR3
## d_Spread    1.00000000 -0.16650557 -0.05336494
## d_EURUSD   -0.16650557  1.00000000 -0.03321764
## d_EURIBOR3 -0.05336494 -0.03321764  1.00000000
```

```r
# 1-Step Ahead Forecast with VAR
library(vars)
```

```r
# Forecast 1 step ahead (next trading day)
var_forecast <- predict(var_restricted_aic, n.ahead = 1, ci = 0.95)

# Print the forecast
print(var_forecast)
```

```
## $d_Spread
##                      fcst     lower    upper       CI
## d_Spread.fcst -0.1525365 -7.792911 7.487838 7.640375
##
## $d_EURUSD
##                        fcst       lower      upper         CI
## d_EURUSD.fcst 0.0001217093 -0.01014212 0.01038553 0.01026383
##
## $d_EURIBOR3
##                          fcst       lower      upper         CI
## d_EURIBOR3.fcst -0.009696606 -0.05195265 0.03255944 0.04225605
```

```r
last_spread <- tail(spread, 1)
forecast_level <- last_spread + var_forecast$fcst$d_Spread[1, "fcst"]
forecast_level
```

```
##              spread
## 2026-04-24 79.14746
```

Test the model

We now use the test portion of our data, to see the accuracy of our training model

```r
library(vars)
library(xts)
library(ggplot2)

# ==========================
# Test set: rolling 1-step-ahead out-of-sample forecast
# ==========================

train_multi <- train_var[,-1]
test_multi  <- test_var[,-1]
sum(is.na(train_multi))
```

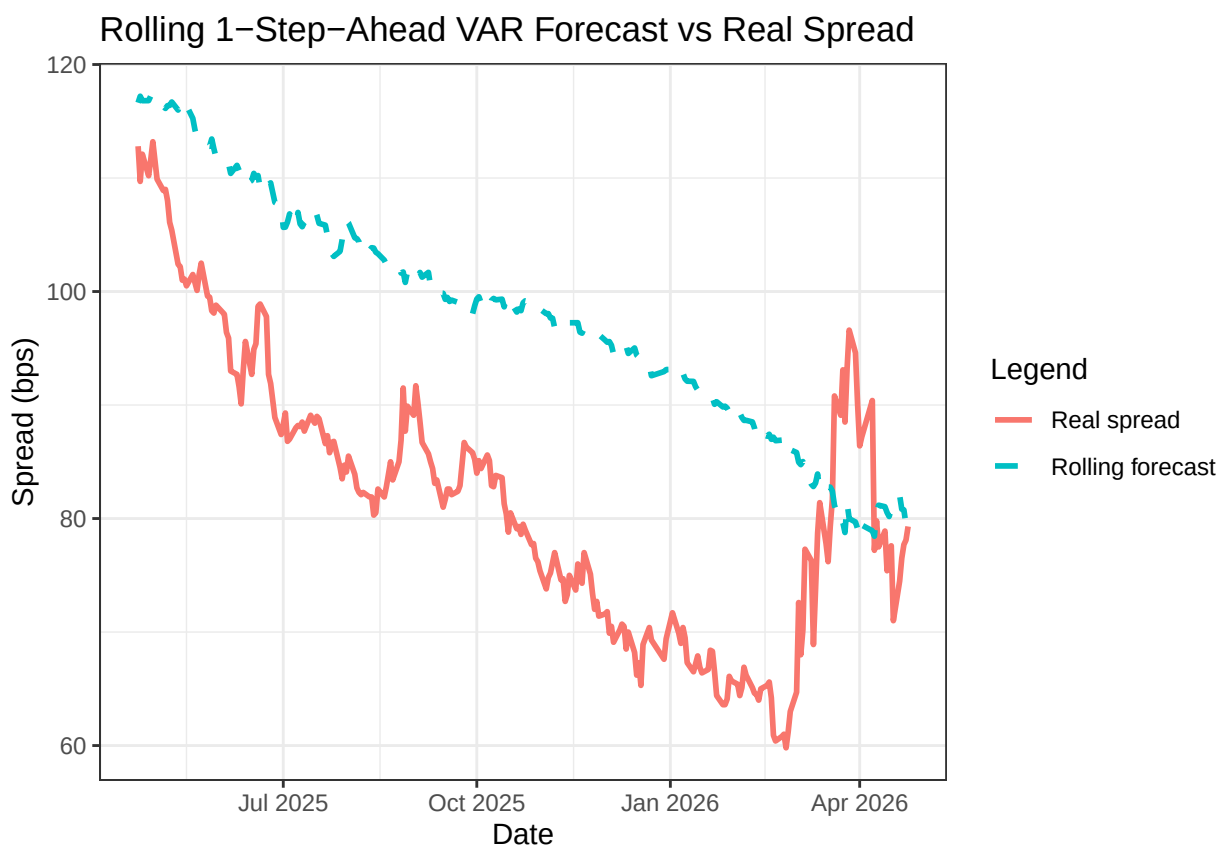
```
## [1] 0
```

```r
# 1. Selezione lag ottimale sul train
lag_sel <- VARselect(train_multi, lag.max = 10, type = "const")
p_opt <- as.numeric(lag_sel$selection["AIC(n)"])

cat("Optimal lag selected on train:", p_opt, "\n")
```

```
## Optimal lag selected on train: 5
```

```r
# 2. Rolling 1-step-ahead forecast sulla serie differenziata "spread"
n_test <- nrow(test_multi)

fc_dspread <- numeric(n_test)
real_dspread <- as.numeric(test_multi[, "d_Spread"])

# Among the candidate rolling window sizes, 400 observations produced the lowest out-of-sample forecast
window_size <- 400


for (i in 1:n_test) {

  if (i == 1) {
    available_data <- train_multi
  } else {
    available_data <- rbind(train_multi, test_multi[1:(i - 1), ])
  }

  if (nrow(available_data) > window_size) {
    current_train <- available_data[(nrow(available_data) - window_size + 1):nrow(available_data), ]
  } else {
    current_train <- available_data
  }

  var_roll <- VAR(current_train, p = p_opt, type = "const")
  pred_1 <- predict(var_roll, n.ahead = 1, ci = 0.95)

  fc_dspread[i] <- as.numeric(pred_1$fcst$d_Spread[1, "fcst"])
}


# 3. Date del test
test_dates <-test_var[,1]

# 4. Ricostruzione dello spread in livelli
# ultimo valore osservato in livelli prima dell'inizio del test
test_start_date <- test_dates[1]
pos_test_start <- which(index(spread) == test_start_date)
last_level_before_test <- as.numeric(spread[pos_test_start - 1])

# forecast in livelli
fc_level <- last_level_before_test + cumsum(fc_dspread)

# valori reali in livelli
real_level <- as.numeric(spread[test_dates])

# 5. Data frame finale
df_roll <- data.frame(
  Date = as.Date(test_dates),
  RealSpread = real_level,
  ForecastSpread = fc_level,
  RealDiff = real_dspread,
  ForecastDiff = fc_dspread
```

```r
)

# 6. Grafico livelli
ggplot(df_roll, aes(x = Date)) +
  geom_line(aes(y = RealSpread, color = "Real spread"), linewidth = 1) +
  geom_line(aes(y = ForecastSpread, color = "Rolling forecast"),
            linewidth = 1, linetype = "dashed") +
  labs(
    title = "Rolling 1-Step-Ahead VAR Forecast vs Real Spread",
    y = "Spread (bps)",
    x = "Date",
    color = "Legend"
  ) +
  theme_bw()
```




```r
# 7. Metriche sui livelli
err_level <- df_roll$RealSpread - df_roll$ForecastSpread

mae_level <- mean(abs(err_level), na.rm = TRUE)
rmse_level <- sqrt(mean(err_level^2, na.rm = TRUE))

cat("MAE levels:", round(mae_level, 4), "\n")
```

```
## MAE levels: 17.2539
```

```r
cat("RMSE levels:", round(rmse_level, 4), "\n")
```

```
## RMSE levels: 18.4631
```

```r
# 8. Benchmark naive
naive_fc_level <- rep(last_level_before_test, length(fc_level))

mae_naive <- mean(abs(real_level - naive_fc_level), na.rm = TRUE)
rmse_naive <- sqrt(mean((real_level - naive_fc_level)^2, na.rm = TRUE))

cat("Naive MAE:", round(mae_naive, 4), "\n")
```

```
## Naive MAE: 35.439
```

```r
cat("Naive RMSE:", round(rmse_naive, 4), "\n")
```

```
## Naive RMSE: 37.4674
```

```r
# 9. Valore finale out-of-sample 1-step-ahead
final_forecast_date <- tail(test_dates, 1)
final_forecast_diff <- tail(fc_dspread, 1)
final_forecast_level <- tail(fc_level, 1)

cat("Final forecast date:", as.character(final_forecast_date), "\n")
```

```
## Final forecast date: 2026-04-24
```

```r
cat("Final out-of-sample 1-step forecast (diff):", round(final_forecast_diff, 4), "\n")
```

```
## Final out-of-sample 1-step forecast (diff): -0.4663
```

```r
cat("Final out-of-sample 1-step forecast (level):", round(final_forecast_level, 4), "\n")
```

```
## Final out-of-sample 1-step forecast (level): 78.9942
```

Window ampia (es. 300): Il modello vede molta storia. I coefficienti sono più stabili perché mediati su un lungo periodo, ma il modello è "lento" a reagire a cambiamenti strutturali recenti (es. un improvviso aumento della volatilità o un cambio di politica monetaria).

Window piccola (es. 150): Il modello è molto reattivo. Si adatta rapidamente agli shock più recenti, ma i coefficienti sono più "nervosi" (instabili) perché basati su pochi dati. Questo può portare a previsioni molto diverse anche per piccoli cambiamenti nei dati.

SPiegare comportamento rol alto mae basso e viceversa